資料結構與攤銷分析

資料結構把抽象操作變成可維護的狀態。先問「哪些狀態合法、操作保證什麼」，再問「一串操作總共花多少工作」。

本章依序學習：**表示與操作規格 → 攤銷分析 → 雜湊 → 搜尋樹與擴充 → B-tree → Fibonacci heap → 整數 universe**。

前置知識為第 1 章的規格與不變量、第 2 章的遞迴分析、第 3 章的機率與 heap。這裡先用兩個 stack 和動態陣列，把後面要用的成本工具建立起來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

操作名稱相同，前提可能不同

抽象資料型態（ADT）描述可觀察行為；陣列、鏈結與樹則是實作。先固定契約，複雜度才有意義。

操作	必須先回答
DELETE	給的是 key、陣列索引，還是現成的節點 handle？
SEARCH	同鍵可否重複？要回傳哪一筆？
POP／DEQUEUE	空結構回傳錯誤、特殊值，還是根本不允許呼叫？
指標更新	節點物件身分要保留嗎？外部 handle 是否仍有效？

雙向鏈結串列刪除已知節點可為常數時間，但「先按 key 找到節點再刪除」通常還要加搜尋成本。後面的每個定理都必須配合它自己的輸入規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀佇列：實體位置不等於邏輯順序

固定容量 $C \geq 1$，使用陣列 $A[0..C-1]$、首筆索引 h 與元素數 s。合法狀態滿足 $0 \leq h < C$、$0 \leq s \leq C$，第 j 筆資料位於

$$Q[j] = A[(h + j) \bmod C], \qquad 0 \leq j < s.$$

空為 $s = 0$，滿為 $s = C$；不靠兩個相等的索引猜測。下一個插入位置是 $(h + s) \bmod C$，但只有未滿時才能寫入。每個實體格都可使用。

陣列中的舊值不等於仍在 queue 的資料。出列後可清除該格以釋放持有的資源，但成員資格由 h, s 決定。固定容量的 ENQUEUE／DEQUEUE 都是最壞 $O(1)$；配置陣列與元素複製成本須另列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先檢查邊界，再更新佇列狀態

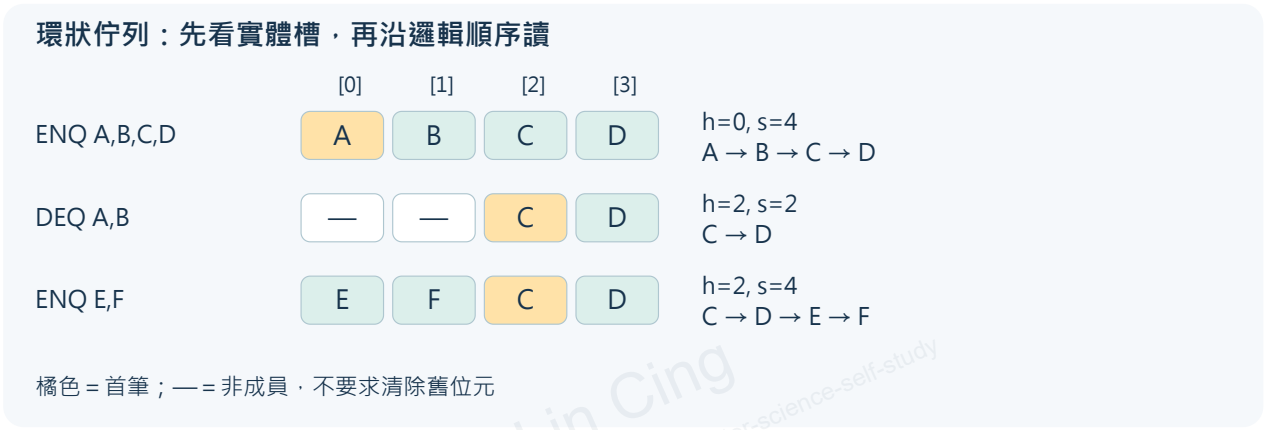
```
ENQUEUE(x):
    if s == C: return overflow
    A[(h+s) mod C] = x
    s = s + 1

DEQUEUE():
    if s == 0: return underflow
    x = A[h]
    h = (h+1) mod C
    s = s - 1
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

假設元素讀寫成功，操作循序執行。失敗呼叫不改 h, s 或任何有效資料；資料值與錯誤狀態用不同回傳型態區分。此處不是多執行緒 lock-free queue，不能省略同步後直接共用。

繞回後，仍然依 C、D、E、F 出列



先放入 A～D，取出 A、B，再新增 E、F。實體為 [E,F,C,D]，但 $h = 2, s = 4$，讀取索引為 $2 \to 3 \to 0 \to 1$。再插入 G 必須拒絕：此時下一個插入位置等於 h，卻是滿態。

環狀索引為何維持 FIFO？

插入前 $s < C$，索引 $(h + s) \bmod C$ 不等於任一有效位置 $(h + j) \bmod C$，其中 $0 \le j < s$：若相等，便有 $C \mid (s - j)$，但 $1 \le s - j < C$，矛盾。因此新資料不覆寫舊資料，恰接在邏輯序列尾端。

出列回傳 $A[h] = Q[0]$。更新後 $h' = (h + 1) \bmod C, s' = s - 1$，對 $0 \le j < s'$ 有

$$A[(h' + j) \bmod C] = A[(h + j + 1) \bmod C] = Q[j + 1].$$

所以剩餘序列剛好去掉首筆，次序不變。初態 $s = 0$ 合法；成功操作維持不變量，失敗操作保持原態，歸納便涵蓋所有合法呼叫序列。

另一種表示法：保留一格，換掉 size

CLRS 第 10.1 節用 head 指首筆、tail 指下一個插入格，保留一格：以本節 0 起算記號，空為 $h = t$，滿為 $(t+1) \bmod C = h$，可存 $C-1$ 筆。

另一常見慣例以 f 指首筆之前、r 指末筆，空為 $f = r$、滿為 $(r+1) \bmod C = f$。兩者換算為 $h = (f+1) \bmod C$、$t = (r+1) \bmod C$。不能只把變數名稱替換，卻保留相同讀寫順序。

本章採 h, s，可存滿 C 格；需要時可計算 $t = (h+s) \bmod C$。此時 $t = h$ 同時可能是空或滿，必須查看 s。保留一格、額外計數、full 旗標是不同設計，不能混用判斷式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單向鏈結插入：先保留原本的後繼

給定有效節點 p，把尚未加入串列的新節點 x 插在它之後。節點身分固定，key 可重複；x 不得已在同一串列裡。

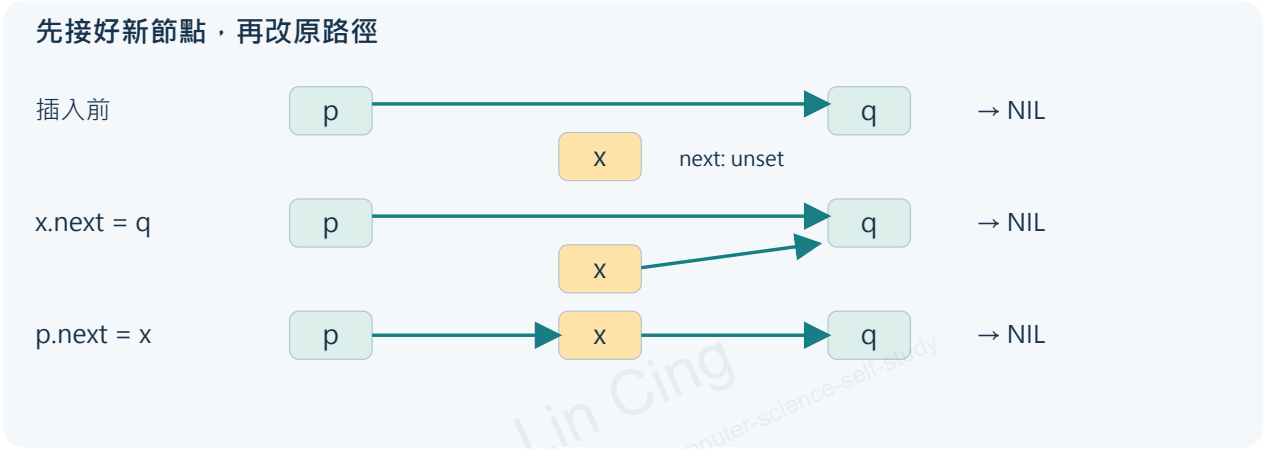
```
q = p.next
x.next = q
p.next = x
```

原路徑 $p \to q$ 變成 $p \to x \to q$，其餘 next 不變，元素只增加 x。若 q 為 NIL，同樣適用；若另存 tail，須把原 tail=p 的情況更新為 tail=x。

若先寫 `p.next=x`，再寫 `x.next=p.next`，會得到 x 指向自己的迴圈，原 q 也可能失去可達性。$O(1)$ 指的是已知 p 的改接；尋找 p、配置 x 及複製非定額資料另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三個狀態看清楚：新增節點如何接回原路徑



先讓 x.next 接到 q，再讓 p.next 接到 x。中間態中，x 還不是由 head 可達的成員；完成後才成為串列的一部分。圖中的節點代號是身分，不以 key 值取代。

單向刪除：已知節點不等於已知前驅

刪除 p 後面的節點時，先存 `x=p.next`；若 x=NIL 即無後繼可刪。否則存 `q=x.next`，令 `p.next=q`。若 x 是 tail，令 tail=p；最後才回收 x。只給 x 而不給前驅，刪除尾節點一般需從 head 找前驅，最壞 $\Theta(n)$。

把「把下一筆資料複製到 x，再刪下一筆」當作常數刪除，會改變 x 的資料身分，也不適用尾節點；不符合這裡保留節點身分的規格。

原地反轉：先保留尚未處理的後綴

前提是以 NIL 結束、無環的單向串列：

```
prev = NIL; cur = head
while cur != NIL:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
head = prev
```

若另存 tail，先記 old_head，最後令 tail=old_head。迴圈中 prev 是已處理前綴的反向串列，cur 是尚未處理後綴的首筆；保存 nxt 才能在反向改接後繼續前進。每步少一個未處理節點，故時間 $\Theta(n)$、額外空間 $O(1)$。

雙向環狀串列：sentinel 把端點接起來

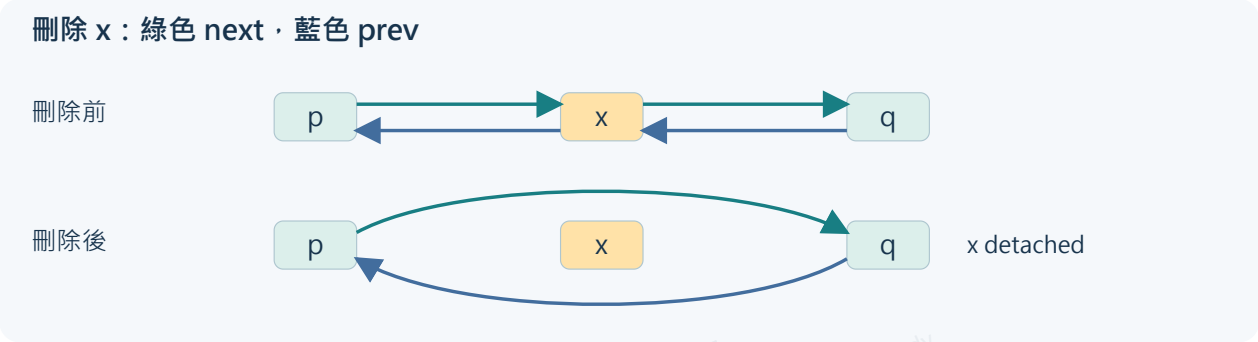
配置固定的虛擬節點 z（sentinel），不算資料元素。空串列滿足 `z.next=z.prev=z`；非空時 z.next 是首筆、z.prev 是末筆。

對每個在環中的節點 v，包含 z，維持 `v.next.prev=v` 與 `v.prev.next=v`。只接受本串列中的有效 handle，禁止刪除 z。檢查陌生 handle 是否屬於本串列的成本，不包含在已知成員的常數改接內。

```
DELETE(x):                  // x != z, valid member
    p = x.prev; q = x.next
    p.next = q
    q.prev = p
```

p 或 q 可以是 z，故同一段涵蓋首筆、末筆與唯一資料節點。改接完成才回收 x；外部持有 x 的 handle 隨後失效。這份規格採循序操作，中間指標狀態不可直接視為可供並行讀取。

雙向刪除：改兩條鄰居連線，保留其餘路徑



原本 p.next=x、q.prev=x，改成 p.next=q、q.prev=p。其餘成員的相鄰關係不變，x 從環中移除；x 的舊欄位本身不代表它仍是成員。

只有一筆資料時 p=q=z，兩次寫入分別使 z.next=z、z.prev=z，正好回到空態。不要把「兩個指標寫入」誤解為「刪除任意 key 都只花兩步」：先找 key 仍可能掃過整條串列。

雙向插入：四個連結都要對得上

把 detached 節點 x 插到已知 p 之後；p 可以是 z。先存 q=p.next：

```
x.prev = p
x.next = q
q.prev = x
p.next = x
```

完成後 p↔x↔q，其餘連結不變。原本空時 p=q=z，z.next、z.prev 都會變為 x。若 p 是末筆，q=z，所以 z.prev 自動更新為新末筆。

每條新的雙向相鄰關係都有對應回鏈，故恢復不變量。這裡的寫入順序只保證循序呼叫完成後正確；不提供 concurrent publication 保證。sentinel 簡化邊界，但每個串列多占一個節點，並不改變搜尋的漸近界。

用陣列索引表示節點：位置和順序分開

不用原生 pointer，也能用平行陣列 key、next、prev 表示節點；同一索引的三個欄位屬於同一物件。索引 0 保留為 NIL，合法物件使用 1～5。

索引	狀態	key	next	prev
1	free	—	4	—
2	live	B	3	5
3	live	C	0	2
4	free	—	0	—
5	live	A	2	0

head=5，依 next 得 $5 \to 2 \to 3 \to 0$，所以資料順序是 A→B→C；free=1，空格鏈為 $1 \to 4 \to 0$。實體相鄰的索引不是邏輯相鄰的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

配置與回收：live list 和 free list 不可重疊

固定物件池的槽位分成互斥的三類：live list、free list、呼叫者持有。ALLOCATE 把 free 槽交給呼叫者，初始化並接入後才變成 live。free list 只用 next 欄位，採 stack 方式管理。

```
ALLOCATE():
    if free == 0: return exhausted
    x = free; free = next[x]
    return x                    // initialize before linking

FREE(x):                        // detached, not already free
    next[x] = free
    free = x
```

配置與回收各改常數個欄位，所以在固定物件池內為 $O(1)$；不包含擴充池或配置不定長資料的成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

物件池數例：從配置到回收

沿用前面的五個槽，head 始終為 5：

完成的操作	live 鏈（到 NIL=0）	free 鏈	呼叫者持有
起態	5(A)→2(B)→3(C)→0	1→4→0	無
ALLOCATE 得 1	5→2→3→0	4→0	1
D 接在 C 之後	5→2→3→1(D)→0	4→0	無
刪除並 FREE(2)	5→3→1→0	2→4→0	無

接入 D：key[1]=D、prev[1]=3、next[1]=0、next[3]=1。刪 B：先改 next[5]=3、prev[3]=5，再令 next[2]=4、free=2。

重複 FREE 會破壞 free list；舊 handle 也可能誤指重用後的新物件。索引在範圍內並不足以證明生命週期有效。

兩個 stack 如何保持 FIFO

用 S 接收新資料，用 T 提供下一筆出列資料。下列序列記號依指定方向讀取：

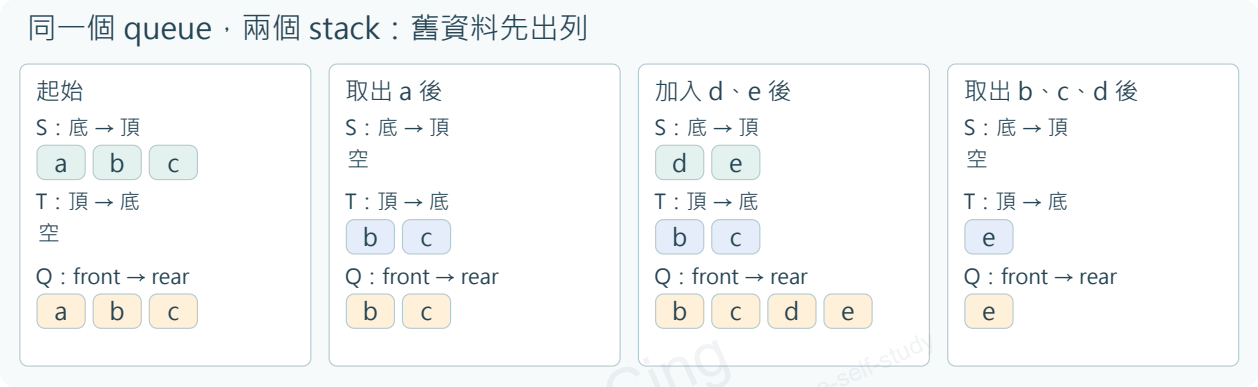
$$Q = \mathrm{reverse}(T_{\mathrm{bottom}\to\mathrm{top}}) \parallel S_{\mathrm{bottom}\to\mathrm{top}}.$$

$\parallel$ 表示串接。因此 **T 的 top 是 queue 的 front**；只有 T 空時，才把 S 全部倒進 T。

ENQUEUE 把新資料 push 到 S，等於接在 Q 尾端。若 T 非空，DEQUEUE pop 它的 top，正好移除 Q 首項。若 T 空，倒轉 S 後，最早進入的資料成為 T 的 top；順序仍等於原來的 Q。

兩者皆空時回傳 underflow，狀態不變。**不能每次出列都倒轉 S**：T 尚有舊資料時，新資料必須等在它們後面。

先看資料順序，再看昂貴的搬移



圖中的 S 由底讀到頂，T 由頂讀到底。加入 d, e 時，b, c 仍在 T 等待，所以不用搬動。只有 b, c 都已出列，才倒轉 S，接著取出 d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

昂貴一次，不等於每次都昂貴

以一次 stack PUSH 或 POP 為 1 單位；暫不計每次 queue 呼叫固定的判斷開銷。ENQUEUE 成本 1。若 DEQUEUE 需搬移 k 筆，成本是 $2k + 1$：每筆先 pop S 再 push T，最後 pop T。

從空 queue 出發，每筆資料至多經過四步：**進 S → 離開 S → 進 T → 離開 T**。所以 e 次入列、d 次成功出列的總成本至多 $3e + d$。

任意長度 m 的合法操作序列都有 $e + d \le m$，故總成本至多 $3m$；補回每次固定判斷後仍為 $O(m)$。這個結論沒有假設操作隨機，也不需要「大多數情況便宜」。

單次 DEQUEUE 仍可能是 $\Theta(n)$。攤銷界是整段序列的保證，不能用來宣稱每次呼叫有相同延遲。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

勢能法：把延後的工作記在狀態上

第 i 次操作把狀態 D_{i-1} 變成 D_i，實際成本為 c_i。選一個只依狀態決定的函數 Φ，定義攤銷成本

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

$\Delta\Phi > 0$ 表示本次多收一點，為以後存下額度；$\Delta\Phi < 0$ 表示用先前累積的額度支付工作。它不是另一種實際執行時間。

把前 m 次相加，中間勢能成對抵消：

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

若所有可達狀態都滿足 $\Phi(D_i) \geq \Phi(D_0)$，則總實際成本不超過總攤銷成本。常用的充分條件是 $\Phi(D_0) = 0$ 且 $\Phi \geq 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

初始狀態與負攤銷費用，都不能略過

若只證明 $\Phi \geq 0$，卻沒有 $\Phi(D_0) = 0$，正確結論是

$$\sum c_i \leq \sum \hat{c}_i + \Phi(D_0).$$

例如 queue 一開始已有 n 筆資料在 S，一次出列就可能搬移 n 筆。不能把這些初始資料當成之前已付費；必須計入建立初態的成本，或保留 $O(n)$ 初始項。

個別 $\hat{c}_i$ 可以為負。只要整體勢能條件成立，望遠鏡等式仍正確；負值表示這次釋放的信用比實際工作還多。

攤銷（amortized）針對所有合法序列給總界；**期望**（expected）必須指定亂數或輸入分布。同一資料結構也可能同時有「期望攤銷」界，兩層量詞不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩個 stack 的勢能，只計尚未搬過的資料

令 $\Phi(S, T) = 2|S|$，初態兩個 stack 皆空，所以 $\Phi_0 = 0$。

操作	實際成本 c	$\Delta\Phi$	$\hat{c}$
ENQUEUE	1	2	3
T 非空時 DEQUEUE	1	0	1
T 空，搬移 $k \geq 1$ 筆後出列	$2k + 1$	$-2k$	1
空 queue 回報 underflow	0	0	0

最後一列僅計 stack primitives；若計呼叫開銷，四列各加固定常數。T 中的資料已支付搬移費，不再需要額外勢能。

每個前綴都滿足總成本 $\leq 3e + d$。這與逐筆資料最多做四次 primitive 的 aggregate 證明相符。

交錯操作的完整帳目

兩個 stack 均以**底→頂**列出；$\Phi = 2|S|$。起終勢能皆為 0，兩種成本均合計 19。

操作	S	T	c	Φ	$\hat{c}$
ENQ a	a	空	1	2	3
ENQ b	a,b	空	1	4	3
ENQ c	a,b,c	空	1	6	3
DEQ → a	空	c,b	7	0	1
ENQ d	d	c,b	1	2	3
ENQ e	d,e	c,b	1	4	3
DEQ → b	d,e	c	1	4	1
DEQ → c	d,e	空	1	4	1
DEQ → d	空	e	5	0	1

計費法：每個前綴都要付得起

第 i 次操作實付 c_i，分析時收取 a_i，餘額為

$$B_t = B_0 + \sum_{i=1}^{t} (a_i - c_i).$$

從空結構開始，取 $B_0 = 0$。若能對任意合法序列的每個前綴證明 $B_t \geq 0$，便得到 $\sum_{i=1}^{t} c_i \leq \sum_{i=1}^{t} a_i$。

不能先透支，再承諾未來補回：呼叫者可能在透支當下停止，該前綴的界就已失效。若初態自帶信用 $B_0 > 0$，必須把它列入總成本界，不能當成免費工作。

信用可附在元素上，方便證明它不會被重複花用；若總信用只由目前狀態決定，取 $\Phi(D) = B(D)$，就得到前面的勢能法。

MULTIPOP：大量刪除的成本由誰支付？

空 stack 支援 PUSH、POP 與 MULTIPOP(k)，其中 $k \geq 0$；後者最多移除 k 個元素，遇空即停。每次成功 push／pop 的基本成本為 1。

PUSH 收 2：付當次成本 1，另存 1 在新元素上。POP 與 MULTIPOP 收 0，用被移除元素的信用付款。**每個仍在 stack 的元素恰有 1 單位信用**，由歸納可知餘額從不為負。

若 m 次呼叫內有 p 次 PUSH，成功移除總數 $q \leq p$，所以實付 $p + q \leq 2p \leq 2m$。同一物件再次 push 是新一次插入，必須重新付費。

此處只數成功的元素操作。若每次 API 呼叫另收固定的檢查／派送成本 1，總界改為 $m + p + q \leq 3m$；因此空 MULTIPOP 不代表程式完全不用時間。

Binary counter：先固定寬度與溢位規則

使用固定 $k \geq 1$ 個 bit，$A[0]$ 是最低位，值為 $\sum_{j=0}^{k-1} A[j]2^j$。初值全 0；INCREMENT 計算模 2^k 的加一，故全 1 之後回到全 0。

```
INCREMENT(A, k):
    j = 0
    while j < k and A[j] == 1:
        A[j] = 0
        j = j + 1
    if j < k:
        A[j] = 1
```

成本 c_i 只計被翻轉的 bit 數。測試與迴圈控制的實際時間為 $\Theta(1 + c_i)$，因此另加每次常數不改變總時間的漸近界。不要把增加位寬的任意精度整數，直接當成這份規格。

進位不是同時改掉所有 bit

進位：只翻轉尾端 1 與下一個 0

橘色 = 本次翻轉；bit 由高位到低位排列



0111→1000 雖翻轉四個 bit，卻同時用掉先前存下的勢能。下一頁把一般進位與溢位分開計算。

一般進位與溢位，分開計算

一般進位若遇到 t 個尾端 1，先把它們清零，再把下一個 0 設為 1：$c = t + 1$。取勢能 $\Phi =$ 目前 1 的個數，便有 $\Delta\Phi = 1 - t$、$\hat{c} = 2$。

溢位是獨立分支：$t = k$，全部清零而沒有新的 1，故 $c = k$、$\Delta\Phi = -k$、$\hat{c} = 0$。不能在這個分支仍寫 $t + 1$ 次翻轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一串加一，三種分析得到同一個界

從全 0 開始做 $m \geq 1$ 次加一。第 j 個 bit 每 2^j 次操作翻轉一次，包含歸零的溢位；故總翻轉數恰為

$$T(m, k) = \sum_{j=0}^{k-1} \left\lfloor \frac{m}{2^j} \right\rfloor < 2m.$$

聚合分析直接加總各 bit 的次數。**計費法**每次收 2；把 0 設為 1 時，付 1 並留 1 給未來清零，每個 1 都帶有自己的信用。**勢能法**以 1 的個數記錄同一筆信用，普通加一收 2、溢位收 0。

例如 $k = 4, m = 8$：各 bit 翻轉 $8, 4, 2, 1$ 次，共 15；不是 $8 \times 4 = 32$。一般證明涵蓋任意 m，這個數例只用來核對計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整帳目：從 0000 到溢位歸零

四位元，起態 $\Phi_0 = 0$；bit 字串依最高位到最低位書寫。前 8 次攤銷總額 16，扣除終態勢能 1，實付 15。

第幾次	操作後狀態	c_i	Φ_i	累積實付
1	0001	1	1	1
2	0010	2	1	3
3	0011	1	2	4
4	0100	3	1	7
5	0101	1	2	8
6	0110	2	2	10
7	0111	1	3	11
8	1000	4	1	15

完整帳目：最高位也要付清

接續同一串操作。第 9～15 次各收 2，第 16 次溢位收 0。全程實付與攤銷總額均為 30，終態沒有剩餘信用。

第幾次	操作後狀態	c_i	Φ_i	累積實付
9	1001	1	2	16
10	1010	2	2	18
11	1011	1	3	19
12	1100	3	2	22
13	1101	1	3	23
14	1110	2	3	25
15	1111	1	4	26
16	0000	4	0	30

哪些改動會破壞常數攤銷界？

非零初態：若從 k 個 1 開始，第一次就花 k。原勢能仍可用，但只能推出 $\sum c_i \le 2m + \Phi_0 \le 2m + k$；不能省掉初始信用。

加入 DECREMENT：對 $k \ge 2$，先從 0 加到 $2^{k-1} - 1$，再反覆交替 INCREMENT、DECREMENT。狀態在 $0\underbrace{11\cdots1}_{k-1}$ 與 $1\underbrace{00\cdots0}_{k-1}$ 間往返，每次都翻轉 k 個 bit。

令準備階段長度 $r = 2^{k-1} - 1$，再做 $2r$ 次交替，總呼叫 $3r$、僅交替部分已花 $2rk$，因此可達 $\Omega(mk)$。每次至多翻 k 個 bit，亦有 $O(mk)$；這個壞序列的成本為 $\Theta(mk)$。

問題在於同一批昂貴進位可以立即被借位反轉，先前「每個 1 預付一次清零」的信用不再足夠。操作集合改了，必須重新證明成本界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態陣列：先把搬移成本算清楚

狀態為 (n, M)：n 是有效元素數，M 是容量。從 $(0, 0)$ 開始；首次插入配置 1 格，滿表時先把容量倍增，再加入新資料。

成本只計**寫入新資料 1 次，加上搬移的舊資料數**。配置、釋放與每次判斷的固定開銷暫略。若配置會初始化每一格，必須另計；漸近結論仍可成立，精確常數會改變。

第幾次插入	插入後 (n, M)	本次成本	累積成本
1	(1,1)	1	1
2	(2,2)	2	3
3	(3,4)	3	6
4	(4,4)	1	7
5	(5,8)	5	12

五次插入共搬移 $1 + 2 + 4 = 7$ 筆，**搬移量不一定小於插入次數**。要證明線性總成本，須用完整幾何級數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

只擴張時，aggregate 與勢能給同一總界

插入 $m \geq 2$ 筆，令 L 為小於 m 的最大 2 冪。搬移總數為 $1 + 2 + \cdots + L = 2L - 1$，所以 $T(m) = m + 2L - 1 < 3m$。$m = 0, 1$ 分別有成本 0、1。

勢能取 $\Phi(0, 0) = 0$，非空表取 $\Phi(n, M) = 2n - M$。僅插入的可達狀態滿足 $M/2 < n \leq M$，故勢能非負。

分支	c	$\Delta\Phi$	$\hat{c}$
首次插入 $(0, 0) \to (1, 1)$	1	1	2
未滿，不搬移	1	2	3
滿表 $(M, M) \to (M + 1, 2M)$	$M + 1$	$2 - M$	3

因此 $T(m) \leq 3m$。**首次插入的攤銷成本是 2**；不能把含 $\log 0$ 的公式直接套用初態。

縮表若太積極，就會反覆搬同一批資料

考慮錯誤策略：滿表時倍增；刪除後使用率一到 $1/2$ 就減半。

取 $k \geq 2$ 為 2 的冪，從滿表 (k, k) 插入一筆，擴成 $(k + 1, 2k)$，搬移 k 筆。再刪除這筆，縮回 (k, k)，又搬移 k 筆。之後反覆插入、刪除，每對操作搬移 $2k$ 筆，成本恰為 $2k + 2$。

建好初態需 $O(k)$；接著做 $2r$ 次交替操作，總成本為 $\Omega(kr)$。取 $r = k$，操作次數 $O(k)$，成本卻為 $\Omega(k^2)$。

修正方法是把兩個門檻分開：**滿表才擴張，刪除後至多四分之一滿才縮表**。縮完回到半滿，下一次反向調整前有足夠操作累積信用。

擴縮規格：門檻、空表與資料順序

本例只教 **APPEND 與 DELETE-LAST**。刪除中間元素若要搬動後綴，必須另計該成本。

- 空容量首次 APPEND 配置 1 格；滿表時容量乘 2，再寫入新資料。
- DELETE-LAST 先刪最後一筆。若 $M \geq 2$ 且刪後 $n \leq M/4$，容量減半，搬移剩下的 n 筆；否則容量不變。
- 容量是 2 的冪；用過後最小保留容量 1。空結構刪除回報 underflow，狀態不變。
- 每次成功操作成本 $1+$ 搬移筆數。搬移保持所有存活資料的順序；外部 raw pointer 可能因重新配置失效。

門檻慣例：本節採 $n \leq M/4$；原書 §17.4 採 $n < M/4$，在 $M \geq 4$ 時會晚一次刪除才縮表。兩者皆可有常數攤銷界，但不能共用同一張逐步帳目。

在這個規格下，一次操作最多觸發一次 resize。對 $M \geq 4$，刪除觸發縮表時恰有 $n = M/4$；因前一個狀態尚未觸發門檻，而且每次只刪一筆。

同一批資料，追蹤到縮表完成

擴縮門檻分開，才能在下一次搬移前存足信用

			成本
滿表	n=4, M=4	a b c d	成本 —
APPEND e	n=5, M=8	a b c d e · · ·	成本 5 = 1 + 4
DELETE e	n=4, M=8	a b c d · · · ·	成本 1
DELETE d	n=3, M=8	a b c · · · · ·	成本 1
DELETE c	n=2, M=4	a b · ·	成本 3 = 1 + 2

半滿先保留容量；四分之一滿才縮表，縮完回到半滿。

擴成 8 格後，刪到 4 筆時先保留容量；刪到 2 筆才縮為 4 格。兩個門檻之間的間隔，就是避免 resize 來回震盪的 **hysteresis（遲滯）**。

稠密與稀疏狀態，分別存下搬移信用

對 $M \geq 2$ 定義

$$\Phi(n, M) = \begin{cases} 2n - M, & n \geq M/2, \\ M/2 - n, & n < M/2. \end{cases}$$

另定義 $\Phi(0,0) = \Phi(0,1) = 0$、$\Phi(1,1) = 1$。兩段在半滿時同為 0，所有可達狀態皆非負。

稠密區的勢能隨插入增加，準備支付擴張；稀疏區的勢能隨刪除增加，準備支付縮表。這不是把同一筆信用算兩次：每個狀態只用其中一段。

下一頁逐一核對分支。$M = 1, 2$ 的空表例外另列，避免把 $M/4$ 當成一定是可達整數元素數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般容量的四種分支，都有常數攤銷界

下表 M 是操作前容量。未 resize 的分支若跨過半滿邊界，使用兩段公式直接代入。

操作情形	c	$\Delta\Phi$	$\hat{c}$
不擴張的插入	1	-1 或 2	0 或 3
滿表擴張	$M + 1$	$2 - M$	3
不縮表的刪除	1	-2 或 1	-1 或 2
$M \geq 4$，刪至 $M/4$ 後縮表	$1 + M/4$	$1 - M/4$	2

最後一列：刪前 $n = M/4 + 1$，勢能 $M/4 - 1$；刪後容量 $M/2$、元素數 $M/4$，正好半滿，勢能 0。

例 $M = 8$：$(3, 8) \to (2, 4)$，搬移 2 筆，$c = 3$；勢能由 1 降為 0，所以 $\hat{c} = 2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小容量的例外與總成本結論

一般表格之外，再核對以下合法邊界：

狀態變化	c	$\Delta\Phi$	$\hat{c}$
首次 $(0,0) \to (1,1)$	1	1	2
重用空格 $(0,1) \to (1,1)$	1	1	2
刪空 $(1,1) \to (0,1)$	1	-1	0
最小縮表 $(1,2) \to (0,1)$	1	0	1

所有成功操作都有 $\hat{c} \le 3$，起態勢能 0、終態勢能非負。因此任意 m 次成功 APPEND／DELETE-LAST 的實際總成本 $\le 3m$。

若把 underflow 呼叫的判斷成本計為 1，其勢能不變、攤銷成本也為 1；總成本仍線性。這個界不適用於任意位置刪除、任意縮容政策，或沒有列入模型的非定額搬移成本。

Dictionary：先決定同鍵插入的意義

本節維護 key→record 的對應，每個 key 至多一筆；INSERT 遇到既有 key 時更新 value，DELETE 接受 key，找不到則回報不存在。key 的相等判斷與 hash 值相等是兩回事。

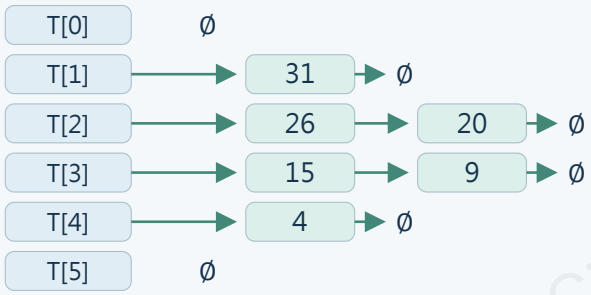
若 universe 為 $\{0, \dots, M-1\}$，直接定址表以 key 當索引，三種操作最壞 $O(1)$，空間及一般的全表初始化成本為 $\Theta(M)$。當實際資料只有 $n \ll M$ 筆，才希望把表縮到與 n 相近的大小。

雜湊表使用 $h : U \to \{0, \dots, m-1\}$。$|U| > m$ 時，鴿籠原理保證任何固定 h 都有碰撞；仍須保留完整 key 供核對。以下假設 key、hash 計算與相等比較可在常數個 word 操作內完成；長字串與任意精度整數須另計讀取與運算成本。

Chaining：同槽資料保留在串列裡

m = 6; h(k) = k mod 6

依序頭插：4, 9, 15, 20, 26, 31



查 20

26 → 20 ✓

查 14

26 → 20 → ø

同桶仍要比完整 key

$m = 6, h(k) = k \bmod 6$，依序頭插 4、9、15、20、26、31。槽 2 為 26→20，槽 3 為 15→9。找 20 檢查兩筆；找不存在的 14 也檢查槽 2 的兩筆。

常數頭插，不代表整個 INSERT 都是常數

令 $\alpha = n/m$，chaining 可有 $\alpha > 1$。搜尋掃描 h(k) 指定的整條串列直到找到 key 或到尾端，最壞 $\Theta(n)$。

已知 key 全新時，頭插的改接是 $O(1)$。但本章 INSERT 具有更新既有鍵的語義，必須先搜尋去重，成本為搜尋加常數改接。DELETE(key) 同樣先搜尋；只有持有有效雙向節點 handle 時，才可直接以 $O(1)$ 解鏈。

CLRS §11.2 的常數時間插入與刪除有上述輸入前提：插入假定物件尚未存在，刪除直接給物件。這與本章 dictionary API 不同，並非兩個成本結論互相矛盾。

失敗搜尋：把每一筆碰撞的成本加起來

先採 simple uniform hashing：固定相異 key 的槽位獨立且均勻分布。固定不存在的 query q，令 $I_x = \mathbf{1}[h(x) = h(q)]$，則

$$L_q = \sum_{x \in K} I_x, \qquad E[L_q] = \sum_{x \in K} \frac{1}{m} = \frac{n}{m} = \alpha.$$

搜尋會檢查這條 chain 的所有記錄，再加常數的 hash 與索引成本，因此期望時間為 $\Theta(1 + \alpha)$。$\alpha = 0$ 時仍要判斷空桶，不是零時間。

線性期望本身不需要碰撞事件互相獨立；上述完整均勻模型是足夠前提，稍後的 universal family 只靠鍵對碰撞上界，就能給出期望時間上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功搜尋：平均的是記錄，不是桶

固定相異鍵的插入順序 $x_1, \ldots, x_n$，採頭插、沒有刪除，且成功 query 等機率選其中一筆。找 x_i 要檢查自己，以及晚於它插入的同槽記錄。

$$E[C_i] = 1 + \frac{n - i}{m}, \qquad E[C] = \frac{1}{n} \sum_{i=1}^{n} E[C_i] = 1 + \frac{n - 1}{2m}.$$

加入 hash 的常數成本後仍為 $\Theta(1 + \alpha)$。較大的桶含更多可被問到的 key，所以成功查詢並不是等機率選桶。

例如六筆表有兩條長度 2 的 chain、兩條長度 1 的 chain；等機率選 key 的檢查總數為 $(1 + 2) + (1 + 2) + 1 + 1 = 8$，平均 $4/3$。這個確定表的平均，不必等於隨機模型的期望 $1 + 5/12$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

除法與字串 hash：公式不等於分布保證

除法法 $h(k) = k \bmod m$ 很直接；若 $m = 2^p$，只保留最低 p 個 bit，例如 0、8 在 m=8 時都到槽 0。選質數可避開某些輸入規律，但不能保證任意 key 集合均勻。

字串可逐字套 Horner：$H_0 = 0$、$H_{i+1} = (BH_i + c_i) \bmod m$。歸納可知結果等於整個 base-B 整數取模，處理長度 ℓ 需 $\Theta(\ell)$ 次定額步驟。

取 B=10、m=7，字元碼序列 [1,2,3] 的中間值是 $1, 5, 4$，對應 $123 \bmod 7 = 4$。若 $B \equiv 1 \pmod m$，結果變成字元碼總和取模，重排字元仍相同；因此正確計算一個公式，不等於它具有低碰撞率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法法：取低 word 裡的高位

$h(k) = \lfloor m \operatorname{frac}(kA) \rfloor$。取 $A = s/2^w, m = 2^p$，則 $r_0 = ks \bmod 2^w$，再取 $h(k) = \lfloor r_0 / 2^{w-p} \rfloor$，其中 $1 \le p \le w$。

寫 $ks = q2^w + r_0$，則 $kA = q + r_0/2^w$，小數部分正是 $r_0/2^w$。乘 2^p 再取 floor，就是把低 word 的最低 w−p 位捨去，留下其中最高 p 位。

下一頁以八位元完整追蹤：w=8、s=159、k=13、p=4，結果應為 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法數例：完整乘積到槽號

w = 8; s = 159; k = 13; p = 4

ks = 2067 = 8 × 256 + 19



$2067 = 8 \cdot 256 + 19$，低 word 是 00010011，取高四位 0001。不是完整乘積的最高四位；運算必須遵守指定的 word 位寬。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Universal family：隨機的是函數選擇

有限函數族 H 稱 universal，若對每對固定相異鍵 x、y，均勻抽取 $h \in H$ 時有

$$\Pr_h[h(x) = h(y)] \le \frac{1}{m}.$$

鍵集合與 query 必須獨立於這次抽樣。這不要求使用者鍵值天生均勻，也不要求所有輸出或碰撞事件互相獨立；更不保證某次抽到的表完全沒有長 chain。

來源慣例：本章採 CLRS §11.3.3 的「不超過」。MIT 2005 Lecture 8 的定義寫「恰為」，並用模質數的 dot-product family 示範。恰為的版本也滿足本章定義，但不能把較弱定義反推成相等。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Universal chaining：完整的期望上界

固定集合 K 與 query q。令 $Y_q = \sum_{x \in K, x \neq q} \mathbf{1}[h(x) = h(q)]$。線性期望與 universal 定義給出

$$E[Y_q] \le \frac{|K \setminus \{q\}|}{m}.$$

q 不存在時，掃描長度期望至多 n/m；q 存在時，包含它的整條 chain 長度期望至多 $1 + (n-1)/m$，搜尋不會比掃完整條更貴。所以任意固定 q 的期望搜尋時間為 $O(1+\alpha)$，不需要成功 query 等機率。

若固定操作序列的同時存在筆數一直是 $O(m)$，更新式 INSERT、DELETE(key) 皆先搜尋，總 t 次操作期望 $O(t)$，另加初始化 $\Theta(m)$。若 query 依賴已觀察到的 h 或先前回應而自適應選擇，這份固定鍵對證明不能直接套用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Affine family：先證明模質數後的對應

取質數 $P > \max U$；均勻獨立抽 $a \in \{1, \dots, P-1\}$、$b \in \{0, \dots, P-1\}$，令

$$h_{a,b}(k) = ((ak + b) \bmod P) \bmod m.$$

對固定 x≠y，記 $r = (ax+b) \bmod P$、$s = (ay+b) \bmod P$。因 $a(x-y) \not\equiv 0 \pmod P$，必有 r≠s。

反過來，每對 r≠s 都唯一決定 $a = (r-s)(x-y)^{-1} \bmod P$、$b = r - ax \bmod P$。所以均勻的 (a,b) 對應均勻的 $P(P-1)$ 個有序相異 (r,s)。P 是質數，才保證非零的 x−y 有乘法逆元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後取 mod m，碰撞機率仍不超過 1/m

固定 r 後，其餘 P−1 個 s 等可能。與 r 同 mod m 的 s 至多 $\lceil P/m \rceil - 1 \le (P-1)/m$ 個，因此

$$\Pr[h(x) = h(y) \mid r] \le \frac{\lceil P/m \rceil - 1}{P-1} \le \frac{1}{m}.$$

再對 r 取平均，便證明 universal。若 m≥P，取 mod m 不會合併相異的 r、s，碰撞機率為 0。

寫 $P = qm + d$、$0 \le d < m$。若 $r \bmod m < d$，該桶含 $q+1$ 個元素，扣除 r 後有 q 個；其餘非空桶含 q 個，扣除 r 後有 $q-1$ 個。因此計數至多為 q；$d = 0$ 時恰為 $q-1$。m 不必是質數，P 才須為質數；模運算成本假設需符合 word model。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MIT 的 dot-product family 為何也成立？

取質數 m，key 為固定長度向量 $x = (x_0, \ldots, x_r) \in \mathbb{F}_m^{r+1}$。均勻抽係數向量 a，定義 $h_a(x) = \sum_i a_i x_i \bmod m$。

x≠y 時，選一個 $x_j - y_j \neq 0$ 的位置。固定其他係數後，碰撞等價於

$$a_j(x_j - y_j) = -\sum_{i \neq j} a_i(x_i - y_i) \pmod m.$$

因差值可逆，m 種 a_j 中恰有一種成立，故機率恰為 1/m。這個族的 table modulus m 必須是質數，與前頁 affine family 的任意 table size 不同；長度可變時，hash 計算成本也隨長度改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Open addressing：三種槽位狀態

每槽直接存一筆 record，所以 n≤m。每個 key 的探測序列 $h(k,0),\ldots,h(k,m-1)$ 固定且可重現；要保證有空槽就能插入，這 m 個位置須是全表排列。

狀態	搜尋動作	刪除的影響
OCCUPIED	比完整 key，相等才成功	可改為 DELETED
DELETED（tombstone）	繼續探測	不是停止點
EMPTY	判定不存在並停止	不能由一般刪除製造

EMPTY 是本次建表後未曾使用的槽。若某 key 曾插到更後面，當時不可能跳過 EMPTY；後續刪除只留下 DELETED，才不會切斷搜尋路徑。重建表則重新定義整張表的狀態。

搜尋與刪除：最多探測 m 次

```
FIND(k):
    for i = 0 .. m-1:
        j = h(k,i)
        if T[j] is EMPTY: return absent
        if T[j] is OCCUPIED and T[j].key == k:
            return j
    return absent
```

DELETE(k) 先 FIND，找到 j 才把 T[j] 改為 DELETED；不存在則保持原態。key 可以是任何合法值，EMPTY／DELETED 必須有獨立的狀態標記，不能與資料值混淆。

迴圈上限不可省：一張有 tombstone、卻沒有 EMPTY 的表，失敗搜尋必須走完 m 槽後返回；不能無限等待空槽。

更新式 INSERT：記住 tombstone，但先查完

```
first = NONE
for i = 0 .. m-1:
    j = h(k,i)
    if T[j] is OCCUPIED and T[j].key == k:
        T[j].value = value; return j
    if T[j] is DELETED and first == NONE: first = j
    if T[j] is EMPTY:
        target = first if first != NONE else j
        T[target] = OCCUPIED(k,value); return target
if first != NONE: T[first] = OCCUPIED(k,value); return first
return full
```

第一個 DELETED 後面仍可能有同 key；立即插入會製造重複記錄。只有遇 EMPTY 或走完全部 m 槽，才能確認不存在，再使用記住的空位。覆蓋全表是此處的探測前提。

Tombstone：保留搜尋路徑，再重用空間

m = 7; home(k) = k mod 7

ø = EMPTY; D = DELETED

	0	1	2	3	4	5	6	
插入 1, 8, 15	ø	1	8	15	ø	ø	ø	探測：1 → 2 → 3
刪除 1	ø	D	8	15	ø	ø	ø	查 15：1 → 2 → 3
更新 8	ø	D	8	15	ø	ø	ø	只更新槽 2 的 value
插入 22	ø	22	8	15	ø	ø	ø	探測到 4，再回填槽 1

遇到 D 繼續查；確認沒有同 key，才重用 D。

$m = 7, h(k,i) = (k \bmod 7 + i) \bmod 7$。插入 1、8、15 時分別占槽 1、2、3。刪 1 後，找 15 仍沿 $1 \to 2 \to 3$，不能在 tombstone 停下。

同一份資料的完整探測帳目

承接前圖。最後仍為三筆；INSERT(22) 在槽 4 確認不存在，才回填槽 1。

呼叫	探測槽	寫入／回傳	探測次數
INSERT(1,a)	1	槽 1 存 (1,a)	1
INSERT(8,b)	1,2	槽 2 存 (8,b)	2
INSERT(15,c)	1,2,3	槽 3 存 (15,c)	3
DELETE(1)	1	槽 1=DELETED	1
FIND(15)	1,2,3	找到槽 3	3
INSERT(8,b′)	1,2	更新槽 2，不新增	2
INSERT(22,d)	1,2,3,4	重用槽 1	4
FIND(29)	1,2,3,4	不存在	4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

探測公式：先確認能否覆蓋全表

Linear probing 用 $(h_0(k)+i) \bmod m$，一定覆蓋 m 槽，但連續的占用區段會吸引更多插入，形成 primary clustering。

Quadratic probing 用 $(h_0(k)+c_1 i+c_2 i^2) \bmod m$，參數不可任意選。例如 m=7、偏移 i^2 的前七項為 $0,1,4,2,2,4,1$，只覆蓋四槽；即使其他槽有空間也可能插不進去。

兩個 key 的 home 相同時，以上兩種方法的整條序列相同，稱 secondary clustering。Double hashing 讓步長也依 key 決定，但仍需證明步長與 m 互質；「多一個函數」本身不是覆蓋證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩種可證的全覆蓋探測

Double hashing：$h(k,i)=(h_1(k)+id)\bmod m$。回到 home 等價於 $m \mid id$。設 $g=\gcd(m,d)$，約分後最小正週期是 m/g，所以覆蓋全表恰需 g=1。m 為質數可取 1≤d<m；m 為 2 的冪可取奇數 d。

三角偏移：對 $m=2^w$，使用 $f(i)=i(i+1)/2 \bmod m$，先做整數除法。m=8 時為 $0,1,3,6,2,7,5,4$。

若 f(i)=f(j)，則 $2^{w+1} \mid (i-j)(i+j+1)$。兩因子一奇一偶，故偶因子需被 2^{w+1} 整除。當 $0 \le i,j < 2^w$，有 $|i-j|<2^w$、$1 \le i+j+1 < 2^{w+1}$，唯一可能為 i−j=0。因此 f 無重複，正好覆蓋全表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniform probing 是較強的理想模型

以下期望探測界假設：每個新 key 的探測序列，是相對於目前占用狀態仍均勻的 m 槽排列；無刪除、無 tombstone，且 n<m。它不是每次隨機選一槽的有放回抽樣。

線性、二次與雙重雜湊只有受限的序列集合，不能因為 home 均勻或函數族 universal，就直接宣稱符合 m! 種排列的理想模型。

因此分開三件事：chaining 的碰撞機率、探測是否覆蓋全表，以及探測排列的分布。前兩者成立，仍不足以推出下一頁的期望公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

失敗搜尋界：由尾機率加總

令 X 是遇到第一個 EMPTY 的探測數，$\alpha = n/m < 1$。前 i−1 槽均占用的機率為

$$\Pr(X \ge i) = \prod_{j=0}^{i-2} \frac{n-j}{m-j} \le \alpha^{i-1}, \quad 1 \le i \le n+1.$$

i=1 為空乘積 1；i>n+1 的機率為 0。各比值不超過 n/m，因 $m \ge n$。由 $X = \sum_{i \ge 1} \mathbf{1}[X \ge i]$ 得

$$E[X] = \sum_{i \ge 1} \Pr(X \ge i) \le \sum_{i \ge 1} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

新 key 插入的探測同樣適用。m=10、n=8 時上界為 5；它不是每次恰好五次，也不是 linear probing 的無條件保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功搜尋界：重現插入時的路徑

固定 n 筆相異鍵的插入順序，無刪除或搬移；成功 query 等機率選記錄。第 i+1 筆插入時只有 i 筆，期望探測至多 $m/(m-i)$；日後搜尋重現其插入路徑。

$$E[C] \le \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^{m} \frac{1}{k}$$

$$\le \frac{1}{\alpha} \int_{m-n}^{m} \frac{dx}{x} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}.$$

因 1/x 遞減，每段 [k−1,k] 的積分至少 1/k。$\alpha = 0$ 時沒有成功 query，不能直接代入 1/α；右式在 α→0 時極限為 1。這裡同時需要理想探測模型與等機率成功查詢，少一項都不能沿用完整公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

低活躍負載，也可能搜尋整張表

m=7，先插入 home 分別為 0～6 的七筆，再全部刪除。此時 n=0、α=0，但七槽全為 DELETED，找任意不存在的 key 都要探測 7 次。

因此不能把無 tombstone 的 $1/(1-\alpha)$ 界套到這張表。可另追蹤 $\beta = (n+d)/m$，d 為 tombstone 數；本例 β=1，才反映沒有 EMPTY 可提早停止。β 本身也不是任意實作的完整成本證明。

重建會清除 tombstone，但必須列入配置、重新計 hash 與插入的成本；在 open addressing 中，重新插入本身也需要探測。若要主張重建的期望攤銷界，還需固定負載門檻、隨機模型與重建規則，不能只說「定期 rehash」就省略成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Perfect hashing：把隨機重試放在建表

FKS 兩層法處理固定的 n 個相異 key。建表後每次查詢最壞 $O(1)$ 個 word 操作，空間 $O(n)$；建表採隨機重試，只有期望線性成本，不能稱最壞線性。

n=0 時直接建空 dictionary，不做 mod 0。以下 n≥1，第一層使用 m=n，均勻抽 universal h。桶 j 大小為 n_j，準備配置 n_j^2 個次層槽。

令 Y 為碰撞的無序鍵對數。對每桶用 $n_j^2 = n_j + 2\binom{n_j}{2}$，可得

$$\sum_j n_j^2 = n + 2Y, \qquad E\left[\sum_j n_j^2\right] \le n + 2\binom{n}{2}\frac{1}{n} = 2n - 1.$$

平方空間和的控制來自碰撞對數，不是宣稱每個桶都一定很小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第一層：真的檢查空間，再接受

令 $S = \sum_j n_j^2$。Markov 不等式給出 $\Pr(S \geq 4n) \leq E[S]/(4n) < 1/2$。每次獨立重抽，直到實際計算得到 S<4n。

每輪只需清空 n 個桶的計數並掃過 n 個 key，成本 O(n)；先檢查 S，**不要在不合格的輪次先配置所有平方表**。接受機率大於 1/2，所以期望試驗輪數少於 2，第一層期望成本 O(n)。

接受後有確定的 S<4n。這比只知道 E[S]<2n 更強：最終成品確實占線性空間，而不是把「期望線性」誤寫成每一次都已線性。極端壞抽樣仍可能重試很多次，不能把期望輪數當硬上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第二層：平方槽數換取無碰撞

條件於已接受的第一層分桶。桶內 s≥2 個 key，配置 s^2 槽，獨立抽 universal h_j。碰撞對數 Z 滿足

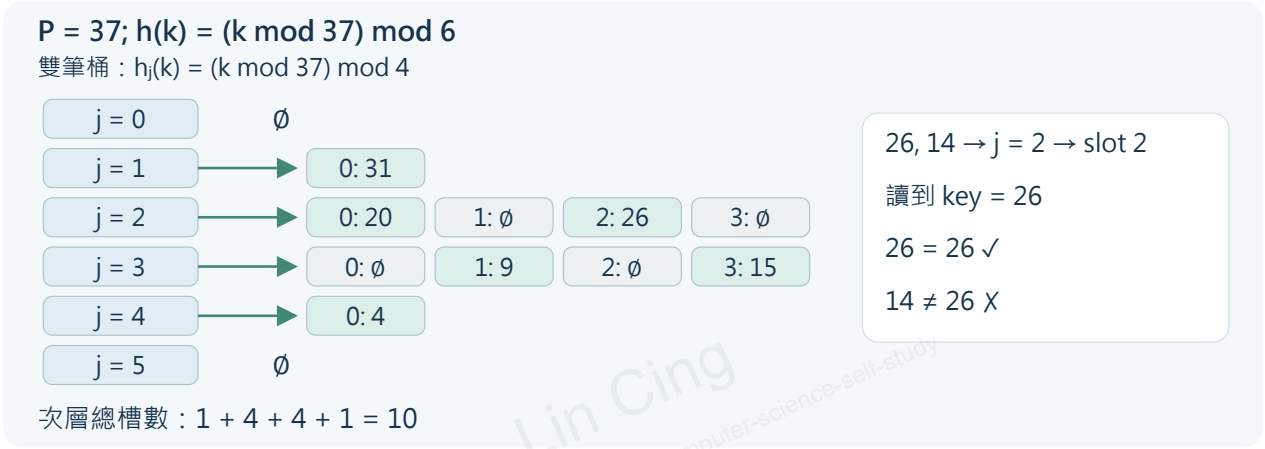
$$E[Z] \leq \binom{s}{2}\frac{1}{s^2} = \frac{s-1}{2s} < \frac{1}{2}, \qquad \Pr(Z \geq 1) \leq E[Z] < \frac{1}{2}.$$

真的檢查無碰撞才接受。每輪清空 s^2 槽、插入 s 筆共 $O(s^2)$，期望少於兩輪。對固定的合格分桶加總，期望成本為 $O(\sum_j s_j^2) = O(n)$；再對第一層取期望，總建表仍 O(n)。

空桶不配置次層；單筆桶可直接存記錄。存每桶大小、hash 參數與指標只另需 O(n)。查詢最壞界依賴定額 hash 與 key 比較，增加新 key 則超出這份靜態規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩層配置：同一組鍵值完整落到槽位



$K = \{4, 9, 15, 20, 26, 31\}$、P=37、$h(k) = (k \bmod 37) \bmod 6$。桶大小 [0,1,2,2,1,0]，平方和 10<4n=24。兩個雙筆桶用 $h_j(k) = (k \bmod 37) \bmod 4$，分別放到槽 (0,2) 與 (1,3)，實際檢查都無碰撞。

沒有存入的 key，仍可能撞到已占用槽

查 26：第一層算得 2，次層算得 2，讀到 key=26，相等才成功。查 14：同樣走 2→2，讀到的卻是 26，所以必須回不存在。

查 5：第一層到空桶 5，直接回不存在，不計算不存在的次層函數。查 10：第一層到單筆桶 4，讀到 4≠10，仍回不存在。

Perfect 只保證已存集合內不互撞，不替未存鍵提供成員資格證明。 Lookup 必須先處理空桶，再檢查次層槽是否占用，最後比完整 key。建表成功後，無論查詢是否存在，都只走常數層；這才是查詢最壞常數時間的保證。

BST 的限制來自所有祖先

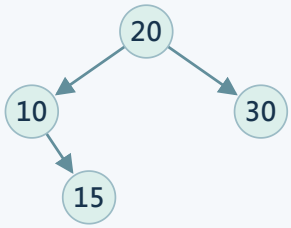
BST（binary search tree，二元搜尋樹）把比較結果變成分支。本節延續唯一 key 的 dictionary：每個節點存 key、value、left、right、parent；空連結為 NIL，root.parent=NIL。同 key 插入只更新 value。

每個節點的整棵左子樹都小於它，整棵右子樹都大於它。 只檢查父子大小並不足夠。往左把允許區間的上界縮到目前 key，往右則更新下界。

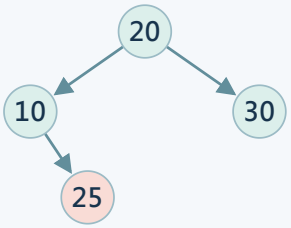
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

祖先區間：局部大小正確仍可能違規

每個祖先都提供一個界限



15 ∈ (10, 20)
所有限制皆成立



25 > 10 ✓
25 < 20 ✗
違反祖先上界
(10, 20)

CLRS §12.1 也容許重複 key，使用左側 ≤、右側 ≥ 的性質。本節採嚴格次序；若要存重複資料，可另存 count 或以 (key,唯一編號) 排序，不能直接混用兩套規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋：每次比較排除一整棵子樹

```
FIND(T,k):
    x = T.root
    while x != NIL:
        c = compare(k,x.key)   // 三向比較：小於、等於、大於
        if c == 0: return x
        x = x.left if c < 0 else x.right
    return absent
```

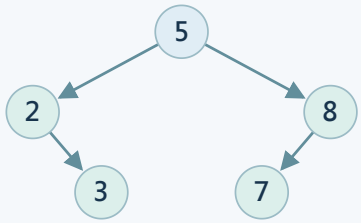
迴圈不變量：若 k 存在，必在目前 x 的子樹中。比較不等時，BST 的全域次序排除另一側；每次嚴格往下一層，有限樹必終止。到 NIL 表示候選子樹已空。

深度 d 按 root 到節點的**邊數**算，root 深度為 0。成功搜尋深度 d 的節點，訪問 d+1 筆；失敗搜尋只計遇到的非 NIL 節點。以下一次三向 key 比較算一次；NIL 測試與比較器內部指令不混入這個數。

高度 h 按最長路徑的邊數算，空樹高度 −1；非空樹搜尋為 O(h+1)，空樹另需 O(1)；統一可寫 O(h+2)。

同一棵樹的成功與失敗搜尋

深度從 0 起；成功比較數 = 深度 + 1



d = 0　查 7：5 → 8 → 7，3 次

d = 1　查 6：5 → 8 → 7 → NIL

d = 1　失敗也比較 3 次

d = 2　查 9、10 都在 8 的右側失敗

查 7 走 5→8→7，共三次 key 比較；查 6 也走同樣三個節點，最後才到 NIL。查 9 或 10 則走 5→8→NIL，只比較兩次。

成功與失敗都要實際數訪問節點。到 NIL 的最後一次指標測試，不是額外一次 key 比較。

平均成本：先決定查詢怎麼抽

若只在五個已存 key 中等機率查詢，成功比較總數為 1+2+2+3+3=11，平均 **11/5**。若 query 改為整數 1～10 均勻抽取，還要加上五個失敗查詢。

query	1	4	6	9	10
訪問 key	5,2	5,2,3	5,8,7	5,8	5,8
比較數	2	3	3	2	2

因此混合平均為 (11+12)/10=**2.3**。失敗間隙不是等機率事件：區間 (8,+∞) 在此包含 9、10，權重 2/10；不能把每個 NIL 連結各算同一權重。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中序走訪與極值

中序（inorder）依「左子樹 → 目前節點 → 右子樹」列出 key。歸納假設兩個子樹各自輸出遞增序列；BST 性質又讓左側每個值 < 根 < 右側每個值，因此串接後仍遞增。每個節點處理一次、空連結測試 O(n) 次，總時間 Θ(n+1)，遞迴堆疊 O(h+2)。

找 MINIMUM(x) 時，前提為 x≠NIL；沿 left 直到沒有 left child。若還有左子樹，最小值一定在其中；沒有時，目前 key 小於所有右子樹 key，故答案就是 x。MAXIMUM 對稱。空 dictionary 的極值操作應直接回 absent。

用 BST 排序，不能只算最後的走訪。 依序插入 1,2,⋯,n 會得到右鏈；建樹比較數為 0+1+⋯+(n−1)=n(n−1)/2，再加 Θ(n) 的走訪，仍是 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼：有右子樹就向下，否則向上找

```
SUCCESSOR(x):                    // x 是有效節點
    if x.right != NIL: return MINIMUM(x.right)
    p = x.parent
    while p != NIL and x == p.right:
        x = p; p = p.parent
    return p                    // NIL 表示沒有後繼
```

有右子樹時，其中最小 key 就是下一個；它比 x 大，且不大於任何其他右子樹 key。沒有右子樹時，向上經過「右孩子 → 父」所遇到的父鍵都更小；第一次由左側回到的祖先才是最小的較大 key。

在樹 5(左 2、右 8)，2.right=3、8.left=7 中：succ(5)=7，路徑 8→7；succ(3)=5，路徑 3→2→5；succ(8)=NIL。PREDECESSOR 交換 left/right 即得。

每次呼叫只沿單一路徑，最壞 O(h+1)。從最小值開始反覆找 successor 可列出全部節點；每條樹邊至多向下、向上各經過一次，整段是 O(n)，不是 n 次最壞界相乘後的必然 Θ(nh)。

插入：找到的是空連結，不是任意葉節點

```
INSERT(T,k,v):
    p = NIL; x = T.root
    while x != NIL:
        p = x; c = compare(k,x.key)
        if c == 0: x.value = v; return x
        x = x.left if c < 0 else x.right
    z = Node(k,v,NIL,NIL,p)
    if p == NIL: T.root = z
    elif k < p.key: p.left = z
    else: p.right = z
    return z
```

迴圈保留 k 的祖先區間；到 NIL 後，新節點恰好落在所有祖先允許的範圍內。既有 key 的分支只改 value，不改 key 或節點身分；在樹內任意改 key 可能破壞次序。

最壞 O(h+2)，包含空樹插入；新增節點空間 O(1)。

Transplant：先換父節點的連結

```
TRANSPLANT(T,u,v):              // u 必須是樹內有效節點
    p = u.parent
    if p == NIL: T.root = v
    elif u == p.left: p.left = v
    else: p.right = v
    if v != NIL: v.parent = p
```

此程序只把「指向 u 的連結」改成指向 v，並更新非空 v 的 parent；**不會替呼叫者接好 v.left、v.right**。v=NIL 合法，但本節的 NIL 不是可寫入 parent 的節點。

刪除 z 若沒有左孩子，直接 TRANSPLANT(T,z,z.right)，也包含 z 是葉節點的情況。若只有左孩子，則改用 z.left。所有被保留的子樹仍滿足 z 的祖先區間。

例如 root=20 只有右孩子 40：移接後 root=40、40.parent=NIL。若 20 是最後一筆，則 root=NIL。這些分支都只改常數個連結；DELETE(key) 尚須先付搜尋成本。

兩個孩子：先把後繼從原位取出

```
DELETE-NODE(T,z):
    if z.left == NIL: TRANSPLANT(T,z,z.right)
    elif z.right == NIL: TRANSPLANT(T,z,z.left)
    else:
        y = MINIMUM(z.right)
        if y.parent != z:
            TRANSPLANT(T,y,y.right)
            y.right = z.right; y.right.parent = y
        TRANSPLANT(T,z,y)
        y.left = z.left; y.left.parent = y
```

y 是右子樹最小值，所以沒有左孩子，但**可能有右孩子**。深處的 y 要先由 y.right 接替，才能把整棵原右子樹接到 y。若 y 就是 z.right，則保留原本 y.right；不可執行 y.right=z.right，否則會得到 y.right=y 的自環。

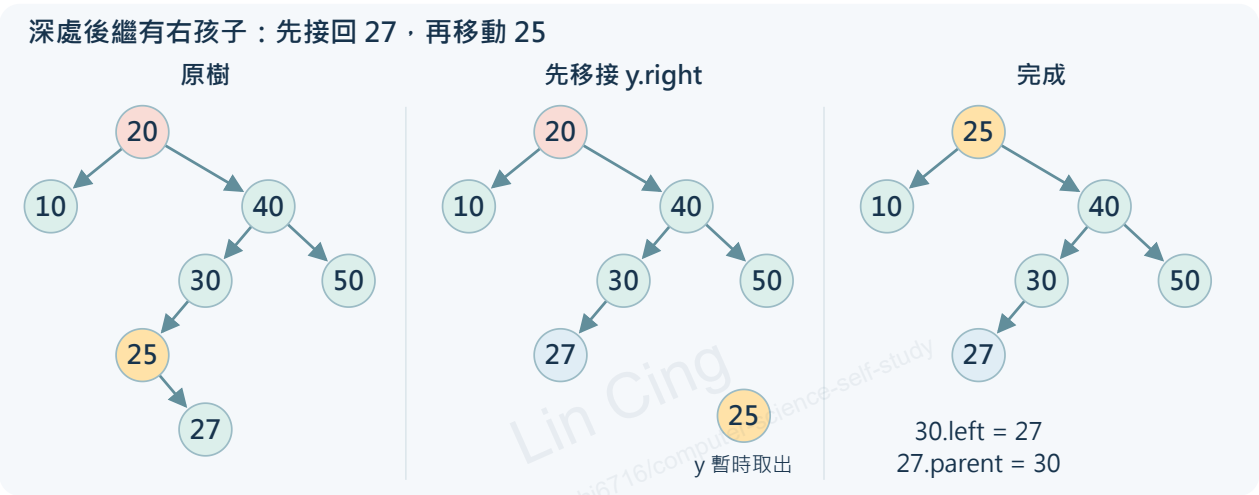
後繼就是右孩子：保留它的右連結



刪 z=20：y=40，且 y.parent=z。先讓 root 指向 y，y.parent=NIL；再接 y.left=10、10.parent=y。**y.right=50 從頭到尾不變。**

中序由 [10,20,40,50] 變為 [10,40,50]。保留下來的所有鍵值、value 與節點身分不變；只有 z 離開樹。非根的 z 也同樣處理，只是改接的是 z 原父的 left 或 right。

深處後繼：它的右孩子不能遺失



刪 root z=20，右子樹最小節點 y=25，且 y.right=27。依序更新：① 30.left=27、27.parent=30；② y.right=40、40.parent=y；③ root=y、y.parent=NIL；④ y.left=10、10.parent=y。

深處後繼：核對次序與可達性

圖中的中間態把 y 畫成暫時取出的物件；它的舊連結要清理或覆寫，不能在完成前把整個物件池當成一棵合法 BST。完成後 root=25、25.right=40、40.left=30、30.left=27。

原中序 [10,20,25,27,30,40,50] 只刪去 20。若第一步誤寫 30.left=NIL，27 會遺失；若忘了 parent 更新，後續 successor 的向上搜尋也會壞掉。

為何新連結仍符合次序？y=25 是原右子樹的最小值，所以原左子樹的 key 全小於 y，剩下右子樹的 key 全大於 y。27 原在 25 的右側、30 的左側，符合 25<27<30，接成 30.left 仍合法。

移接保留原來除 z 以外的每個物件恰好一次；沒有新建副本。最後還要核對 root.parent=NIL、每個非空 child 的 parent 指回擁有它的節點，且沒有循環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪鍵與刪物件：兩種實作的差別

以 z.key=20、y.key=25 為例，兩種做法都能留下正確的鍵集合，但對 handle 的效果不同。

做法	留在 z 原位的物件	離開樹的物件	外部 handle 的影響
複製後繼 payload，再刪 y	原 z，內容改成 25 的記錄	y	z 的意義改變；y 的 handle 失效
移接後繼節點 y	原 y，位置改到 z 原位	z	z 的 handle 失效；y 的身分保留

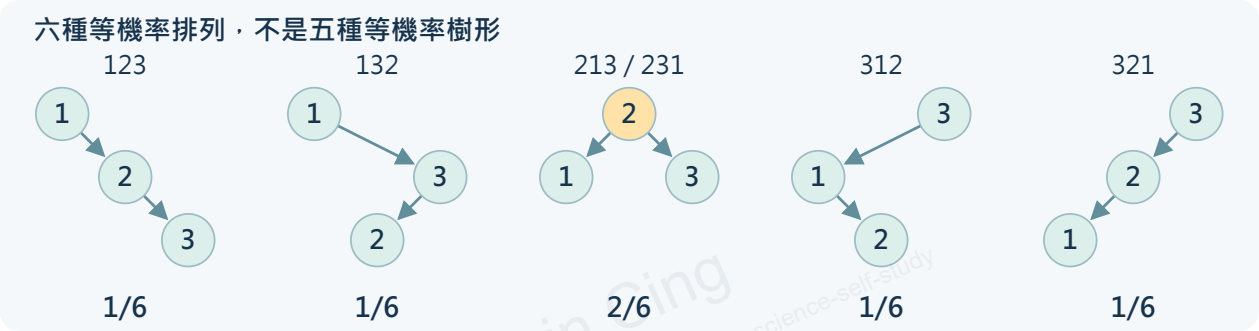
來源慣例：紙本筆記示範前者，CLRS 第三版 §12.3 採後者。本節以移接為準。複製法若採 key→value 記錄，必須搬對應 value，不能只抄 key；附加欄位、資源所有權與外部索引也要維護。

兩種方法都可改用前驅（左子樹最大值），但指標方向要整組對稱更換。不要從前驅版取圖、再套後繼版程式的左右分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機順序不等於隨機樹形

固定 n 個相異 key，均勻抽取 n! 個插入排列。首筆為 root；給定 root 秩 R=r，兩側的相對排列仍各自均勻，故子樹可遞迴套同一模型。此處只分析插入建樹。



root=2 的兩個排列得到同一平衡樹，其機率為 2/6；其餘四個排列各得到不同樹形，各為 1/6。若直接把五種形狀各設為 1/5，就換了機率模型。

固定秩的期望深度：逐一數祖先

將鍵依大小記為 $x_1<\cdots<x_n$，D_i 是 x_i 的深度。對 $j\neq i$，x_j 成為 x_i 的祖先，**當且僅當閉區間 $[x_{\min(i,j)}, x_{\max(i,j)}]$ 中最先插入的是 x_j**。

區間外先插入的 key 只能把兩者一起帶往同側；區間內第一筆若是 x_j，它會成為通往 x_i 的路徑節點。若先出現別筆，就會先到 x_i，或把 x_j 與 x_i 分開。因此

$$\Pr(x_j \text{ 是 } x_i \text{ 祖先}) = \frac{1}{|j-i|+1}.$$

深度就是祖先數，套線性期望（不需各事件獨立）得

$$E[D_i] = \sum_{j<i} \frac{1}{i-j+1} + \sum_{j>i} \frac{1}{j-i+1} = H_i + H_{n+1-i} - 2.$$

其中 $H_t = \sum_{k=1}^{t} 1/k$。n=5、i=3 時，期望深度為 $2H_3 - 2 = 5/3$，成功搜尋的期望比較數再加 1，為 8/3。

期望建樹成本：把所有深度加起來

節點插入時比較過的節點，恰好是它在完成樹中的祖先；後續插入不改動既有節點的深度。因此建樹總 key 比較數為 $C = \sum_i D_i$。利用

$$\sum_{i=1}^{n} H_i = \sum_{k=1}^{n} \frac{n-k+1}{k} = (n+1)H_n - n,$$

可得

$$E[C] = 2(n+1)H_n - 4n = \Theta(n \log n), \qquad n \geq 2.$$

常數節點配置與連結另共 Θ(n)。等機率成功查詢的平均比較數，其期望為 $E[C]/n + 1$。n=3 時 E[C]=8/3，平均成功比較數為 17/9；n=1 時建樹比較 0 次、成功查詢 1 次。

這也連到前章的 quicksort：同一相對順序遞迴以首筆為 pivot 時，兩個 key 是否比較，取決於它們之間誰最先被選到。但**平均深度為對數，尚不能推出最大深度也有同樣期望界**。

高度分析：先處理 max，再取期望

令 X_n 為高度（邊數），n≥1 時設 $Y_n = 2^{X_n}$；另設 **$Y_0=0$ 作為輔助值**，不是 $2^{X_0} = 2^{-1}$。單節點 $Y_1=1$。n≥2、條件在 root 秩 R=r 時，至少一側非空，故

$$Y_n = 2\max(Y_{r-1}^{L}, Y_{n-r}^{R}).$$

令 $a_j=E[Y_j]$。給定 R 後，兩側相對排列均勻，配合 max(u,v)≤u+v 得

$$a_n \leq \frac{2}{n}\sum_{r=1}^{n}(a_{r-1} + a_{n-r}) = \frac{4}{n}\sum_{j=0}^{n-1} a_j, \qquad n \geq 2.$$

這裡用的是**條件期望**，沒有把隨機大小的子樹與 R 當成無條件獨立。也沒有使用錯誤等式 E[max(U,V)]=max(E[U],E[V])。

例如 $Y_1=1$、$a_2=2$；n=3 的兩個平衡排列高度 1、四個鏈狀排列高度 2，所以 $a_3=(2・2+4・4)/6=10/3$。

解出指數高度界，再回到原高度

強歸納證明 $a_n \le n^3$（$n \ge 1$）。$a_1 = 1$；對 $n \ge 2$，$a_0 = 0$，且 $\sum_{j=1}^{n-1} j^3 = [n(n-1)/2]^2$，所以

$$a_n \le \frac{4}{n} \sum_{j=1}^{n-1} j^3 = n(n-1)^2 \le n^3.$$

為何可以從這裡推 $E[X_n]$？令 $\mu = E[X_n]$。由 $e^u \ge 1 + u$ 得 $2^x \ge 2^\mu[1 + (\ln 2)(x - \mu)]$；取期望，線性項消失，得到 Jensen 在此所需的形式：

$$2^{E[X_n]} \le E[2^{X_n}] = a_n \le n^3 \quad \Longrightarrow \quad E[X_n] \le 3 \log_2 n.$$

$e^u - 1 - u$ 的導數是 $e^u - 1$，在 $u = 0$ 取最小值 0，故上述切線界成立。此處高度的支集有限，期望均存在。

另一方面，h 高的二元樹至多 $2^{h+1} - 1$ 個節點，故每棵樹 $h \ge \log_2(n+1) - 1$。兩側合起來，隨機建樹的期望高度為 $\Theta(\log n)$，但單次最壞仍為 $n-1$。

紅黑樹：用路徑不變量限制高度

紅黑樹仍是嚴格次序的 BST，只多存 color。以下 B 表示黑、R 表示紅；圖中的顏色與 B／R 標籤並用，避免只能靠顏色辨識。

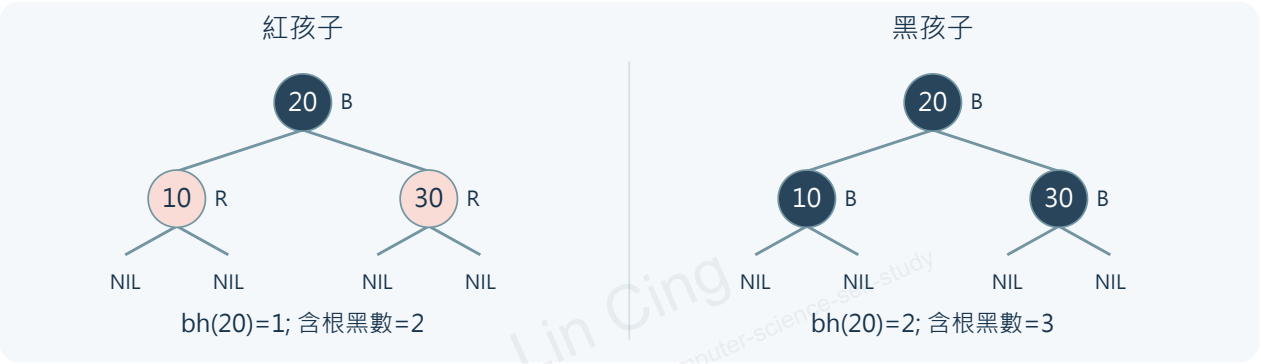
1. 每個節點非紅即黑，根節點 為黑。
2. 所有外部空位置 NIL 為黑；資料節點沒有 孩子，也仍連到 NIL。
3. 紅節點的兩個 孩子 都黑，故不會連續兩個紅節點。
4. **從任一節點往下到各個 NIL 的路徑，黑節點數都相同。** 不能只檢查少數 根節點–NIL 路徑。

用一個共享 哨兵節點 表示所有 NIL，令 NIL.left=NIL.right=NIL、NIL.color=B，root.parent=NIL。一般 child.parent 一致性只檢查非 NIL 的 孩子。同鍵 插入只更新 value，不改形狀或顏色。

前節 BST 不寫 NIL.parent；本節刪除修復將暫借它記錄一個缺黑位置的父節點。它不是所有空位置同時有效的 parent。旋轉遇到 β=NIL 時仍不寫 NIL.parent。

黑高先固定計數範圍

定義 $bh(x)$：從 x 往下到任一 NIL 的黑節點數，**不含 x，含末端 NIL**；另定 $bh(NIL)=0$。紅黑性質保證此數不因選哪條路而變。



若 c 是非 NIL 的 孩子，c 紅時 $bh(c)=bh(x)$；c 黑時 $bh(c)=bh(x)-1$。這是因為父的計數含 c，而 孩子 自己的 bh 不含自己。c=NIL 時同樣有 $bh(x)=1+bh(NIL)$。

高度界：先數最少需要多少節點

本章沿用內部節點邊數高度 h：單節點 h=0、空樹 h=−1。另以 H 計 根節點 到外部 NIL 的最長邊數，非空時 H=h+1，空樹 H=0；高度證明使用 H，不偷偷換掉 h 的定義。

引理： x 的子樹至少含 $2^{bh(x)}-1$ 個內部節點。對子樹高度歸納：x=NIL 時，$0=2^0-1$。若 x 非 NIL，兩個 孩子 的黑高各至少 bh(x)−1；套歸納假設再加上 x：

$$n_x \geq 2\left(2^{bh(x)-1}-1\right)+1=2^{bh(x)}-1.$$

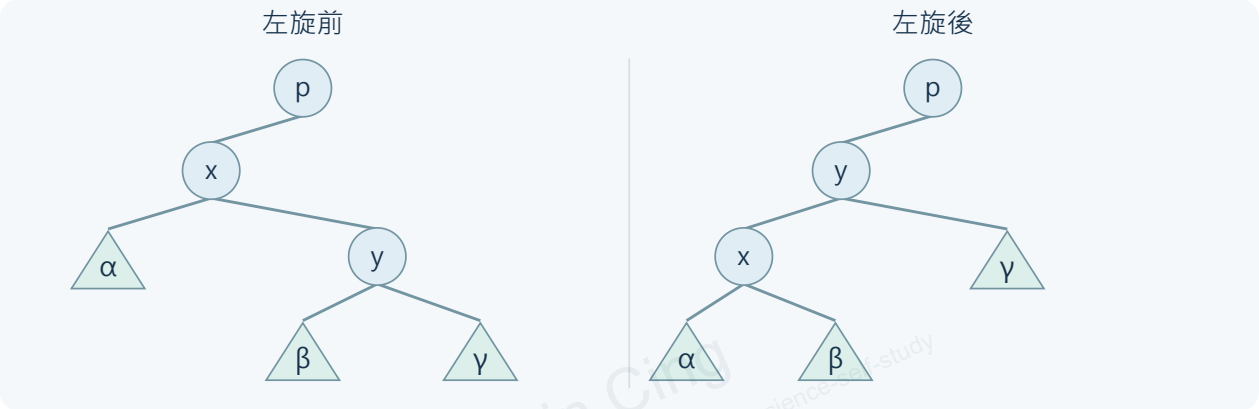
再取一條最長 根節點–NIL 路。含 根節點 與 NIL 共 B 個黑點、R 個紅點。兩端黑，沒有相鄰紅，所以 $R \leq B-1$；路徑邊數

$$H=B+R-1 \leq 2(B-1)=2bh(root).$$

因此 $n \geq 2^{bh(root)}-1 \geq 2^{H/2}-1$，得到 $H \leq 2\log_2(n+1)$。空樹 H=0 也成立。搜尋路徑加終止測試需 O(H+1)，統一寫 O(log(n+2))，才能涵蓋空樹常數成本。

這只保證搜尋高度；插刪還須證明修復會終止且不超出同一時間界。紅黑樹也不要求每個節點左右高度差至多 1，那是 AVL 的條件。

左旋：β 換父，舊父改接新根



前提 x≠NIL、y=x.right≠NIL。α、β、γ 表示整棵子樹，允許為 NIL；p 是 x 的**舊父**。旋轉後 y 取代 x 的局部根位置，β 從 y.left 移到 x.right。

左右旋轉：完整且對稱的接線

```
LEFT-ROTATE(T,x):                RIGHT-ROTATE(T,y):
    p=x.parent; y=x.right           p=y.parent; x=y.left
    beta=y.left                     beta=x.right
    x.right=beta                    y.left=beta
    if beta!=NIL: beta.parent=x     if beta!=NIL: beta.parent=y
    y.parent=p                      x.parent=p
    if p==NIL: T.root=y             if p==NIL: T.root=x
    elif p.left==x: p.left=y         elif p.left==y: p.left=x
    else: p.right=y                 else: p.right=x
    y.left=x                        x.right=y
    x.parent=y                      y.parent=x
```

先保存舊父 p；β=NIL 時不改 sentinel.parent。前提分別為 x.right≠NIL 與 y.left≠NIL。

旋轉接線：逐條核對數值與次序

取 p=30、x=10、y=20、β=15，且 p.left=x。其餘子樹內容不變。

順序	寫入	目的
1	10.right=15	接回 β
2	15.parent=10	讓 β 認新父
3	20.parent=30	新局部根接外父
4	30.left=20	更新原父指向局部根的 連結欄位
5	20.left=10	舊根變成左 孩子
6	10.parent=20	舊根認新父

若 x 原為 根節點，第 3 項改為 20.parent=NIL，第 4 項改為 T.root=20。

鍵值區間 α<x<β<y<γ 不變，故中序與 BST 次序保持；節點身分及 鍵值與資料 也不變。**旋轉本身不保證紅黑性質**。

插入紅葉：先辨認唯一可能的違規

先依前節 INSERT 搜尋；同鍵 則更新 value 並返回。新節點 z 以 R 插入，left=right=NIL，parent 指向搜尋到的父節點。

原本的一個黑 NIL 被「紅 z 加黑 NIL」取代，任何穿過此位置的路徑黑數不變。若插成黑色，只有經此位置的路增加一黑，通常會破壞黑數相等。

修復迴圈入口保持三件事：**z 紅；若 z.parent 是 根節點，該父必黑；最多只剩根紅或 z 與父同紅其中一種違規。** BST 與各路黑數相等均維持。

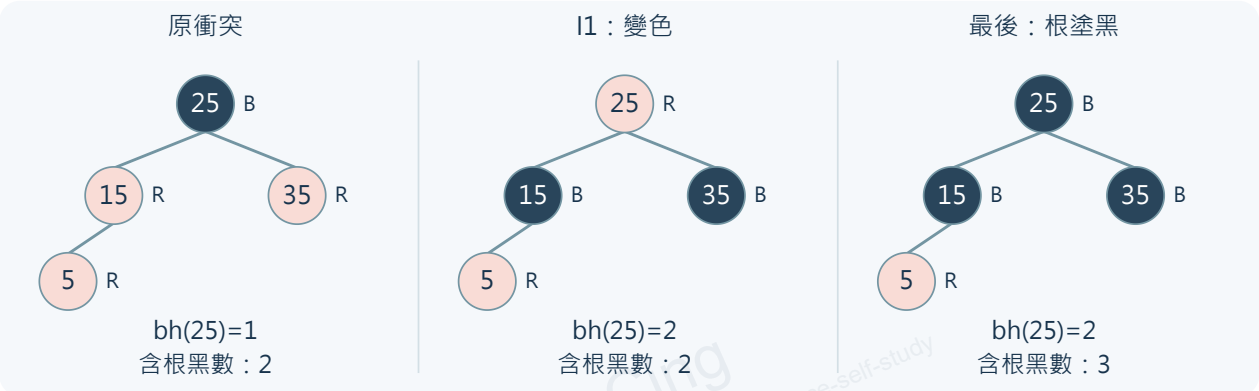
若父黑，直接結束；若 z 是新根，最後塗黑即可。只有父 p 紅才進迴圈；p 不可能是根，因此祖父 g 存在且黑。叔 u 是 g 的另一 孩子，可能為 NIL，但 color 仍可安全讀取。

I2 不是另一個互斥結局：執行後必接 I3。旋轉會改 parent；I3 使用的是當下的 z.parent 與祖父，不能沿用旋轉前快取的 p、g。

插入修復：父在左側的三個分支

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:               // I2
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                    // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: apply the explicitly mirrored branch
T.root.color=B
```


I1：重新著色不改祖先路徑的黑數



先插 25、15、35，再插 5。新增的 5R 與父 15R 衝突，叔 35R 也紅。把 15、35 塗黑、25 塗紅，z 移到 25；此時兩條子路徑的黑數仍相同。

I1：哪些黑數改變，哪些不變？

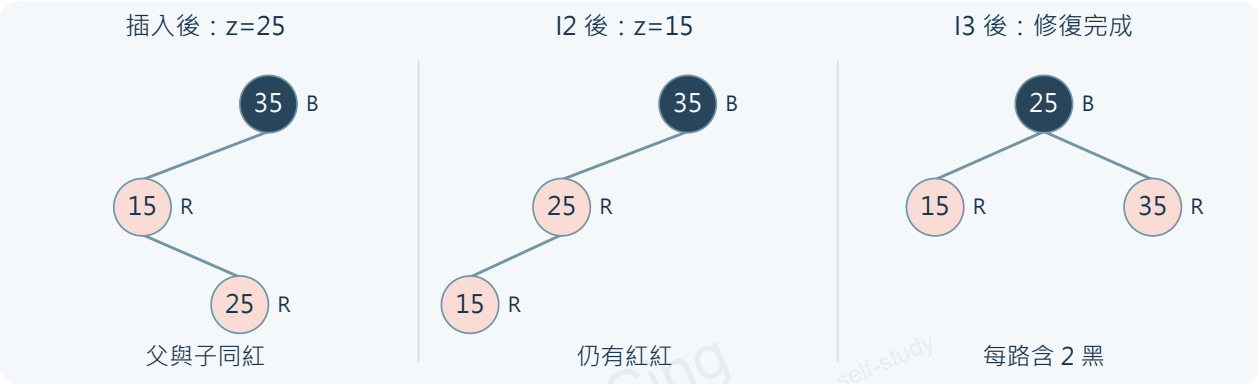
令 p／u 以下（不含 p／u）的每條路有 b 黑點。原先經 g 的完整路有 1+b 黑；改成 gR、p/uB 後仍是 1+b。**穿過 g 的祖先到 NIL 路徑黑數保持不變**，但不含 g 的 bh(g) 從 b 變為 b+1。

違規只可能移到 g 與它的父。若 g 恰為 根節點，迴圈結束後才把它塗黑，使所有 根節點–NIL 路一起加一黑。這個最後步驟不能套到非根 g。

計數範圍	I1 前	I1 後
不含 g 的 bh(g)	b	b+1
含 g 到 NIL	1+b	1+b
任一祖先經 g 到 NIL	外部黑數+1+b	外部黑數+1+b

若 g=25 為根且 b=1，I1 後 bh(25)=2、含根黑數=2；最後根塗黑才把含根黑數改為3。

I2 接 I3：折線先拉直，再修復

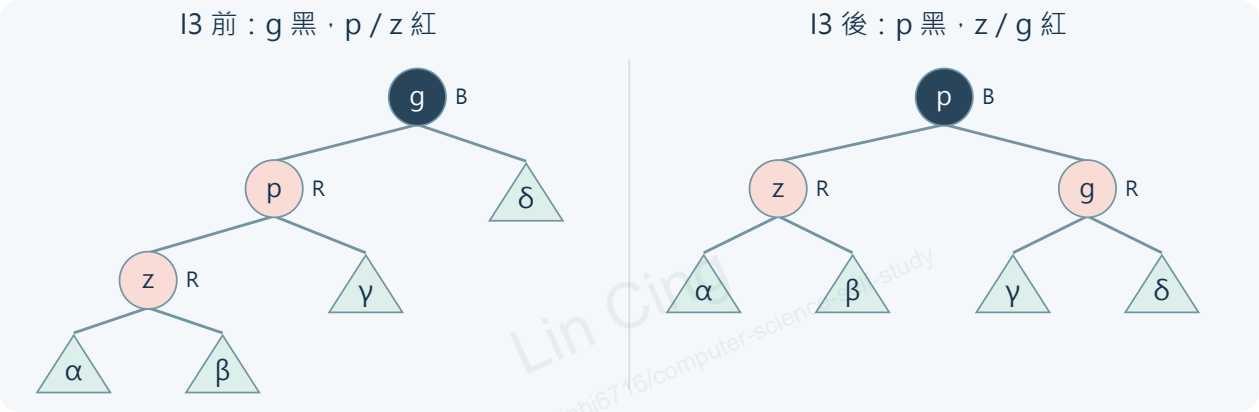


插 35、15、25，叔為黑 NIL，且父 15 在祖父 35 左側、新 25 在父右側。I2 先令 z 指舊父 15，再左旋 15；此時 25R 成為 15R 的父，**紅紅仍存在**。

I3 把目前父 25 塗黑、祖父 35 塗紅，再右旋 35。結果為 25B、15R、35R，每路含 2 黑；I2 後不能返回。

I3 為何保留黑數，也消除紅紅

設祖父 g 黑、其左孩子 p 紅，而 p 的左孩子 z 也紅；叔 u 黑。z 的兩個孩子為 α、β，p 的原右子樹為 γ，g 的原右子樹為 δ。把 p 塗黑、g 塗紅，再右旋 g：



I3：逐條驗證修復後的不變量

α、β 因 z 紅而為黑根；γ 因 p 紅而為黑根；δ 就是黑叔。各邊界子樹從根到 NIL（**含子樹根**）的黑數皆為 b；原經 g 的路貢獻 1+b，改後經 p 也貢獻 1+b。

新局部根 p 黑，不會與外父形成紅紅。z 仍紅但父 p 黑；g 改紅，其 孩子 γ、δ 都黑。故局部所有紅黑性質恢復，接回原父也不改外部黑數，修復可以結束。

若原本已是外側鏈（插 35、25、15），跳過 I2，直接 I3 即可；黑數證明完全相同。

通往哪棵子樹	修復前的局部黑節點	修復後的局部黑節點
α、β	gB；再加 b	pB；再加 b
γ	gB；再加 b	pB；gR 不計；再加 b
δ	gB；再加 b	pB；gR 不計；再加 b

四種目的地前後皆 1+b。子樹內部連結及顏色不變，因此每個內部節點原有的路徑相等性也保留。

父在右側：把鏡射動作寫出來

以下取代主修復程式的 else；條件是 z.parent=祖父.right。u 這次取左叔，旋轉方向也交換。

```
u=z.parent.parent.left
if u.color==R:                                  // I1 mirrored
    z.parent.color=B; u.color=B
    z.parent.parent.color=R
    z=z.parent.parent
else:
    if z==z.parent.left:                         // I2 mirrored
        z=z.parent
        RIGHT-ROTATE(T,z)
    z.parent.color=B                            // I3 mirrored
    z.parent.parent.color=R
    LEFT-ROTATE(T,z.parent.parent)
```


插入為何至多旋轉兩次？

I1 不旋轉，每次把 z 上移兩層，最多 O(H) 次。I2 接 I3 最多兩次基本旋轉，隨即結束；只做 I3 則一次。因此 CLRS 這套 bottom-up 插入的**旋轉總數至多 2 次**，不是每層各兩次。連搜尋、重新著色一起算，總 O(log(n+2))，新增節點空間 O(1)。

第一種離開迴圈的情況是父黑：沒有紅紅，最後只須確保根黑。第二種是 I3 把局部根塗黑：它與外父也不會衝突。I1 則嚴格往上走，不能無限重複。

修復分支	基本旋轉	下一步
I1：叔紅	0	z 上移兩層，可能重複
I2：叔黑、內側	1	立刻接 I3
I3：叔黑、外側	1	局部根黑，修復結束
最後根塗黑	0	完成

可能的旋轉段只有 I3 或 I2→I3；I1 可以出現很多次，但不會增加旋轉數。

兩種修復時機，不能混接程式

比較項目	本節 bottom-up	top-down 2-3-4 分裂式插入
何時處理雙紅 孩子	先插紅葉，再從違規處向上修復	搜尋途中遇 4-node 先分裂、變色
何時可能旋轉	叔黑的 I2／I3	向下處理時若產生相鄰紅便局部修復
共同不變量	BST 次序、無相鄰紅、各路黑數相等	相同，但修復入口條件不同

兩者可得到不同的合法樹形；不能把 top-down 的變色時機插入本節程式卻沿用原證明。「一次雙旋調整」可能包含兩次基本旋轉；比較旋轉數前，先固定計數單位與具體演算法。

也可把紅邊視為紅 孩子 與黑父的連結，收縮後對應 2-3-4 tree；一般紅黑樹允許左紅或右紅。額外要求紅邊左傾的是另一套規格，不能默認。

刪除前先分清 z、y、x 與原色

z 是要刪除的節點。若 z 至多有一個孩子，令 y=z；若有兩個孩子，令 y 為右子樹的最小節點，稍後把 **y 這個物件移到 z 的位置**。

x 負責填補 y 的舊位置：後繼 y 沒有左孩子，因此取 x=y.right；若 y=z，則取 z 唯一的非空孩子，沒有孩子時取 NIL。先保存 y 的原色，再讓移來的 y 繼承 z 的顏色。**是否需要修復，取決於 y 搬動前的顏色。**

```
RB-TRANSPLANT(T,u,v):
    若 u.parent==NIL: T.root=v
    否則若 u==u.parent.left: u.parent.left=v
    否則: u.parent.right=v
    v.parent=u.parent              // 即使 v==NIL 也寫
```

本節允許寫入 NIL.parent，只用它保存當前缺黑位置 x 的父節點，供修復時尋找兄弟；它不是所有空連結共用的正確父節點。旋轉遇到 β=NIL 時不可覆寫此欄位。修復完成後才重設 NIL.parent=NIL；NIL 始終為黑色。

RB-DELETE：先移接，再依原色決定修復

```
RB-DELETE(T,z):                  // z is an existing non-NIL record
    original=z.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(T,z,x)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(T,z,x)
    else:
        x,original=RELOCATE-SUCCESSOR(T,z)
    if original==B: DELETE-FIXUP(T,x)
    NIL.parent=NIL                // only after repair finishes
    reclaim z
```

刪除物件 z；其餘物件的 鍵值與資料 與身分保持不變。原色為紅時不減黑數；原色為黑時，從填補舊位置的 x 開始處理。若外部 API 接收 key，先搜尋，找不到便回傳 absent，不能對 NIL 呼叫此程序。

兩個孩子：移動後繼前先保存原色

```
RELOCATE-SUCCESSOR(T,z):          // requires two non-NIL children
    y=MINIMUM(z.right); original=y.color; x=y.right
    if y.parent==z:
        x.parent=y                // even when x==NIL
    else:
        RB-TRANSPLANT(T,y,x)
        y.right=z.right; y.right.parent=y
    RB-TRANSPLANT(T,z,y)
    y.left=z.left; y.left.parent=y
    y.color=z.color
    return x,original
```

後繼移接：資料跟著物件走

依序插入 40、20、60、10、30、50、70、45、55，得到根 40B：左子樹為 20B(10R,30R)，右子樹為 60R，其左側是 50B(45R,55R)，右側是 70B。

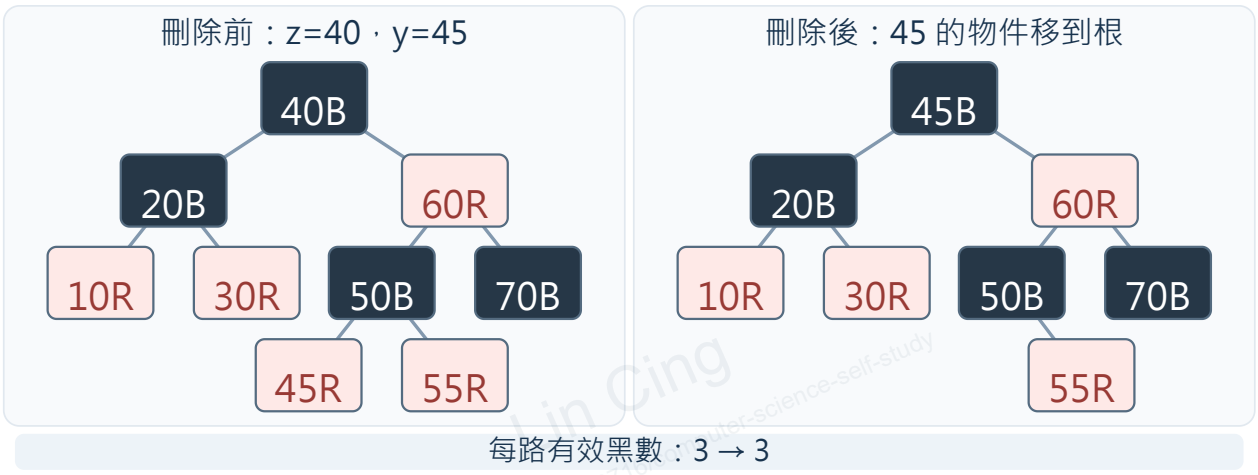
刪除 z=40 時，後繼 y=45R、x=NIL，且 y 的舊父為 50。依序執行：

1. 令 50.left=NIL、NIL.parent=50，移出原來的 45。
2. 令 45.right=60、60.parent=45，接上原右子樹。
3. 以 45 取代根 40，令 45.parent=NIL。
4. 令 45.left=20、20.parent=45，並讓 45 繼承 40 的黑色。

結果為根 45B。45 的原色是紅色，舊位置沒有少掉黑節點，因此無須修復；45 原有的資料與物件身分都保留，只有節點 40 被移除。

後繼若就是 z.right，x 的新父應為 y；若後繼更深，第一次移接已保存 x.parent，後續搬動 y 時不可破壞它。這兩種情況對 x=NIL 也必須成立。

深處後繼：同一棵樹的移接前後



兩圖是同一數例的移接前後：45 從 50 的左側搬到根。原位置由 x=NIL 補上；第一次移接後 NIL.parent=50，搬動 45 不可改寫它。修復或清理完成後才重設 NIL.parent=NIL。未畫出的空孩子皆為黑 NIL。

額外黑記在位置上，不是第三種顏色

若 y 的原色為紅，移除它不會減少路徑黑數；移到 z 位置的節點又已繼承 z 的顏色，所以不必修復。若 y 原為黑，則在填補舊位置的 **x 處暫記一份額外黑（extra black）**，讓經過該位置的路徑仍具有原來的有效黑數。

實際的 x.color 仍只有紅、黑兩種。若 x 紅，最後把它塗黑便吸收額外黑；若 x 已是根，則從全樹所有路徑同時取消這一份額外黑。只有 x 為黑且不是根時，才進入修復迴圈。

此時兄弟 w 不可能為 NIL：若 w=NIL，兄弟側只有末端 NIL 的一份黑；x 側至少已有 NIL 再加額外黑，共兩份，與兩側有效黑數相等矛盾。

x 在父 p 的左側時，w=p.right；稱 w.left 為**近側孩子 near**，w.right 為**遠側孩子 far**。x 在右側時，這兩個角色及旋轉方向全部鏡射。圖中的 x +1 表示額外黑；未畫出的空孩子皆為黑 NIL，路徑計數含起點及末端 NIL。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

修復迴圈：x 表示目前唯一的額外黑位置

```
DELETE-FIXUP(T,x):
    while x!=T.root and x.color==B:
        if x==x.parent.left: x=REPAIR-LEFT(T,x)
        else: x=REPAIR-RIGHT(T,x)
    x.color=B
```

兩個 輔助程序 回傳下一個修復位置：D2 回傳父節點，D4 回傳 根節點 使迴圈結束。最後的塗黑有兩種意義：x 紅時吸收額外黑；x 已是黑根時，把證明中的額外黑從全部路徑一起去掉，實際 color 不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左側缺黑：D1～D4 的完整接續

```
REPAIR-LEFT(T,x):
    p=x.parent; w=p.right
    if w.color==R:                          // D1
        w.color=B; p.color=R; LEFT-ROTATE(T,p)
        w=p.right
    if w.left.color==B and w.right.color==B:  // D2
        w.color=R
        return p
    if w.right.color==B:                    // D3
        w.left.color=B; w.color=R; RIGHT-ROTATE(T,w)
        w=p.right
    w.color=p.color                         // D4
    p.color=B; w.right.color=B; LEFT-ROTATE(T,p)
    return T.root
```


右側缺黑：明列鏡射，避免近側與遠側搞反

```
REPAIR-RIGHT(T,x):
    p=x.parent; w=p.left
    if w.color==R:                          // D1 mirrored
        w.color=B; p.color=R; RIGHT-ROTATE(T,p)
        w=p.left
    if w.left.color==B and w.right.color==B:  // D2 mirrored
        w.color=R
        return p
    if w.left.color==B:                     // D3 mirrored
        w.right.color=B; w.color=R; LEFT-ROTATE(T,w)
        w=p.left
    w.color=p.color                         // D4 mirrored
    p.color=B; w.left.color=B; RIGHT-ROTATE(T,p)
    return T.root
```

先看兄弟，再看近側與遠側顏色

分支	前提（先處理紅兄弟）	去向
D1	兄弟紅	變成黑兄弟，接 D2／D3／D4
D2	黑兄弟，兩個 孩子 黑	缺黑移到父；可能被紅父吸收
D3	黑兄弟，near 紅、far 黑	轉成 D4
D4	黑兄弟，far 紅	消除額外黑並結束

x 在父左側時，near=w.left，far=w.right。D1／D3 後都要重新取 w；D3 不會返回，必接 D4。D2 的 return 是把父交回外層迴圈，不是宣告整個刪除已完成。

x 在父右側時，near=w.right、far=w.left。幾何角色以「靠近／遠離 x」定義，不能把 near 永遠記成 left。兩個 輔助程序 都不更新 NIL.parent；它由移接步驟保存，直到本次修復結束。

共享 NIL 不會讓此處左右判斷含糊：進入迴圈時兄弟必非 NIL，因此父不可能兩側同時為 NIL。

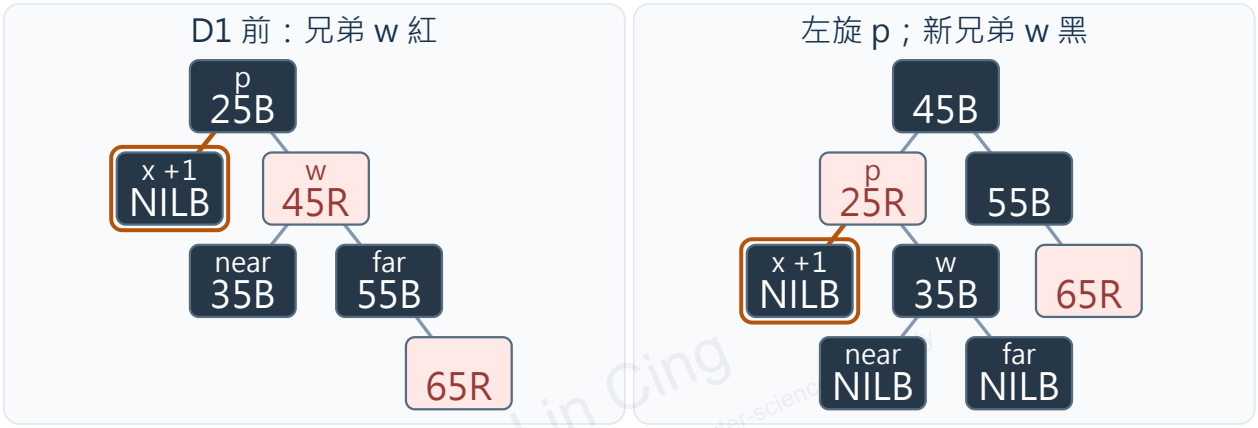
D1：兄弟轉黑，缺黑位置不動

先以 b 表示 x 子樹從根到 NIL 的**實際黑數**，含 x 與末端 NIL；加上額外黑後，有效黑數為 b+1。D1 的兄弟 w 紅，因此父 p 及 w 的兩個孩子必黑。w 紅而整棵 w 子樹的黑數為 b+1，故它的兩個孩子也各有 b+1 黑。

把 w 塗黑、p 塗紅，再左旋 p。新局部根為黑色 w；其左側的紅色 p 下，x 的有效黑數為 b+1，原近側孩子也為 b+1。w 右側原遠側孩子仍為 b+1。因此旋轉前後，從局部根開始的有效黑數均為 b+2。

額外黑仍留在 x，尚未消除。重新取得 x 的兄弟後，它已是黑色，接著依兩個孩子的顏色進入 D2、D3 或 D4。

D1 數例：重新取得旋轉後的兄弟



每路有效黑數：3 → 3

依序插入 15、25、35、45、55、65，再刪 15：x=NIL 位於 25B 左側，兄弟為 45R。把 45 塗黑、25 塗紅，左旋 25，得到新根 45B、左側 25R（右孩子 35B）、右側 55B（右孩子 65R）。x 仍是 25.left，**新兄弟改為 35B**；缺黑位置與祖先的有效黑數不變。

D2：把額外黑交給父節點

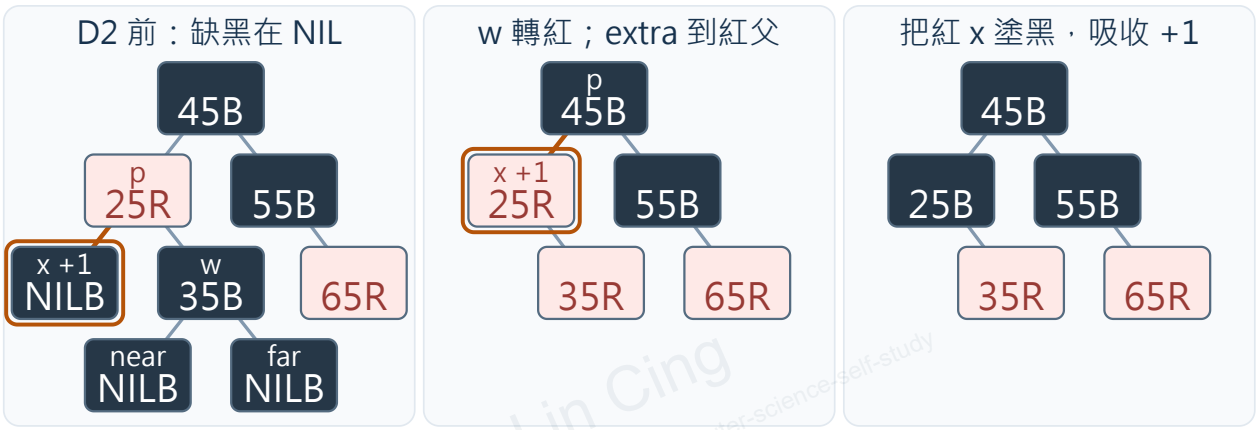
兄弟 w 黑，近側與遠側孩子也都黑。設 x 的實際黑數為 b；含額外黑後為 b+1。w 子樹的完整黑數為 b+1，而 w 的兩個孩子各為 b。

將 w 改為紅色後，w 子樹的黑數降為 b，與 x 側的**實際**黑數相等。但整棵 p 子樹對外仍少一黑，因此改把額外黑記在 p，令 x=p：

p 原來的狀態	接下來的處理
紅色	離開迴圈，最後塗黑，吸收額外黑
黑色且非根	沿祖先繼續修復
根節點	從全樹路徑同時取消額外黑，結束

D1 剛把 p 塗紅。因此 D1 後若接 D2，會由紅父吸收，不會再向上遇到第二次 D1。

D2 數例：紅父吸收額外黑



承接 D1：35B 的兩個孩子都是黑 NIL。把 35 改紅，將修復位置從 NIL 上移到 25R；迴圈停止，最後把 25 塗黑。結果為根 45B、左側 25B（右孩子 35R）、右側 55B（右孩子 65R）。額外黑由 25 的紅轉黑補齊，無須再旋轉。

D2 到達根：所有路徑一起少一黑

原樹為根 25B、兩個黑葉 15B 與 35B。刪除 15 後，D2 把 35 改紅，再令 x=25。x 已是根，因此從全樹所有路徑同時取消額外黑：



最後為 25B，其右孩子為 35R。根的黑高由 2 降為 1，各條路徑仍相等。這裡是**整棵樹一起少一黑**，與上一例「紅父塗黑以補回一黑」是不同的結束方式。

D3：近側紅，先轉成遠側紅

兄弟 w 黑，近側孩子 N 紅、遠側孩子黑。設 x 的實際黑數為 b。w 的兩個孩子從根到 NIL 均有 b 黑；因 N 紅，N 的兩個孩子也各有 b 黑。

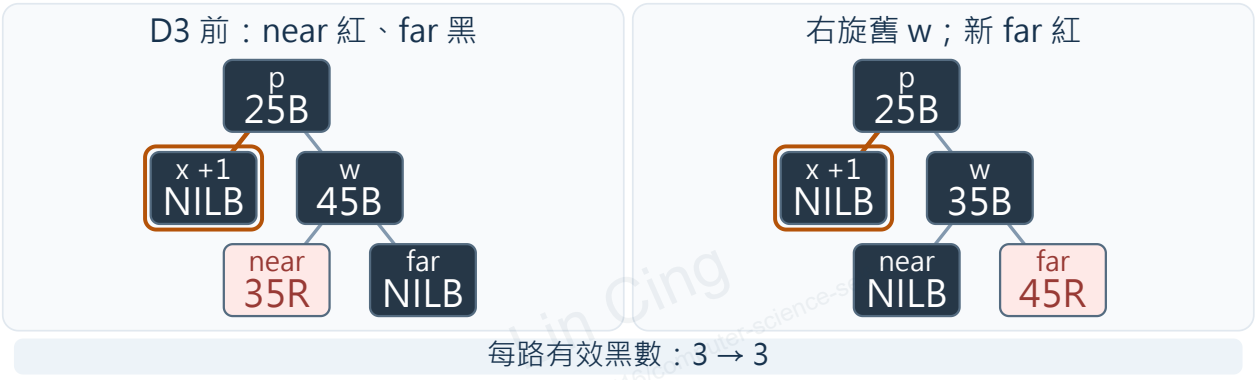
把 N 塗黑、w 塗紅，再右旋 w。新兄弟為 N：左側是原 N.left，黑數為 b；右側是紅色 w，它的兩個孩子為原 N.right 與原遠側孩子，黑數也都為 b。因此新兄弟的完整黑數仍為 b+1，父 p 及祖先看到的有效黑數不變。

新兄弟的遠側孩子就是原來的 w，現在為紅色，故**立即接 D4**。額外黑仍在 x；D3 既沒有把缺黑上移，也沒有完成修復。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D3 數例：右旋兄弟，再接 D4

依序插入 15、25、45、35，得到根 25B、左孩子 15B，以及右孩子 45B（左孩子 35R）。刪除 15 後，x=NIL、兄弟 w=45B、近側為 35R、遠側為黑 NIL。



把 35 塗黑、45 塗紅，再右旋 45，得到根 25B、右孩子 35B，其右孩子為 45R。新兄弟 35 的遠側已紅，下一步必須執行 D4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D4：兩側補齊，消除額外黑

設父 p 原色的黑色貢獻為 c∈{0,1}，x 的實際黑數為 b。兄弟 w 黑，其近側與遠側孩子的完整黑數皆為 b，且遠側孩子原為紅色。

讓 w 繼承 p 的原色，把 p 與遠側孩子塗黑，再左旋 p。新局部根為 w：

新根的哪一側	從該側根到 NIL 的黑數
左側 p	p 黑，加上 x 或近側子樹的 b，得到 b+1
右側原遠側孩子	由紅轉黑，從 b 增為 b+1

兩側實際黑數已相等，不再需要額外黑。新局部總黑數為 c+b+1，等於原本從 p 開始、含額外黑的有效總數。

若新根 w 紅，它的兩個孩子 p 與原遠側孩子都黑；w 對外又保留 p 的原色，因此不會與外父新增紅紅衝突。令 x=T.root 結束迴圈，最後確保根黑。

D4 數例：完成同一次刪除

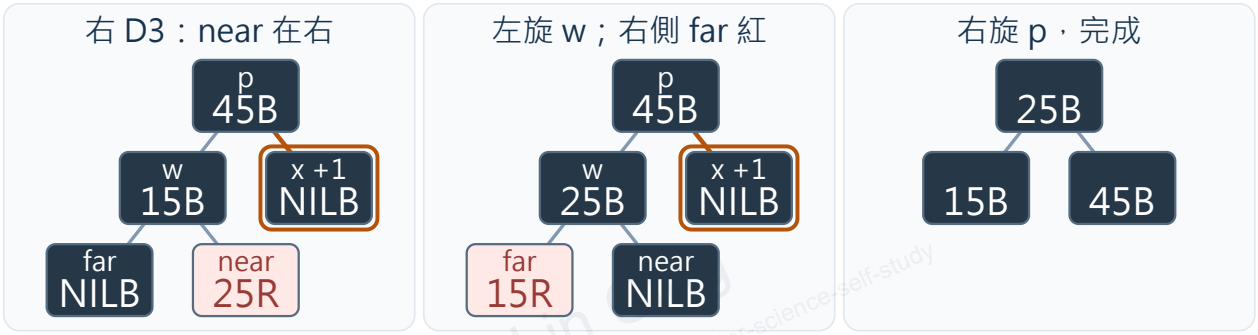
承接 D3 的數例：兄弟 35 繼承父 25 的黑色，25 與遠側孩子 45 也塗黑，再左旋 25。



結果為根 35B、左孩子 25B、右孩子 45B。留下的鍵為 25、35、45；每條根到 NIL 的路徑均有 3 黑，額外黑已完全消除。

右側缺黑：把 D3、D4 完整鏡射

依序插入 15、35、45、25，再刪除 35。後繼 45B 就是右孩子，移接後成為新根；x=NIL 位於 45.right，故必須令 NIL.parent=45。



兄弟為 15B，近側是 25R，遠側是黑 NIL。右側 D3 **左旋 15**，接著右側 D4 **右旋 45**；最後為根 25B、兩個黑孩子 15B 與 45B。近側、遠側是相對於 x 的位置，不是固定的左、右方向。

左側缺黑的例子：逐路加總實際黑數與額外黑

以下每條路**含起點與 NIL**。D3 不改 x=NIL 的位置；D4 才消除 extra。

階段與路徑	實際黑數	extra	有效總數
D3 前：25B → NIL(x)	2	1	3
D3 前：25B → 45B → 35R → NIL	3	0	3
D3 後：25B → 35B → 45R → NIL	3	0	3
D4 後：35B → 25B → NIL	3	0	3
D4 後：35B → 45B → NIL	3	0	3

黑點各加 1、紅點加 0，只有穿過 x 的路再加 extra。未列出的另一側 NIL 路依同一規則也為 3；可對照前後圖逐一驗算。D4 後無須 extra，兩側已各補齊一黑。

終止與總成本：旋轉少，不代表刪除為常數

BST 移接先保住鍵序與所有未刪除的節點身分。移除紅色位置不減黑數；移除黑色位置時，先在 x 記額外黑，使祖先看到的有效黑數維持平衡。

修復步驟	額外黑的位置	是否可能繼續向上
D1、D3	留在原 x	只轉換局部情況
D2	移到父節點	黑父且非根時繼續
D4	消除	結束

D2 每次上移一層，最多 O(log(n+2)) 次。D1 後父節點變紅：若接 D2 就由紅父吸收，否則經 D3／D4 結束。因此 D1、D3、D4 各至多出現一次，**整次刪除至多 3 次基本旋轉**。

各分支已分別證明黑數及紅紅關係的維護；最後塗黑處理紅 x 或黑根的結束情況，紅黑性質全部恢復。刪除最後一個節點則得到 T.root=NIL。

搜尋 z、找後繼、向上修復合計仍需 **O(log(n+2))**。至多三次旋轉只限制結構重排；D2 的傳播與重新著色仍可能經過整條祖先路徑。

用子樹大小支援動態順序統計

資料反覆插入、刪除後，若還要查「第 k 筆」或「這個節點排第幾」，可以在平衡搜尋樹加入**子樹大小 size**。size 包含節點本身，NIL.size=0：

$$\text{size}(x) = 1 + \text{size}(x.left) + \text{size}(x.right).$$

秩（rank）採中序次序的 1-based 位置；select(k) 回傳第 k 個節點。本節數例的鍵互異。若允許重複鍵，可用 (key,identity) 定義全序；只給一個重複值，無法指定唯一節點的秩。

擴充時必須一起說清楚**摘要怎麼算、修改後怎麼修、查詢為何正確**。本節摘要由自身資料與兩個孩子在 O(1) 時間算出，且不依節點顏色；依顏色的摘要，須另算重新著色的維護成本。查詢與維護皆先依樹高 h 分析為 O(h+1)；只有底層維持紅黑性質時，才能換成最壞 O(log(n+2))。加一個欄位不會自動讓普通 BST 平衡。

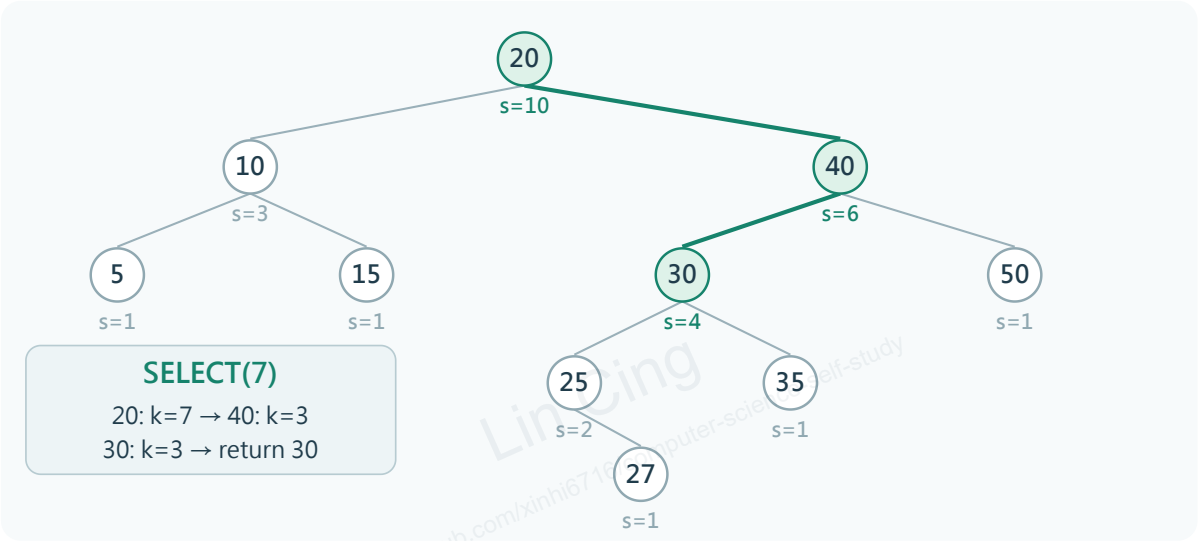
Select：每層都使用目前子樹的局部秩

```
SELECT(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

迴圈不變量是「答案仍為目前 x 子樹中的第 k 個節點」。左子樹佔前 r−1 位，x 佔第 r 位；往左時 k 不變，往右時扣掉左子樹與 x，共 r 筆。

合法的 k 保證選到的子樹非空，且新 k 仍合法。每次下降一層，因此會在有限步驟內找到答案，不會先走到 NIL。公開介面須先拒絕 k=0、k>root.size；空樹沒有合法的 k。

同一棵樹，同時核對鍵值、size 與路徑



圓內是鍵，圓下 s 是包含自身的子樹大小。綠色路徑追蹤 SELECT(7)：20→40→30。接下來的 select、rank 與刪除數例，都以這棵樹為共同起點。

Select(7)：往右才扣掉已略過的前綴

目前鍵	局部 k	left.size	r	選擇與新 k
20	7	3	4	往右到 40；7−4=3
40	3	4	5	往左到 30；仍為 3
30	3	2	3	回傳節點 30

中序為 **5,10,15,20,25,27,30,35,40,50**，第 7 筆確為 30。第一步略過 {5,10,15,20}，剩餘子樹中要找的是第 3 筆；不能把全樹的 7 原封不動拿去和每一層的 r 比較。

往左未略過目前子樹中的任何前綴，所以不扣 k；往右略過的正是 r 筆。這也說明為何 k−r 不會是 0：只有 k>r 時才往右。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank：向上累加互不重疊的前綴

```
RANK(T,x):                    # x is a valid node in T
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

不變量：r 是原節點 x 在目前 u 子樹中的中序位置。u 若是左孩子，父節點與右側資料都排在 x 後，不新增前綴；u 若是右孩子，就加上父節點及其整棵左子樹。

新增的資料都在先前 u 子樹之外，又都排在 x 之前，因此不重複。到達根時，r 就是全樹的秩。此介面接受樹內的有效節點，不能傳 NIL、已刪除的物件或另一棵樹的節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank(27)：哪些祖先需要加，哪些不用？

初始 r=size(27.left)+1=1。

u→父節點	u 位於哪側	新增量	更新後 r
27→25	右側	size(NIL)+1=1	2
25→30	左側	0	2
30→40	左側	0	2
40→20	右側	size(10)+1=4	6

最後得到第 6 筆，和中序次序一致。第一次加入節點 25，第二次加入 {5,10,15,20}；加上原本的 27，共 6 筆。這些前綴彼此不重疊，30、40 則都在 27 後面，不能算進來。

摘要由下往上重算，旋轉也要同步維護

```
PULL(v):                        # v != NIL
    v.size=1+v.left.size+v.right.size
    v.max=max(v.high,v.left.max,v.right.max)
REPAIR-UP(v):
    while v!=NIL:
        PULL(v); v=v.parent
```

此版本同時保存 size 與區間的最大上端點 max；只有順序統計時，可省略 max。NIL.size=0、NIL.max=−∞。

插入新葉時 size=1；接好連結後，從其父節點向根重算。刪除則先完成 BST 移接，再從正確起點沿**最終的父鏈**重算，之後進入紅黑修復。旋轉改好指標後，先 PULL 下移節點，再 PULL 升起節點。

size 與 max 都不依顏色，重新著色不必重算。刪除區間或縮短上端點時，max 可能下降，不能只保留歷史最大值。

刪除後，從哪裡開始修摘要？

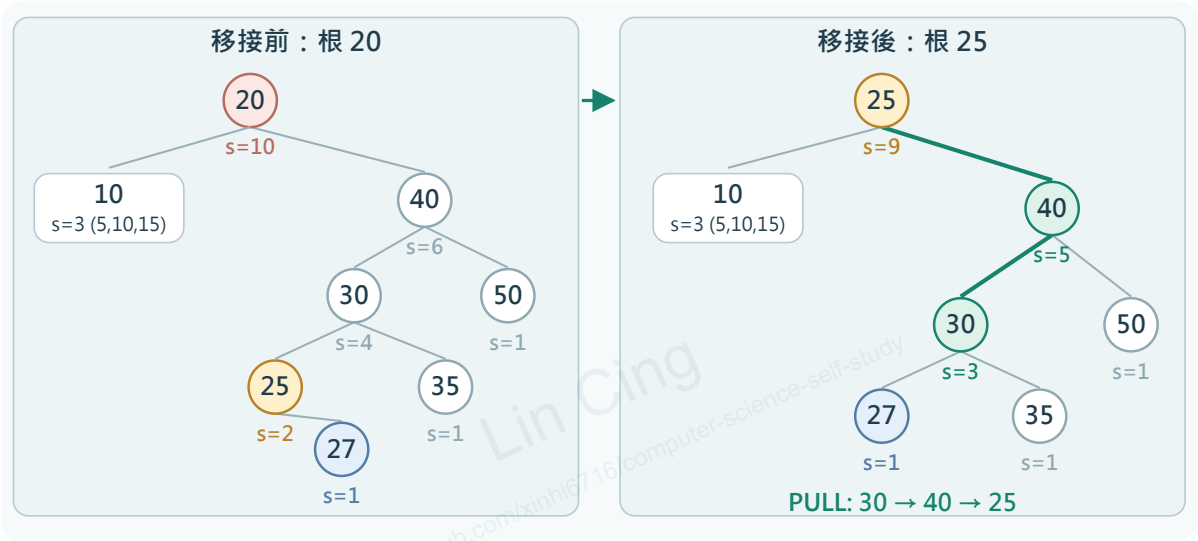
在移接前先保存起點 a；完成全部連結後，執行 REPAIR-UP(a)，再做紅黑顏色修復。

刪除分支	保存的 a	最終父鏈涵蓋的變動
z 至多一個孩子	z 的原父節點	孩子的子樹不變；祖先失去一筆
後繼 y 就是 z.right	y	y 接上新的左子樹，再修祖先
後繼 y 位於更深處	y 的原父節點	原右子樹、搬上去的 y、z 的舊祖先

必須保存真正的節點，不能在釋放 z 後再讀 z.parent。共享的 NIL.parent 也不是所有空連結的共同父節點；修復摘要的 a 應另外保存。若 a=NIL，代表沒有祖先需要重算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除 20、移接 25：連結與摘要一起看



25 的物件身分保留，27 接到 30 的左側；25 再接上原左子樹 10、原右子樹 40。色塊表示移接角色，不是紅黑顏色。圖中先完成 BST 移接與摘要重算，顏色修復依前節程序接續。

先算 30，再算 40，最後才算新根 25

移接前保存 a=30。移接後，這條父鏈為 **30→40→25**，依序重算：

節點	使用已更新的孩子	新 size
30	1+size(27)+size(35)=1+1+1	3
40	1+size(30)+size(50)=1+3+1	5
25	1+size(10)+size(40)=1+3+5	9

25 原本的 size=2 已失效。若先重算 25，會讀到尚未更新的 40.size=6，誤得 10。若只從舊根 20 的父節點開始，起點是 NIL，更會完全漏掉這三個需要更新的節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

旋轉後只修兩格，需要額外理由

左旋時 y=x.right 升起，β=y.left 改接給 x。完成指標改接後，**先 PULL(x)，再 PULL(y)**，因為新 y 的摘要依賴新 x。

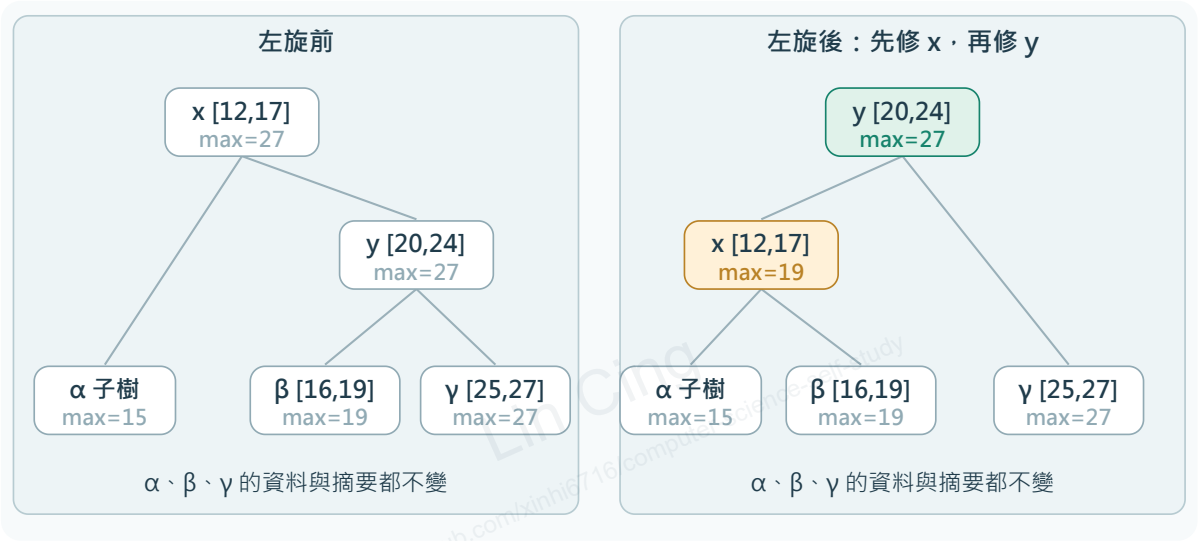
size 與子樹最大 high 都只依「子樹含有哪些資料」。旋轉後，新 y 包含的資料恰為舊 x 的整棵子樹，因此新 y 的摘要等於舊 x，外部祖先看到的摘要不變，只需 O(1) 局部重算。

由孩子算得出，不代表只影響兩格。 height 也有局部公式，旋轉後局部樹高卻可能改變，仍須向祖先傳播。一般摘要每次旋轉最多增加 O(h+1) 工作；紅黑插入最多 2 次、刪除最多 3 次旋轉，所以仍為 O(log(n+2))。

若直接在每個節點保存全域 rank，插入新最小鍵會讓所有舊節點的 rank 加 1，需要 Θ(n) 次明確寫入。選子樹摘要，正是為了把更新留在少數祖先路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一組區間左旋：x.max 下降，整體 max 不變



先算 x.max=max(17,15,19)=19，再算 y.max=max(24,19,27)=27。新 y 仍涵蓋原局部子樹的全部資料；先前最大上端點 27 沒有消失，只是內部連結改變。

區間樹：依下端點排序，保存最大上端點

本節使用閉區間 [l,h]，允許 l=h 的單點區間。兩區間 i、q 重疊，當且僅當：

$$i.low \le q.high \quad \text{且} \quad q.low \le i.high.$$

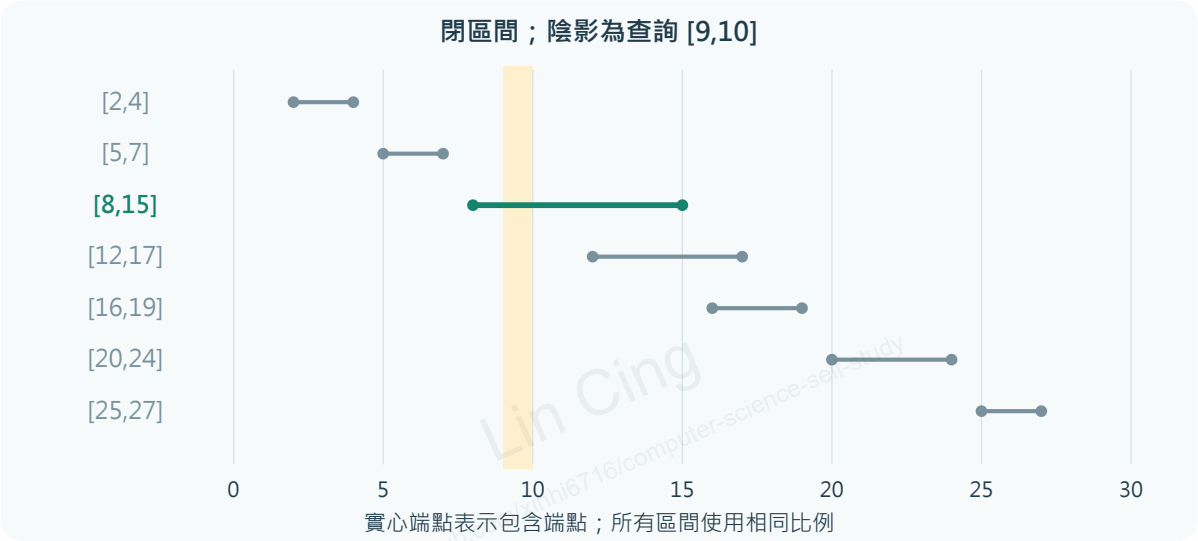
也就是排除 i 完全在 q 左方或右方的兩種情況。

BST 依下端點 low 排序；同 low 可再用物件識別碼決定先後，仍有 left.low≤x.low≤right.low。每節點保存

$$x.max = \max\{x.high,\ x.left.max,\ x.right.max\}, \qquad NIL.max = -\infty.$$

max 是整棵子樹的最大 high，不是目前節點的 high，更不是最大的 low。數例的根 [12,17] 雖然 high=17，max 卻是右側後代 [25,27] 提供的 27。

把區間放在同一條座標軸上



陰影是查詢 [9,10]。只有 [8,15] 穿過陰影；[5,7] 在它左側，[12,17] 在右側。端點位置與後面的搜尋樹使用的資料完全一致。

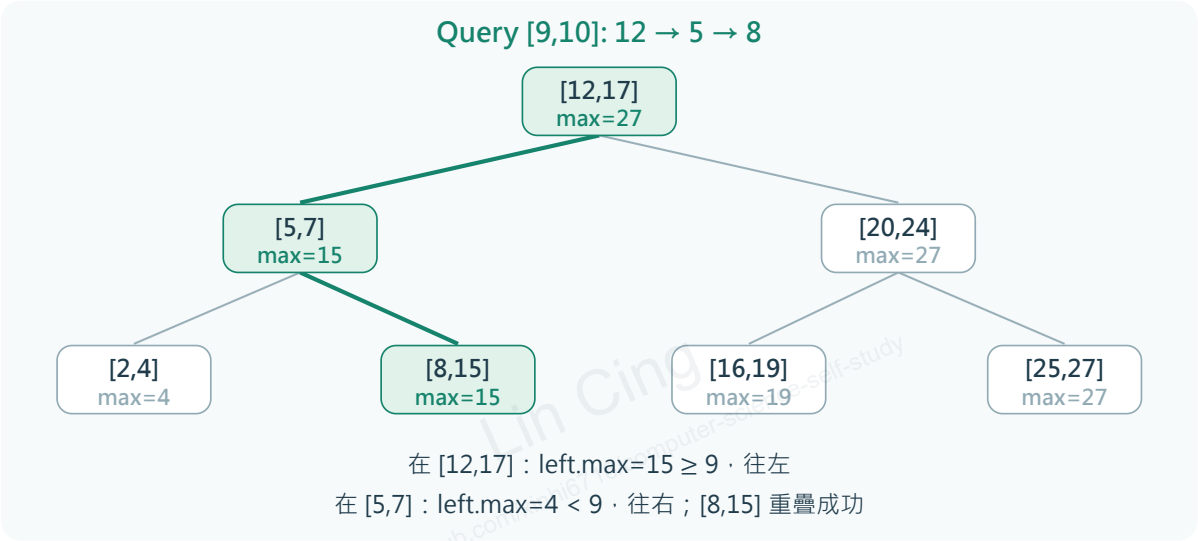
重疊搜尋：每一步只選一棵子樹

```
INTERVAL-SEARCH(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

找到目前節點重疊就回傳；否則依左子樹的 max 決定方向。此程序只找**任意一個**重疊區間，不保證 low 最小，也不列出全部答案。

真正需要證明的是：若目前子樹有答案，選出的那一側仍保留至少一個答案。left.max≥q.low 本身不保證左側一定重疊，不能省略後面的正確性論證。

同一棵區間樹：綠線追蹤查詢 [9,10]



第一步讀到左側 max=15≥9；到 [5,7] 時，改讀其左側 max=4<9，因此轉向右側，找到 [8,15]。每一步讀的是「當前節點左孩子的子樹摘要」。

三次查詢：成功與失敗都能逐步核對

查詢	訪問順序	分支依據／結果
[9,10]	[12,17]→[5,7]→[8,15]	15≥9 往左；4<9 往右；命中
[18,18]	[12,17]→[20,24]→[16,19]	15<18 往右；19≥18 往左；命中
[28,29]	[12,17]→[20,24]→[25,27]→NIL	左側 max 依序不足；最後無解

查詢 [28,29] 走到 [25,27] 時，目前區間的 high=27<28，且左右孩子皆空，故最後回傳 NIL。**目前節點不重疊**與**某棵子樹可排除**是不同判斷，不能只看 x.high 就決定剪掉左側。

往右：先證明左側完全沒有答案

維持不變量：「若原樹存在重疊區間，目前 x 子樹也至少有一個。」起點為根，顯然成立；若 x 重疊就直接回傳。

當左孩子為 NIL，左側沒有候選。否則若 left.max<q.low，對任何左側後代 j 都有

$$j.high \le left.max < q.low.$$

因此它們全在查詢左方，不可能重疊。加上 x 已經不重疊，若仍有答案，只能在右側，往右保持不變量。

這裡用的是**整棵左子樹的 high 上界**。若只將根改成 [12,13]，查詢 [14,14]，雖然根的 high=13<14，左側 [8,15] 仍是答案；單看根的 high 不足以剪枝。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

往左：為什麼不必回頭搜尋右側？

若 left.max≥q.low，左子樹中必有一個區間 j，其 high **恰等於**這個 max。左側若本來就有答案，走左當然安全。

若左側完全無解，j.high≥q.low 卻仍不重疊，只可能是 j.low>q.high。對任一右側後代 k，BST 的鍵序給出

$$k.low \ge x.low \ge j.low > q.high.$$

所以右側也全在查詢右方；加上 x 不重疊，此時目前子樹根本沒有答案。因此不論左側有沒有解，都不會丟掉原本存在的所有解。

走到 NIL 時，由不變量得知原樹也無解。每次下降一層，成本為 O(h+1)，紅黑底層則為 O(log(n+2))。關鍵是**精確 max 對應實際存在的區間**，不是隨意給的上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

過大的舊 max，也可能讓搜尋漏掉答案

將 [8,15] 改成 [8,27] 後再刪掉。左子樹只剩 [5,7]、[2,4]，精確 max 應為 7；若錯留 27，查詢 [22,22]：

位置	使用的比較	錯誤路徑
[12,17]	舊 left.max=27≥22	往左，放棄有答案的右側
[5,7]	left.max=4<22	往空的右孩子
NIL	迴圈結束	錯誤回報找不到

真正答案是右側 [20,24]。精確 left.max=7 會讓第一步往右找到它。**舊上界 27 沒有左側區間作見證**，因此上一頁「左無解則右亦無解」的論證不再成立。修改後必須由下往上重新取 max，允許摘要下降。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

端點慣例與資料修改，也屬於演算法規格

區間慣例	非空條件	overlap(i,q)	左側可能有解
閉區間 [l,h]	l≤h	i.low≤q.high 且 q.low≤i.high	left.max≥q.low
半開 [l,h)	l<h	i.low<q.high 且 q.low<i.high	left.max>q.low

[4,5] 與 [5,7] 在 5 重疊；[4,5) 與 [5,7) 則沒有交集。[5,5) 是空集合，不能把單點閉區間的結論直接套過來。

修改 low 會改變排序鍵，應刪除舊資料再插入；不能留在原位置只改數值。只改 high 且仍滿足非空條件時，鍵序不變，但須從該節點往上重算 max；縮短區間也不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

找一個、找最左與列出全部，是不同問題

本節搜尋只回傳任一重疊區間。若要列出全部，可在 left.max≥q.low 時訪問左側、檢查 x，再於 x.low≤q.high 且 right.max≥q.low 時訪問右側。每個訪問節點 O(1)，最壞仍可為 O(n)，不能直接沿用單一路徑的對數界。

另一個容易完整分析的方法，是反覆搜尋、暫時刪除命中資料，記錄後最後插回。若共有 k 個答案，包含 k 次成功及最後一次失敗搜尋，總成本為 **O((k+1)log(n+2))**，另需 O(k) 儲存結果。這會暫時修改集合，必須符合介面契約。

要得到 O(log n+k)，還需要另一個演算法或更細的訪問節點數證明。size、max 可同時維護，但這裡的 rank 是依 (low,identity) 排序的位置，並非依 high 排序，也不是重疊區間的數量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree：把一頁內的空間換成較少的層數

一個節點保存多個已排序的鍵，以及分隔各鍵範圍的子指標。搜尋先在頁內選出範圍，再讀取下一頁；成本要分成**頁存取（I/O）與頁內比較、搬移（CPU）**。

本節採鍵互異的集合。每個鍵與它的資料或 record pointer 一起搬移；內節點也能保存實際資料。重複插入回傳 EXISTS，不新增一筆。這與 B+ tree「資料集中在葉、父節點另存 separator」的表示不同。

一節點能否放進一頁，取決於頁大小、鍵與資料指標大小、子指標及頁頭。以下的 I/O 分析假設這個配置已成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小度 t：先固定每種節點的容量

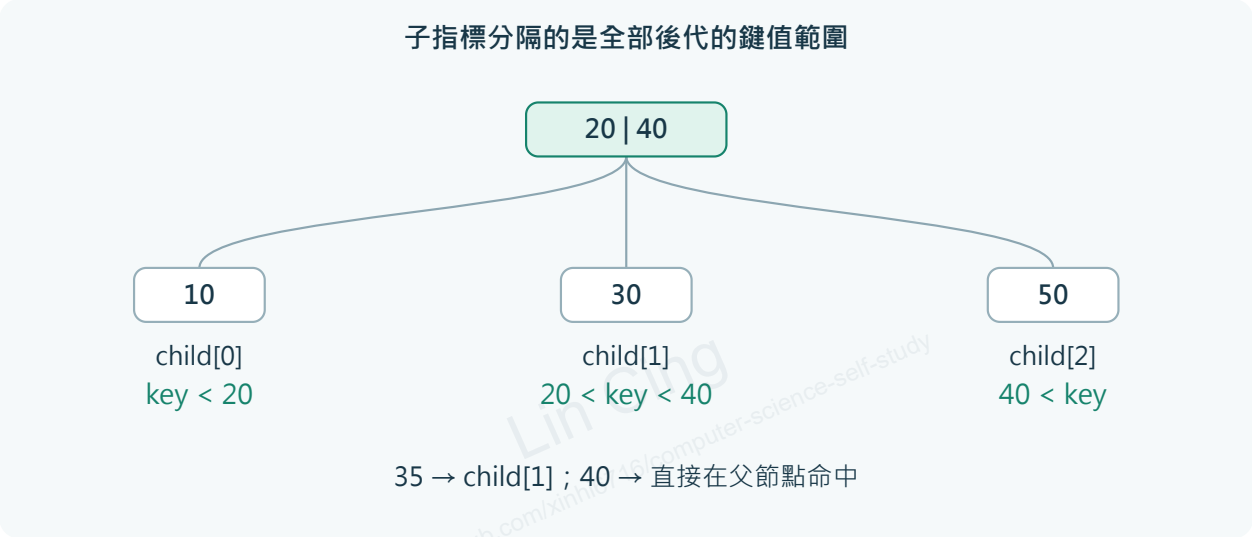
固定整數 t≥2；鍵陣列與子指標陣列皆採 0-based。

節點種類	鍵數 k	子指標數
非根節點	t−1≤k≤2t−1	葉為 0；內節點恰為 k+1
非空根	1≤k≤2t−1	葉為 0；內節點至少 2
空樹	零鍵葉根	0

所有葉深度相同；根深度為 0，葉深度為高度 h。非空樹 h=0 表示資料全部放在根；空樹也約定 h=0，但下文的非空高度定理不能直接套 n=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每個子指標負責整段鍵值範圍



鍵 $a_0<\cdots<a_{k-1}$ 將整個子樹分成 k+1 段。範圍限制適用於**全部後代**，不是只比較子節點的第一個鍵。搜尋 35 要走中間；搜尋 40 直接在父命中。

order m 與 minimum degree t 不能混用

本節採用的定義	非根內節點的分支數	非根鍵數
order m：最大分支數為 m	$\lceil m/2 \rceil \cdots m$	$\lceil m/2 \rceil - 1 \cdots m-1$
minimum degree t	$t \cdots 2t$	$t-1 \cdots 2t-1$

m=2t 時容量才完全對應。 m=3 是 2-3 tree，不能令 t=1.5，也不能令 t=2：後者是最多 4 個分支的 2-3-4 tree。

一般「order」的定義並非各書一致；先確認它指最大分支數還是其他容量。上表固定採最大分支數，不從名稱猜公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先分裂與溢位後分裂：前提不同

本節插入先分裂已有 $2t-1$ 鍵的滿節點：提升一鍵，左右各 $t-1$ 鍵，兩邊立即合法。

order m 的溢位後分裂則先讓節點暫存 m 鍵，再提升中位鍵。例如 m=3：葉 [2,8] 插 5，暫成 [2,5,8]，提升 5，留下 [2]、[8]。

若 m=3 卻在滿葉 [2,8] 插入前就分裂，提升任何一鍵都會讓一側零鍵，違反下限 1。因此**容量公式、修復時機與正確性證明必須配套**。刪除同樣有「先修好再下降」與「刪後處理不足」兩種安排。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

高度上界：先數最稀疏的合法樹

假設 n≥1。高度 h 的根至少有 1 鍵；h≥1 時至少有 2 個子節點。深度 i≥1 至少有 $2t^{i-1}$ 個節點，每個至少 t−1 鍵，所以

$$n \geq 1 + \sum_{i=1}^{h} 2t^{i-1}(t-1) = 2t^h - 1.$$

移項並取以 t 為底的對數：

$$h \leq \log_t \frac{n+1}{2}.$$

h=0 時總和為空，得到 n≥1，仍符合。根的最少鍵數是 1，不能誤把根也算成 t−1 鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

高度下界與十億筆資料的頁數

每節點最多 2t−1 鍵、最多 2t 個分支，因此

$$n \leq (2t-1)\sum_{i=0}^{h}(2t)^i = (2t)^{h+1} - 1.$$

取 t=100、n=10^9：h=2 最多 7,999,999 鍵，不夠；h=3 最多 1,599,999,999 鍵，容量足夠。另一方面 h=4 最少 199,999,999 鍵；h=5 最少 19,999,999,999 鍵，太多。故合法樹只能有 h=3 或 4。

一次搜尋走 4 或 5 個節點；若根常駐記憶體、其他路徑頁皆未快取，便是 **3 或 4 次頁讀取**。若根也不在記憶體，再多讀一頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋：頁內定位，再走唯一可能的子樹

```
SEARCH(x,k):
    i=lower_bound(x.keys,k)       // first key >= k, or len(keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x.children is empty: return NOT_FOUND
    read x.children[i] if not cached
    return SEARCH(x.children[i],k)
```

若沒有相等鍵，全域範圍限制保證只有 child i 可能包含 k；其餘全部排除。每次下降一層，最後在葉命中或失敗，故終止且正確。lower_bound 可用線性掃描或二分搜尋；兩者不改變頁路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

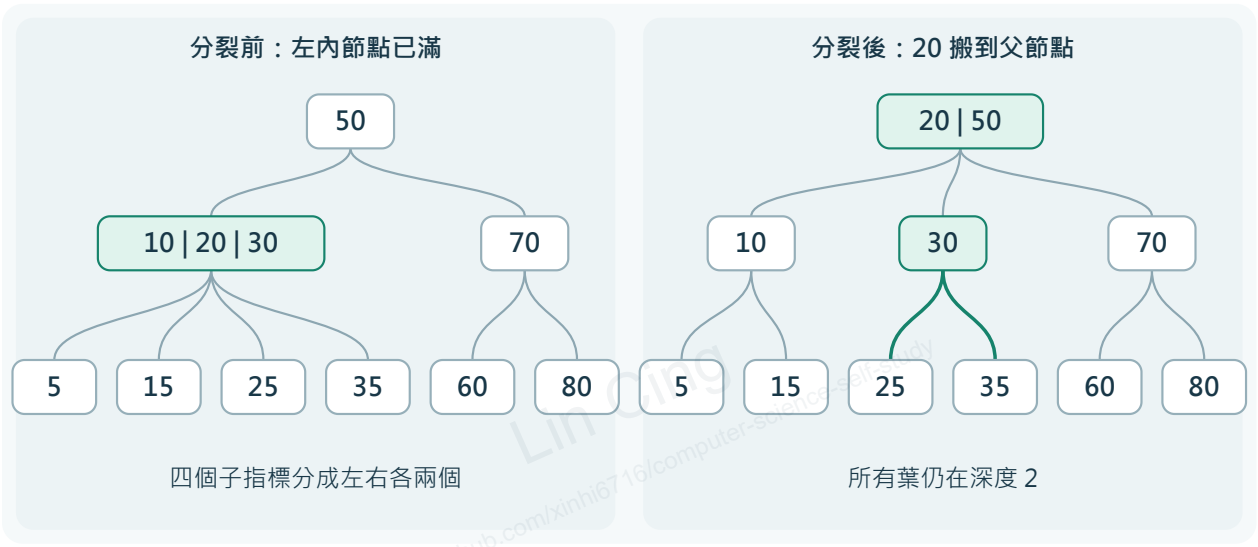
SPLIT：提升中位鍵，也切開子指標

```
SPLIT(p,i):                       // p nonfull; child i full
    y=p.children[i]; old=y.keys; kids=y.children
    median=old[t-1]
    z=Node(keys=old[t:], children=kids[t:] if kids else [])
    y.keys=old[:t-1]
    y.children=kids[:t] if kids else []
    p.keys.insert(i,median); p.children.insert(i+1,z)
    write y,z,p
```

切片採左含右不含，且 old、kids 代表改動前的陣列。左右各 t−1 鍵；若為內節點，各有 t 個子指標。median 搬到父節點，**不在子節點保留副本**。父原先未滿，增加 1 鍵仍合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分裂內節點：四個舊子樹不能遺失

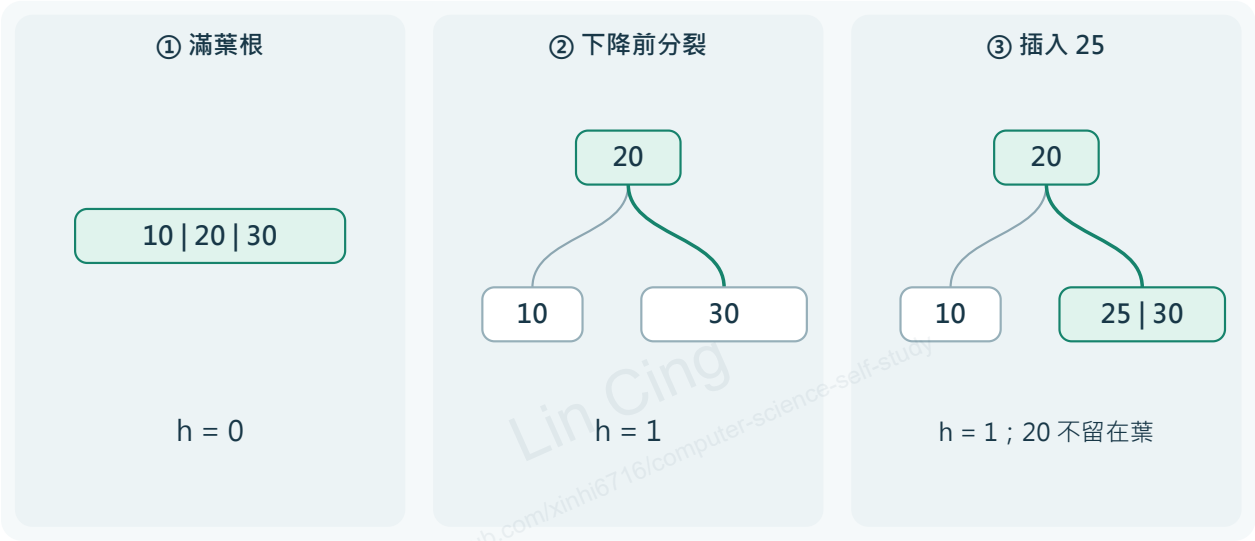


t=2，原父 [50] 的左子節點為 [10,20,30]。提升 20 後，父為 [20,50]；[10] 接 [5]、[15]，[30] 接 [25]、[35]。原父的右子樹 [70] 不變。所有葉仍深度 2。

插入：下降時永遠選未滿節點

```
INSERT(k):
    if SEARCH(root,k) succeeds: return EXISTS
    if len(root.keys)==2*t-1:
        root=Node([], [root]); SPLIT(root,0)
    x=root
    while x.children is not empty:
        i=lower_bound(x.keys,k)
        if len(x.children[i].keys)==2*t-1:
            SPLIT(x,i)
            if k>x.keys[i]: i+=1
        x=x.children[i]
    insert k in sorted order into x.keys; write x
    return INSERTED
```

根分裂：高度從頂端增加



t=2，滿葉根 [10,20,30] 插 25：先提升 20，再因 25>20 選右葉，得到 [25,30]。只有根分裂新增一層；所有舊葉一起加深，葉深度仍相等。

插入正確性：為何不會向上連鎖補救？

迴圈開始時 x 未滿。根若原本滿，先建立新根並分裂，故初始化成立。若下一個子節點滿，因 x 未滿，可安全把一鍵提升到 x；分裂後的兩個子節點各有 t−1 鍵，選中的一側仍未滿。

每次下降一層；到葉後只增加一鍵，最多從 2t−2 變 2t−1，不溢位。SPLIT 保存全部鍵及子樹範圍，最後插入的 k 也落在搜尋選定的範圍，因此鍵集合恰增加 k。

與溢位後分裂相比，這裡把修復提前到下降之前，所以無須回到已離開的父頁處理溢位。插入前查重另走一條路徑，仍不改變漸近成本。

刪除：先替即將下降的子問題留一格餘裕

非根節點通常只需 t−1 鍵；本節刪除加強條件：**每次遞迴進入非根節點時，先保證它至少有 t 鍵**。這樣在該層失去一鍵，仍有合法的 t−1 鍵。

情況	處理
在葉找到 k	直接刪除
在內節點找到 k，左側至少 t 鍵	用前驅取代，從左側刪前驅
左側不足 t、右側至少 t	用後繼取代，從右側刪後繼
相鄰兩側皆 t−1	兩側與父鍵合併，再刪 k

未找到 k 時，也要先把目標子節點修到至少 t 鍵再下降。根可例外較少；若最後變成零鍵內根，改用唯一子節點當新根。

向左兄弟借：父鍵下降，兄弟最大鍵上升

```
BORROW-LEFT(p,i):                // i>0
    y=p.children[i]; s=p.children[i-1]
    y.keys.insert(0,p.keys[i-1])
    p.keys[i-1]=s.keys.pop()
    if s.children: y.children.insert(0,s.children.pop())
    write s,y,p
```

前提：y 有 t−1 鍵、s 至少 t 鍵；兩者皆在同一層。s 的最右子樹介於新、舊分隔鍵之間，必須一起移到 y 的最左側。完成後 y 有 t 鍵，s 仍至少 t−1 鍵；父鍵數不變。

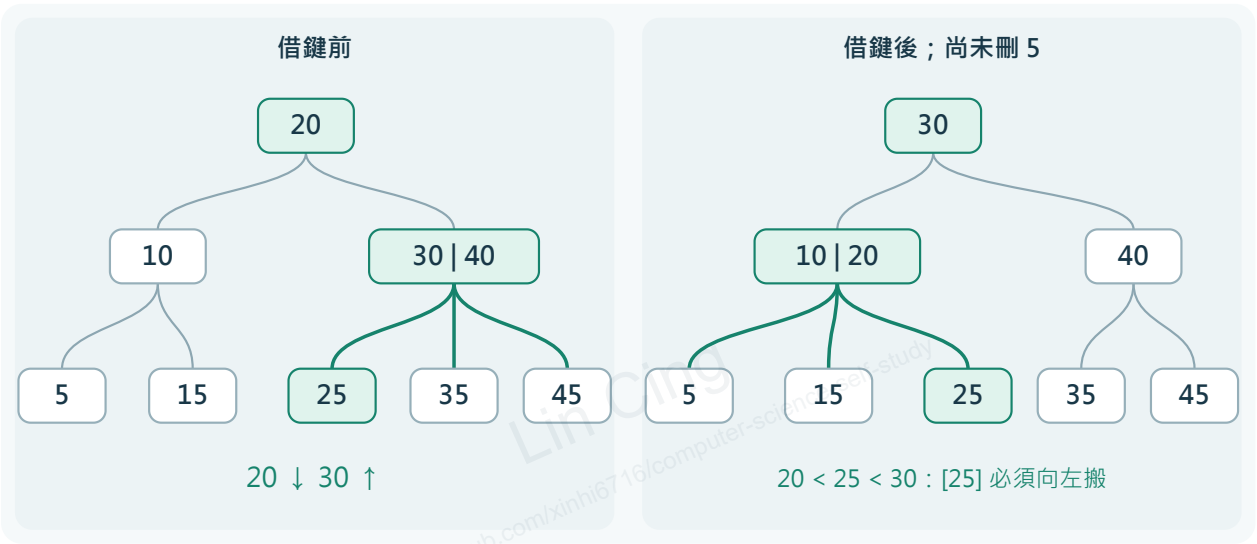
向右兄弟借：子指標也要移到正確一側

```
BORROW-RIGHT(p,i):                // i<len(p.keys)
    y=p.children[i]; s=p.children[i+1]
    y.keys.append(p.keys[i])
    p.keys[i]=s.keys.pop(0)
    if s.children: y.children.append(s.children.pop(0))
    write s,y,p
```

前提同樣是 y 有 t−1 鍵、s 至少 t 鍵。s 的最左子樹移到 y 最右側。只搬鍵、不搬該子指標，會使整段鍵值範圍落在錯誤子樹；即使每個節點內仍排序，也已不再是合法搜尋樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

借鍵數例：25 的子樹為何必須向左搬？



t=2，要刪 5。左內節點 [10] 只有 1 鍵，右兄弟 [30,40] 有 2 鍵。先借：20 下降，30 上升；[25] 介於 20 與 30，必須成為 [10,20] 的最後一棵子樹。圖中只完成下降前修復，尚未刪 5。

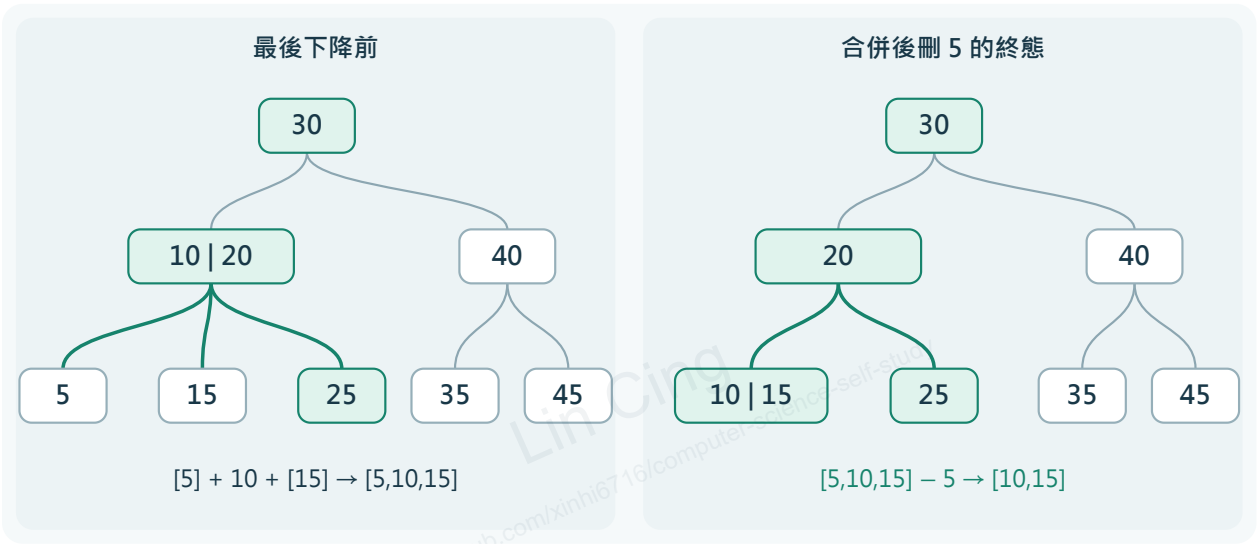
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MERGE：父分隔鍵也是合併內容的一部分

```
MERGE(p,i):                      // adjacent children both t-1 keys
    y=p.children[i]; z=p.children[i+1]
    y.keys.append(p.keys.pop(i))
    y.keys.extend(z.keys)
    y.children.extend(z.children)
    p.children.pop(i+1)
    write y,p; free z
    return y
```

新 y 有 (t−1)+1+(t−1)=2t−1 鍵，恰好滿而不溢位；若為內節點，有 t+t=2t 個子指標。父少一鍵及一個子指標。非根父進入時至少 t 鍵，現在仍有 t−1；若空的是根，最後統一縮高。

接續刪 5：先合併兩個最小葉



在 [10,20] 內，目標葉 [5] 與右兄弟 [15] 各只有 1 鍵。將父鍵 10 一起合入 [5,10,15]，再刪 5 得 [10,15]。左內節點剩 [20]，連 [10,15]、[25]；根 [30] 和右內節點 [40] 不變。中序恰少 5，所有葉仍深度 2。

內節點命中：前驅與後繼都搬整筆資料

t=2 的初始樹：根及兩個葉	刪除	結果
根 [20]；左 [5,10]、右 [30]	20	前驅 10 上移；根 [10]，葉 [5]、[30]
根 [20]；左 [5]、右 [30,40]	20	後繼 30 上移；根 [30]，葉 [5]、[40]

取前驅時，沿左子樹的最右子指標到葉，取最後一鍵；後繼則沿右子樹的最左子指標到葉，取第一鍵。選取後必須從原子樹刪掉那筆資料，不能讓同一鍵在父、葉各留一份。

記錄指標也要同步搬移；只改鍵、保留父的舊資料，會讓查詢回傳錯誤的 record。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

根縮減：只有根能拿掉整層



根 [20] 與葉 [5]、[30] 合成 [5,20,30]，再刪 20 得 [5,30]。舊根零鍵且只有一個子節點，用該子節點取代，h 從 1 變 0。一般內節點不能照做，否則只縮短一側路徑，破壞葉等深度。

刪除主程序：命中鍵的三種內部情況

```
DELETE-NODE(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if not x.children: erase x.keys[i]; write x; return FOUND
        L=x.children[i]; R=x.children[i+1]
        if len(L.keys)>=t:
            q=MAX-RECORD(L); DELETE-NODE(L,q.key); x.keys[i]=q
        elif len(R.keys)>=t:
            q=MIN-RECORD(R); DELETE-NODE(R,q.key); x.keys[i]=q
        else:
            y=MERGE(x,i); DELETE-NODE(y,k)
        write x; return FOUND
    return DELETE-DESCEND(x,i,k)  // next card
```

此處 keys 陣列存整筆記錄，比較使用記錄的 key；q 是保存下來的記錄值或穩定指標，不可在刪除原位置後讀取已釋放的物件。

刪除下降：借鍵優先，合併時修正索引

```
DELETE-DESCEND(x,i,k):
    if not x.children: return NOT_FOUND
    if len(x.children[i].keys)==t-1:
        if i>0 and len(x.children[i-1].keys)>=t:
            BORROW-LEFT(x,i)
        elif i<len(x.keys) and len(x.children[i+1].keys)>=t:
            BORROW-RIGHT(x,i)
        elif i<len(x.keys): MERGE(x,i)
        else: MERGE(x,i-1); i-=1
    return DELETE-NODE(x.children[i],k)
```

往左合併時，原 child i 被併入 child i−1，必須 i−=1。對外 DELETE(k) 呼叫 DELETE-NODE(root,k)，**不論 FOUND 或 NOT_FOUND，最後都檢查零鍵內根並縮高**，然後回傳原本的搜尋結果。

找不到鍵，也可能合法地改變樹形

根 [20]、葉 [5]、[30]，t=2，要求刪 99。先選右子節點；右葉只有 t−1=1 鍵，左兄弟也無法借，所以先合併成 [5,20,30]，索引從 1 改為 0。

在新葉找不到 99，回傳 NOT_FOUND；外層仍須縮掉零鍵內根。最後是葉根 [5,20,30]，集合完全不變，高度從 1 降到 0。

NOT_FOUND 表示集合沒有刪掉元素，不保證樹形完全不變。 若 API 要求失敗時連樹形都不動，可在開始前另做存在性搜尋，並將這條額外路徑算入成本。空樹與刪最後一鍵後皆保留零鍵葉根。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除的一般證明與兩種成本

以剩餘高度歸納：葉直接刪除正確；內部替換後遞迴刪掉前驅或後繼，集合只少 k。借鍵與合併保持全域鍵序、子指標數及葉等深度，並讓下一個子問題至少有 t 鍵。非根父即使因合併少一鍵，仍保有 t−1。根例外由外層縮高處理。每次遞迴嚴格下降，故終止。

每層只碰常數個相鄰頁，I/O 為 $O(h+1)=O(1+\log_t(n+1))$。頁內陣列搬移最多 O(t)，插入、刪除 CPU 上界 O(t(h+1))。前驅／後繼若先找再刪，多走的路徑仍總計 O(h+1)，不是每層都重走整棵樹。

純搜尋在頁內二分，可用 O((h+1)log t) 比較；這不會把插入、刪除的陣列搬移也降為 O(log t)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heap：把整理留到取出最小值時

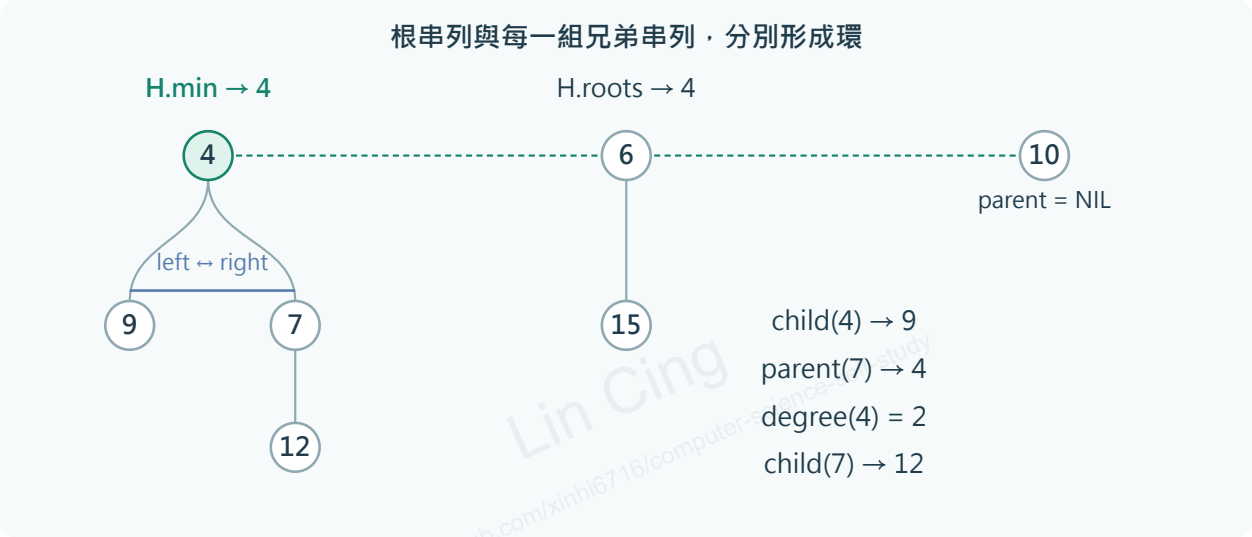
最小堆積只要求父鍵不大於子鍵，不要求不同子樹彼此排序。Fibonacci heap 保留一串滿足堆積序的樹：插入或合併時先接上根串列，直到 EXTRACT-MIN 才合併度數相同的根。

度數（degree）是直接孩子的個數，不是樹高。 經過切割，樹不再保證是完整二項樹。後面將證明最大度數為 O(log(n+1))；這個結論不代表根到葉的路徑也只有對數長度。

DECREASE-KEY(H,x,k) 與 DELETE(H,x) 都要求給定有效的節點參照（handle）。只給鍵值、再尋找對應節點，可能需要走遍整個堆積；這筆搜尋成本另計。相同鍵可以共存，但節點身分不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

指標各管一件事：根串列、子串列與最小值



圖中根為4、6、10；4的孩子是9、7，7還接著12。根串列入口與最小值指標用途不同；每組兄弟另有自己的環狀串列。

欄位保存什麼？標記從哪一次連結開始算？

節點保存 `key`、`parent`、`child`、`left`、`right`、`degree`、`mark`。`child` 指向任一孩子，其兄弟以環狀雙向串列相連；只有一個孩子時，該節點的 left、right 都指向自己。H.roots 是根串列入口，H.min 指向最小根，H.n 記錄總節點數；空堆積的兩個入口皆為 NIL。

標記 記錄非根節點在「這一次接到目前父節點之後」，是否已失去一個孩子。 第一次失去孩子先標記，第二次就切離父節點。這不是終生累積次數；節點切成根、之後重新接到另一個父節點時，紀錄重新開始。

本節統一讓根的 mark=false；提升為根，或重新連結成孩子時，都清除標記。這項慣例也用於後面的勢能計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向環串列：能常數時間合併的實際原因

刪除已知節點 x，只要接 x.left.right=x.right、x.right.left=x.left；若 x 是入口就換入口，若是獨生元素就設 NIL，最後讓 x 左右指回自己。只用 O(1) 次指標更新。

兩個非空環入口 a、b，先存 al=a.left、bl=b.left，再接 al.right=b、b.left=al、bl.right=a、a.left=bl，即把兩環接成一環。空環直接取另一入口。

以下 ATTACH-ROOT(H,x) 將已脫離原串列的 x 接到根串列，設 parent=NIL、mark=false；REMOVE-CHILD(p,x) 維護 p.child、環指標及 度數。這些基本操作都不搜尋鍵，也不走訪全部兄弟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

INSERT、MINIMUM 與破壞式 UNION

操作	完整處理	實際成本
MAKE	roots=min=NIL，n=0	O(1)
INSERT(x)	新節點 degree=0、child=NIL；接根、比較 min、n 加一	O(1)
MINIMUM	回 min；空堆積回 NIL	O(1)
UNION(H1,H2)	接兩根環，取兩個非空 min 的較小者，n 相加	O(1)

UNION 消耗兩個互不重疊的舊堆積；結果取得全部節點，原入口不可再拿來獨立操作。H1=H2 不允許。兩者皆空時結果仍空。這個界不包含逐節點重寫「屬於哪個 heap」欄位；若實作要求那種更新，不能沿用 O(1) 合併界。

EXTRACT-MIN：先提升孩子，再整理根

```
EXTRACT-MIN(H):
    z=H.min
    if z==NIL: return NIL
    while z.child!=NIL:
        c=z.child
        REMOVE-CHILD(z,c)
        ATTACH-ROOT(H,c)
    REMOVE-ROOT(H,z)
    H.n-=1
    if H.roots==NIL: H.min=NIL
    else: CONSOLIDATE(H)
    return z
```

每次取目前 孩子 再移除，避免一邊修改環串列、一邊用失效的 next 繼續走。所有提升節點清 父節點 與 標記；z 被隔離後才交回呼叫者。若只有一個根但它還有孩子，提升後堆積仍非空，不能只憑「原來一個根」就回空。

CONSOLIDATE：讓每個 度數 最後只留一個根

```
CONSOLIDATE(H):
    A=empty degree table; H.min=NIL
    while H.roots!=NIL:
        x=POP-ROOT(H)              // isolate one root
        d=x.degree
        while A[d]!=NIL:           // absent slot means NIL
            y=A[d]; A[d]=NIL
            if y.key<x.key: swap(x,y)
            LINK-ISOLATED(y,x)      // y.parent=x; y.mark=false
            d=x.degree
        A[d]=x
    for each non-NIL x in A:
        ATTACH-ROOT(H,x)
        if H.min==NIL or x.key<H.min.key: H.min=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LINK 與 度數 表的實作契約

LINK-ISOLATED 把同 度數 的兩個獨立根合成一棵：較大鍵 y 成為 x 的 孩子，x.degree 加一；相等鍵選任一根。A 可用按需延伸的陣列，最多 O(D(n)+1) 槽。上面採逐根取出的版本，避免重複處理已被連結的舊根。

```
LINK-ISOLATED(y,x):
    require x!=y; x.degree==y.degree
    require x.key<=y.key; both roots isolated
    y.parent=x; y.mark=false
    splice y into x.child circular list
    x.degree+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整理根的不變量與終止理由

每次外層開始時，尚未處理的根留在 H.roots，已處理節點則分布在 A 指向的互不重疊子樹中；A 每個槽至多存一棵 度數 等於槽號的樹。所有原節點恰好出現在這兩部分之一。

內層維持 d=x.degree；遇到同 度數 的 y 就連結，節點集合與堆積序不變，兩根變一根，d 增一。原有根數有限，每次 link 都讓森林根數減一，故不可能無限循環。

外層每次取掉一個未處理根，因此結束時 A 包含全部剩餘樹，且 度數 各異。重建根串列並逐根比較最小值，得到正確 H.min。注意「度數 各異」只在整理完成後保證；INSERT、UNION、CUT 都可再次造成重複。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

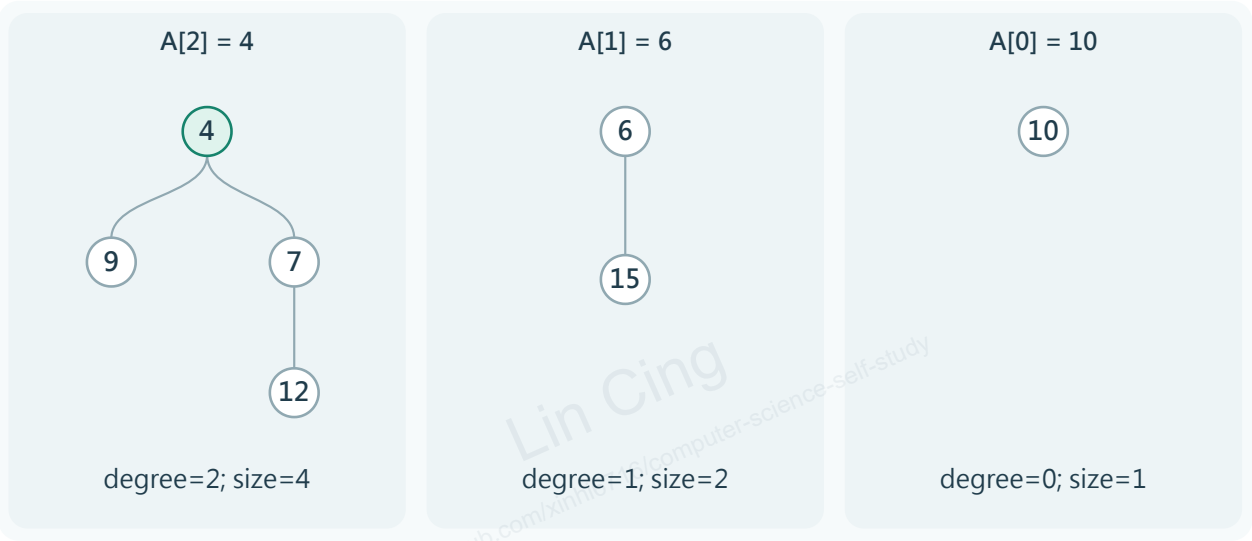
數例：四次 link，逐槽追蹤整理過程

從八個單節點根 1、4、9、7、12、6、15、10 開始，先取出最小根 1，再按 4、9、7、12、6、15、10 的順序整理；它們的初始度數皆為 0。

取出根	這一輪的 link	非空 A 槽（度數:root）
4	無	0:4
9	9 掛到 4	1:4
7	無	0:7，1:4
12	12 掛到 7；再把 7 掛到 4	2:4
6	無	0:6，2:4
15	15 掛到 6	1:6，2:4
10	無	0:10，1:6，2:4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

數例終態：樹形、度數 與根數一起核對



根 度數 依序為2、1、0；節點數4+2+1=7，min=4。綠色根4是最小值，其餘根不必按鍵值排序。

同一數例：根數下降如何釋出勢能

根 4 的孩子是 9、7，7 的孩子是 12；根 6 的孩子是 15；根 10 沒有孩子。三個根的度數分別為 2、1、0，總節點數為 4+2+1=7，每條父子邊都滿足父鍵≤子鍵。

階段	根數 r	標記數 m	勢能 K(r+2m)
取出 1 之前：八個單節點根	8	0	8K
移除 1、完成四次連結之後	3	0	3K

這次勢能下降 5K：移除最小根使根數少一，再由四次連結少四。它把「先前插入時存下的額度」接回後面的攤銷公式；**5K 是勢能變化，不是五條機器指令的實際成本。**

本例恰為未經切割的二項樹。發生 CUT 後，度數 d 不再保證有 2^d 個節點；後面的通用下界改為 F_{d+2}。

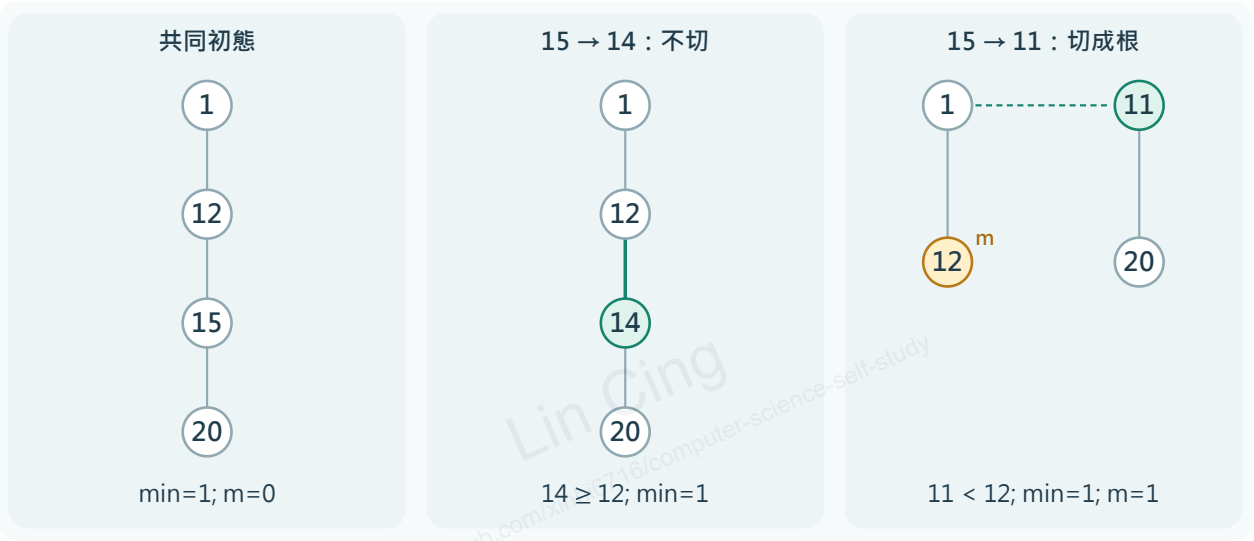
DECREASE-KEY：只有違反父子順序才切

```
DECREASE-KEY(H,x,k):
    if k>x.key: error
    x.key=k; p=x.parent
    if p!=NIL and x.key<p.key:
        CUT(H,x,p)
        CASCADING-CUT(H,p)
    if x.key<H.min.key: H.min=x

CUT(H,x,p):
    REMOVE-CHILD(p,x)       // degree decreases by one
    ATTACH-ROOT(H,x)       // parent=NIL; mark=false
```

只降低 x 的鍵，不會破壞 x 與孩子的順序；可能壞掉的只有 父節點→x。若 x 已是根，或新鍵仍≥父鍵，皆不切。min 只可能保留舊值或改成 x，其他節點鍵值沒有改變。

數例：15→14 不動；15→11 帶著子樹上來



兩個情況都從左圖開始。降低至14不切；降低至11則連孩子20一起提升，並標記第一次失去孩子的非根12。

兩種遞減鍵值，從同一初態分開追蹤

根 1 的孩子 12 有孩子 15，15 又有孩子 20。兩個情況各自從這個初態開始：

新鍵	與父鍵 12 比較	指標與標記的變化
14	14≥12	不切割、不改標記
11	11<12	連同孩子 20 一起切成新根；12 若原本未標記，便標記

切割搬動的是**以該節點為根的整棵子樹**，不是只取出一個鍵；20 仍是新鍵 11 的孩子。第二種情況中，12 是非根且首次失去孩子，因此留下標記，根 1 不受影響。

兩種結果的最小值都仍是 1。若降低的是既有根，例如把根 3 改為 0，只更新最小值指標，不會切割。

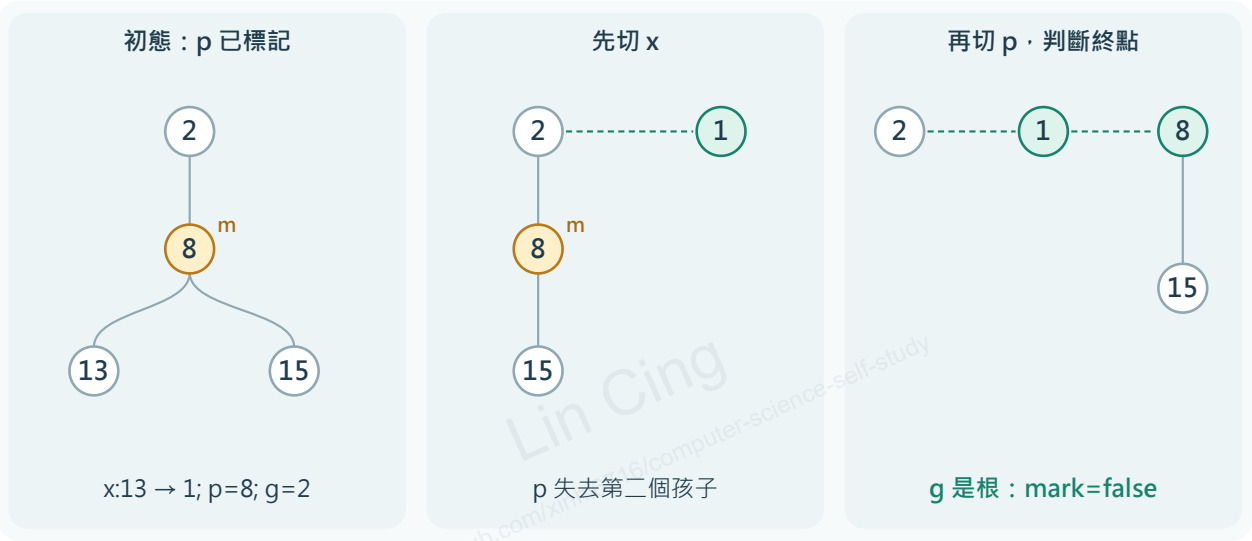
CASCADING-CUT：第一次失去孩子先標記，第二次才切走

```
CASCADING-CUT(H,y):
    while y.parent!=NIL:
        p=y.parent
        if y.mark==false:
            y.mark=true
            return
        CUT(H,y,p)
        y=p
    return                      // root: stop, do not mark
```

呼叫時 y 剛失去一個 孩子。未標記表示這是本次成為 孩子 後的第一次損失，留下記號便停；已標記表示第二次損失，把 y 切成根並清 標記，父節點 p 也因此失去孩子，所以繼續處理 p。

堆積序仍正確：切掉父子邊不會讓剩餘邊失序。級聯的作用是限制非根能失去多少孩子，保住 度數 與子樹大小之間的關係，而不只是修復鍵值順序。

級聯分支一：祖父是根，就停在根



先切x，再切已標記的p。到g時它已是根，所以停止；不新增g的標記。p原有的孩子15仍跟著p。

到根就停：根串列與勢能逐項核對

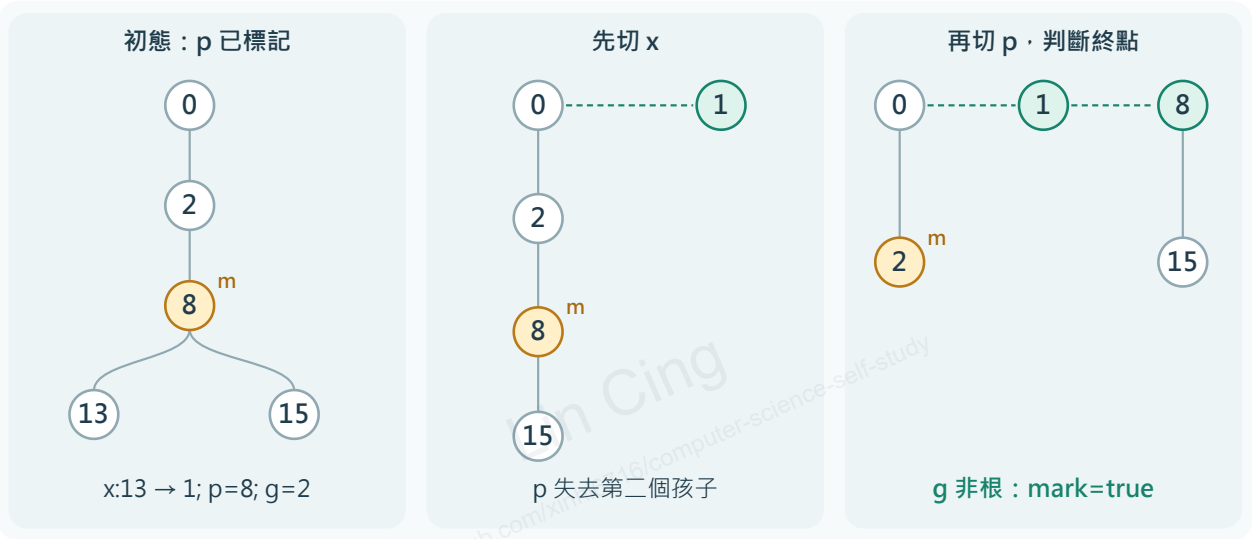
初態只有根 g=2，其孩子 p=8 已標記；p 的孩子為 x=13、s=15。p 的標記表示它在這次成為孩子之後，先前已失去另一個孩子。現在把 x 從 13 降到 1。

先切 x，再切已標記的 p。到 g 時，g.parent=NIL，因此停止，**不把根 g 標記**。最後根為 2、1、8；8 仍帶著孩子 15，最小值改為 1。

狀態	根數 r	標記數 m	未縮放勢能 r+2m
操作前	1	1	3
切 x、切 p，抵達根 g 後	3	0	3

新增兩個根增加 2，清除 p 的標記減少 2，勢能淨變化為 0。這裡先核對 r+2m；實際採用的勢能還會乘上一個固定常數 K。

級聯分支二：祖父非根且未標記，才留下記號



這次g還有父節點q，且原本未標記。切x、p之後，標記g而停；因為根q=0仍最小，min不變。

尚未到根：清除 p 的標記，再標記 g

在前一個局部結構上方接根 q=0，令 g=2 為未標記非根，而 p=8 仍已標記。把 x=13 降到 1：先切 x、再切 p；抵達 g 時，g.parent=q 且 g.mark=false，所以標記 g 後停止。

最後根為 q=0、x=1、p=8；q 仍接 g=2，p 仍接 s=15，最小值保持 0。

狀態	根數 r	標記數 m	未縮放勢能 r+2m
操作前	1	1	3
p 清標記、g 新增標記後	3	1	5

若 g 原本也已標記，就把 g 一起切走，繼續處理 q。**停止位置由父節點與標記共同決定**；「祖父被標記」只適用於祖父非根且先前未標記的分支。

同一個勢能同時支付所有操作

取固定常數 K>0，對所有現存且互不重疊的堆積定義

$$\Phi = K \sum_{H} \big(r(H) + 2m(H) \big), \qquad \hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

K 只需選一次，大到能支付以下常數時間指標操作，之後不可隨操作更換。由空堆積開始 Φ₀=0 且始終非負，故 Σcᵢ≤Σĉᵢ。若從已有森林開始，總界須加初始 Φ₀。

INSERT 增一根、不增 標記，因此 ΔΦ=K，攤銷仍 O(1)。MINIMUM 與 MAKE 不改勢能。UNION 將兩個舊堆積替換成一個，根數與標記數相加，總勢能不變，因此實際與攤銷都 O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何一串級聯切割只收常數攤銷成本？

若沒有切割，實際與勢能變化都 O(1)。否則令 c≥1 為總切割次數，包括最初切 x；其餘 c−1 次都切掉已標記的非根節點，並清除它們的 標記。結尾至多新標記一個非根，x 的舊標記若存在也被清除，因此

$$\Delta r = c, \qquad \Delta m \le 1 - (c-1) = 2 - c,$$
$$\Delta \Phi \le K(4 - c).$$

存在固定 a，使實際成本≤a(c+1)。選 K≥a，得到

$$\hat{c} \le a(c+1) + K(4-c) \le a + 4K = O(1).$$

這是對任何合法操作序列的保證，不需要隨機輸入。單次實際切割仍可能沿長路徑花線性時間；O(1) 是攤銷界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EXTRACT-MIN 的成本：根多時，用先前存的勢能付款

設操作前有 n 節點、r 個根，最大 度數 上界為 D=D(n)。取出最小根後，待整理根數 R≤r−1+D。每次 link 讓根數減一，所以全部 link 次數≤R；處理孩子、掃根、清空與重建 度數 表，實際成本 ≤a(r+D+1)。

整理後至多 D+1 個根。提升與 link 都只清 標記、不新增，因此

$$\Delta\Phi \le K(D+1-r).$$

$$\hat{c} \le a(r+D+1) + K(D+1-r) \le (a+K)(D+1)$$

最後一步用同一個 K≥a；空堆積與單節點另為 O(1)。這裡先得到 O(D+1)，接下來必須證明 D=O(log(n+1)) 才能宣稱 O(log(n+1)) 攤銷時間。連續插入 n 個根後第一次取出，實際可達 Θ(n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

度數 證明的關鍵：孩子最多只少一個孩子

令 x 現在有 d 個孩子，按它們**最近一次接到 x 的時間**排序為 $y_1, \dots, y_d$。對 i≥2，在 y_i 接上當時，目前仍在的較早孩子 $y_1,\cdots,y_{i-1}$ 都已在 x 下面，因此 x 當時 度數 至少 i−1。

LINK 只合併相同 度數 的根，所以 y_i 當時也至少有 i−1 個孩子。之後若失去第二個孩子就會被切離 x；既然 y_i 現在仍是 x 的孩子，至多只少一個。因此

$$\deg(y_1) \ge 0, \qquad \deg(y_i) \ge i-2 \quad (2 \le i \le d).$$

這不是說兄弟的目前 度數 已排序，也不是說每個非根終生只能失去一個孩子；切成根再被連結後，歷史會重新計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從孩子下界推到 Fibonacci 子樹大小

對子樹節點數做強歸納，證明任意 degree=d 的節點 x 都有 size(x)≥F_{d+2}。孩子子樹比 x 小，可套歸納假設；F 數列非遞減，配合上一頁的 度數 下界。

$$F_0 = 0, \quad F_1 = 1, \quad F_j = F_{j-1} + F_{j-2} \ (j \geq 2).$$

d=0 時 size≥1=F₂；d=1 時至少自己與一個孩子，size≥2=F₃。d≥2 時，第一個孩子至少一節點，其餘依序給出

$$\text{size}(x) \geq 2 + \sum_{i=2}^{d} F_i = F_{d+2}.$$

最後的恆等式由 $\sum_{i=0}^{d} F_i$=F_{d+2}−1 得到：d=0 為 0=F₂−1；歸納時加上 F_{d+1}，再用 Fibonacci 遞迴即可。這個證法以子樹大小歸納，不偷用尚未證明的「最小大小隨 度數 單調」。

完成 度數 界：指數大小換成對數 度數

令 ϕ=(1+√5)/2，ϕ²=ϕ+1。證明 F_{d+2}≥ϕᵈ：d=0 為 1≥1，d=1 為 2≥ϕ；d≥2 時

$$F_{d+2} = F_{d+1} + F_d \geq \varphi^{d-1} + \varphi^{d-2} = \varphi^d.$$

所以對 n≥1 的堆積中任意節點，n≥size(x)≥F_{d+2}≥ϕᵈ，得到 $d \leq \lfloor \log_\varphi n \rfloor$。因此最大 度數 為 O(log(n+1))，度數 表大小、整理後根數與 EXTRACT-MIN 攤銷成本皆為 O(log(n+1))。

度數 d	0	1	2	3	4	5	6
子樹節點數下界：F_{d+2}	1	2	3	5	8	13	21

這是保證下界，不宣稱每個 d 都能由某個特定操作序列恰好達到等號。空堆積不取 log 0，另行處理。

DELETE：指定節點必須真的成為唯一最小

DELETE(H,x) 可先將 x 的比較鍵改成比所有一般鍵都小的專用哨兵，再 EXTRACT-MIN。原本若為非根，DECREASE-KEY 會正確處理父節點失去孩子及級聯切割；取出後連帶提升 x 的所有孩子並整理。

不能只寫「改成負無限大」卻允許其他節點也有相同比較值；否則 min 可能仍指向別的節點，刪錯身分。可把比較鍵擴充為 (tag,key)，平常 tag=1，僅本次目標 tag=0，確保它唯一最小；刪完再把回傳節點的原鍵還原。

以同一勢能把兩操作相加，中間勢能相消，攤銷成本為 O(1)+O(log(n+1))。直接拆除內節點的另一種實作也可以正確，但必須另處理孩子提升、父的 標記 與級聯；不能只把孩子放到根串列就算完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

操作成本總表：先確認輸入與計費範圍

操作	單次實際上界	攤銷上界
MAKE、MINIMUM、INSERT、UNION	O(1)	O(1)
DECREASE-KEY（已知 handle）	O(n+1)	O(1)
EXTRACT-MIN、DELETE（已知 handle）	O(n+1)	O(log(n+2))
只給任意鍵的 SEARCH	O(n+1)	堆積序不會自動改善搜尋界

表中使用 log(n+2)，讓 n=0 的空堆積也保留常數成本；非空時與慣用的 O(log n) 同階。DELETE、DECREASE-KEY 要求有效節點，因此不能在空堆積提供這種 handle；空 EXTRACT-MIN 則合法，直接回傳 NIL。

n 為相關堆積的節點數。UNION 要求兩個輸入互不重疊，並消耗舊入口。成本不包含配置器、逐節點所有權重標記，或尋找 handle 的額外工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把操作次數換成圖演算法的總成本

圖演算法若有 O(V) 次插入／取出、O(E) 次已知 handle 的遞減鍵值，便得到 O(E+V log(V+1)) 的堆積總成本。是否值得實作仍取決於常數、記憶體與存取模式；漸近界不是實測速度。

例如 V=1000、E=8000，且每個頂點至多插入與取出各一次，每條邊至多觸發一次遞減鍵值：

工作	最多次數	Fibonacci heap 的總攤銷上界
INSERT	1000	O(V)
EXTRACT-MIN	1000	O(V log(V+1))
DECREASE-KEY	8000	O(E)

相加得到 O(E+V log(V+1))；若使用一般二元堆積，對應上界為 O((E+V)log(V+1))。數字是操作次數上限，並非毫秒或可直接比較的精確成本；兩種結構的常數並不相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

利用整數位元，換一種搜尋模型

紅黑樹只比較鍵值；vEB（van Emde Boas tree）還使用鍵的二進位表示。固定集合 $S \subseteq \{0, \ldots, u-1\}$，$u = 2^m$、$m \geq 1$，**n=|S| 與鍵值範圍（universe）大小 u 是不同參數**。本節不存重複鍵。

假設 word RAM 的一個 word 可容納鍵與位址；位移、遮罩、比較、陣列索引皆為常數時間。這不適用於任意精度整數的位元成本模型。

目標是最壞 $O(1 + \log\log u)$ 的查找、插入、刪除、前驅與後繼，最小／最大值為 O(1)。這不違反比較排序下界：演算法使用了額外的整數運算，也須支付初始化與記憶體成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先看少了什麼：直接定址與遞迴分組

直接定址的 bit vector 能在 O(1) 時間查找、插入與刪除，但若 $S = \{0, u-1\}$，從 0 向右掃描後繼可能花 Θ(u)。在 bit vector 上建立二元摘要樹，可把此查詢降為 O(log u)。

把 universe 分成約 $\sqrt{u}$ 組，再把各組與非空組號都遞迴表示，仍**不保證** O(log log u)：若先遞迴找組號，再遞迴找該組最小值，會有

$$T(u) = 2T(\sqrt{u}) + O(1), \quad R(m) = 2R(m/2) + O(1) = \Theta(m).$$

因此成本仍是 Θ(log u)。關鍵不只在縮小 universe，還要讓每層最多留下**一次非定值成本的遞迴**；min/max 快取就是為此而設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

奇數位元寬度：高位與低位不能各除一半

令 $\ell = \lfloor m/2 \rfloor$、$h = \lceil m/2 \rceil$，$L = 2^{\ell}$、$H = 2^{h}$，所以 u=HL。對 u>2：

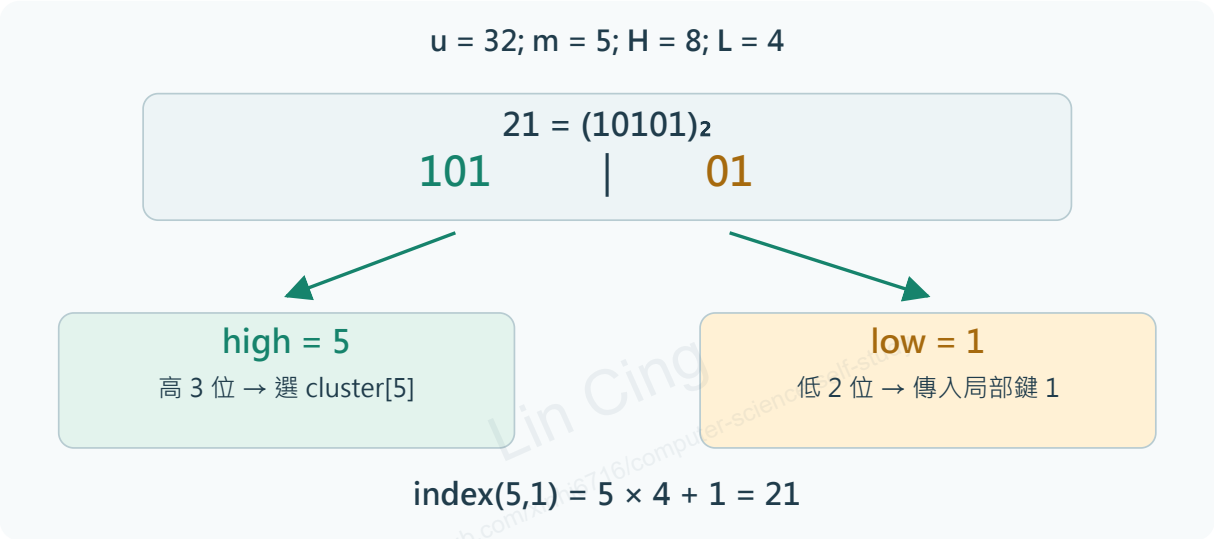
$$\text{high}(x) = \lfloor x/L \rfloor, \quad \text{low}(x) = x \bmod L, \quad \text{index}(i,j) = iL + j.$$

有 H 個分組（cluster），每組的 universe 大小為 L；摘要（summary）存 cluster 編號，其 universe 大小為 H。**每層都依該節點的 u 重新計算**，不能沿用頂層遮罩。

u=32 時，m=5、h=3、ℓ=2、H=8、L=4。$x = 21 = (10101)_2$ 切成 $101 \mid 01$，所以 high=5、low=1，還原為 5×4+1=21。u=2 直接處理，不建立下一層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一個鍵的兩個座標



high 選 cluster；low 是傳給該 cluster 的鍵。summary 裡的 5 表示「第 5 組非空」，不是原集合真的存了鍵 5。這兩種資料不能混成同一組鍵值。

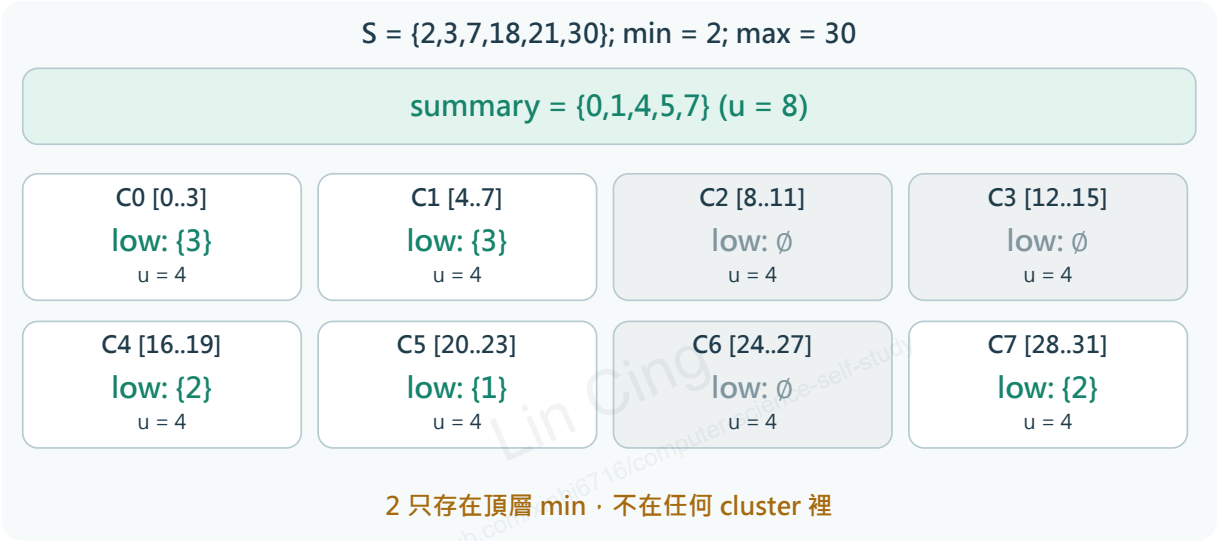
表示不變量：最小值不放進任何 cluster

每個節點保存 u、min、max；u>2 時另有 summary 與 cluster[0..H−1]。採用預先建立的完整結構，空節點的 min=max=NIL。

非空時，min 是該集合最小值；所有其他鍵依 high／low 分到 cluster。**min 本身不存入 cluster**，所以單一元素時所有 cluster 與 summary 都空。當至少兩鍵時，max 也存在某個 cluster。

summary 恰存非空 cluster 的編號：$i \in \text{summary} \iff C_i.\min \neq \text{NIL}$。因此 min=max 可在 O(1) 判定單一元素；向空節點插入只需設定 min=max=x，不必往下填滿一條路。

u=32 的集合：原鍵、組內鍵與摘要分開看



原集合為 {2,3,7,18,21,30}。頂層 min=2、max=30；C_0 只存 low=3，不再存 2。summary={0,1,4,5,7}。圖只展開頂層；每個 C_i 與 summary 仍各自遵守相同不變量。

查找與邊界：先保證參數在 universe 內

```
MEMBER(V,x):                  # require 0<=x<V.u
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return MEMBER(V.cluster[high(x)],low(x))
MINIMUM(V): return V.min
MAXIMUM(V): return V.max
```

high／low 皆使用 V 的尺寸。查找沿一個 cluster 向下；快取命中時提早返回。NIL 必須是獨立狀態，不能用合法鍵 0 代替。

本節後繼定義為集合中**嚴格大於 x** 的最小鍵，前驅為嚴格小於 x 的最大鍵；x 不必在集合中，但須在 universe 內。越界參數由公開介面拒絕，避免索引負值或 H。

後繼：用 max 決定遞迴走哪一邊

```
SUCCESSOR(V,x):
    if V.min==NIL: return NIL
    if x<V.min: return V.min
    if x>=V.max: return NIL
    if V.u==2: return 1
    i=high(x); j=low(x); C=V.cluster[i]
    if C.max!=NIL and j<C.max:
        return index(i,SUCCESSOR(C,j))
    k=SUCCESSOR(V.summary,i)
    if k==NIL: return NIL
    return index(k,V.cluster[k].min)
```

若同組 max 比 j 大，後繼就在同組；否則同組沒有答案，直接問 summary 的下一個非空組。**不用先遞迴失敗一次再換組**。找到下一組後讀 min 是 O(1)。

後繼數例：同組與跨組共用一份狀態



同組先看快取 max；跨組只問 summary。兩條路徑互斥。

逐步核對後繼查詢的座標

以 {2,3,7,18,21,30} 為準。SUCCESSOR(3)：i=0、j=3，C_0.max=3，不可留在同組；summary 的後繼組號是 1，讀 C_1.min=3，還原 1×4+3=7。

SUCCESSOR(20)：i=5、j=0，C_5.max=1>0，因此只遞迴 C_5，得 offset=1，還原為 21。20 不在集合中仍可查詢。SUCCESSOR(30)=NIL，因為沒有嚴格更大的鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前驅：不能忘記獨立保存的 min

```
PREDECESSOR(V,x):
    if V.min==NIL: return NIL
    if x>V.max: return V.max
    if x<=V.min: return NIL
    if V.u==2: return 0
    i=high(x); j=low(x); C=V.cluster[i]
    if C.min!=NIL and j>C.min:
        return index(i,PREDECESSOR(C,j))
    k=PREDECESSOR(V.summary,i)
    if k==NIL: return V.min
    return index(k,V.cluster[k].max)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後的 k=NIL 分支不是無條件回傳 NIL：前面已知 x>V.min，且更小的非空 cluster 不存在，所以答案正是未放入 cluster 的 min。例如 PREDECESSOR(3)=2；即使 C_0 裡沒有更小的 low，也不能漏掉頂層的 2。

插入：換出舊 min，並只更新必要的摘要

```
INSERT-ABSENT(V,x):          # require x not in V
    if V.min==NIL: V.min=V.max=x; return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        i=high(x); j=low(x); C=V.cluster[i]
        if C.min==NIL:
            INSERT-ABSENT(V.summary,i)
            C.min=C.max=j
        else: INSERT-ABSENT(C,j)
    if x>V.max: V.max=x
```

新鍵若小於 min，先交換，接著插入的是**舊 min**。空 cluster 的組號尚未在 summary，故遞迴插入組號，再以 O(1) 建立單一元素 cluster；非空 cluster 的組號已存在，只遞迴該 cluster。兩條昂貴路徑互斥。

插入 1：新 min 在外，舊 min 回到 C_0

由原集合 {2,3,7,18,21,30} 插入 1：

步驟	頂層／局部狀態	要維護的不變量
交換	min=1；待插入 x=2	新 min 不放 cluster
分組	high(2)=0、low(2)=2	用頂層 L=4
進 C_0	原 C_0={3}，插入後 {2,3}	C_0 自己的 min=2、max=3
摘要	仍為 {0,1,4,5,7}	C_0 原本就非空
結束	max=30	原最大值未改

C_0 的 u=4，所以它的下一層改用 L=2：舊局部 min=3 被放到其 C_1 的 low=1。不可把頂層 low=2 再當成「整棵樹的鍵 2」而混用座標。

刪除：先處理單一元素與 u=2

DELETE-PRESENT 的前提是 x 確實存在。先處理以下兩種狀態，再進入下一頁的一般情況：

```
DELETE-PRESENT(V,x):
    if V.min==V.max:
        V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=1-x; return
    DELETE-GENERAL(V,x)
```

第一個分支中 min=max 必為有效鍵，因為呼叫契約排除了空集合；刪除後所有 cluster 原本就空。u=2 且不是單一元素時，集合必為 {0,1}，刪除 x 後只剩 1−x。

若直接對空集合或不存在的鍵呼叫，這段程式可能清掉合法資料。公開介面先 MEMBER，未找到就回傳 NOT_FOUND；不能把這項前提藏在解答裡。

一般刪除：提升下一個 min，再清掉其 cluster 副本

```
DELETE-GENERAL(V,x):         # u>2; at least two keys
    if x==V.min:
        k=V.summary.min
        x=index(k,V.cluster[k].min); V.min=x
    i=high(x); j=low(x); C=V.cluster[i]
    DELETE-PRESENT(C,j)
    if C.min==NIL:
        DELETE-PRESENT(V.summary,i)
        if x==V.max:
            k=V.summary.max
            if k==NIL: V.max=V.min
            else: V.max=index(k,V.cluster[k].max)
    elif x==V.max:
        V.max=index(i,C.max)
```

刪除時的 x，為什麼會改成新 min？

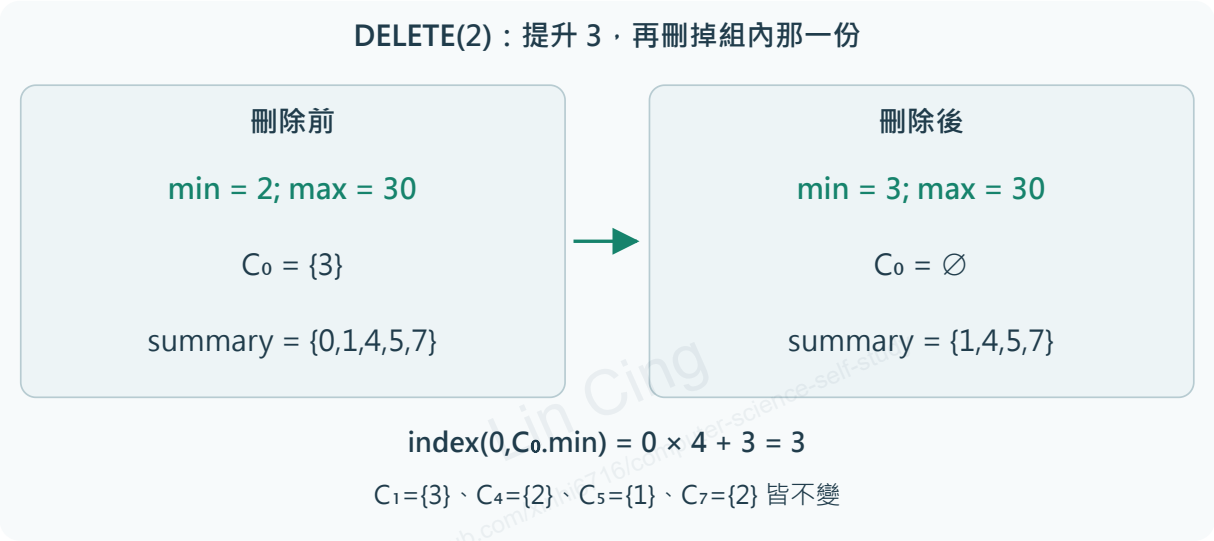
變數 x 在提升時改成新 min 的值；接下來刪的是它在 cluster 中的那份表示。先更新 cluster 與 summary，再重算 max，不能讀取已經失效的舊最大組號。

若集合只有 {2,3} 且刪除 2：提升後 x=3、min=3；C_0 的 3 被刪空，summary 也清空。此時 x 等於舊 max=3，所以設定 max=min=3，留下合法的單一元素集合。

若還有 7、18、21、30，提升後的 x=3 不等於 max=30，最大值便不變。這裡比較的是程式目前的 x，不能在某一行偷換回原刪除參數 2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除 2：提升 3，不是把所有鍵往前搬



3 提升成頂層 min，C_0 清空，summary 刪掉組號 0。

逐步核對提升與摘要刪除

從原集合重新開始：summary.min=0、C_0.min=3，故新 min=0×4+3=3。刪除 C_0 裡的 3 後 C_0 變空，再從 summary 刪掉組號 0，得到 {1,4,5,7}。max=30 不變。

新 min=3 **只在頂層保存**；若還留在 C_0，就破壞表示不變量，也會讓後續刪除／單一元素判斷錯誤。此例中 cluster 的刪除是單一元素分支，只花 O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除最大值與最後兩個元素

仍由原集合出發，連續刪除 30、21、18、7、3、2：

刪除鍵	刪除後 min／max	頂層 summary
30	2／21	{0,1,4,5}
21	2／18	{0,1,4}
18	2／7	{0,1}
7	2／3	{0}
3	2／2	空
2	NIL／NIL	空

刪除 3 後沒有非空 cluster，max 必須回到獨立的 min=2；不能從空 summary 取得組號再索引陣列。刪除最後的 2 只清 min/max，已經沒有其他表示需要清理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除有兩次呼叫，為何仍只剩一條昂貴路徑？

一般刪除確實可能先呼叫 DELETE(cluster)，再呼叫 DELETE(summary)。但第二次只有在 cluster 被刪空時才發生，而刪空表示該 cluster **原本只有一個元素**，第一次呼叫立即走 O(1) 的單一元素分支。

cluster 原本狀態	cluster 刪除成本	是否刪 summary 組號
單一元素	O(1)	是，可能遞迴
至少兩個元素	可能遞迴	否，刪除後仍非空

因此成本是 $\max\{T(L), T(H)\} + O(1)$，不是 $T(L) + T(H) + O(1)$。這是由表示不變量與分支條件推出的**最壞界**，不需要用機率或攤銷分析掩蓋第二次呼叫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用位元寬度解遞迴式

設 T(u) 為任一操作的最壞上界，$R(m) = T(2^m)$。每層只有一次非定值遞迴，子節點位元數最多為 $\lceil m/2 \rceil$：

$$R(m) \le R(\lceil m/2 \rceil) + c, \qquad R(1) = O(1).$$

對 m≥2，$\lceil m/2 \rceil \le 2m/3$；所以至多 $\lceil \log_{3/2} m \rceil$ 個遞迴層後抵達基例。每層 O(1)，得到 $O(1 + \log m) = O(1 + \log \log u)$。

m=5 的最長位元路徑為 5→3→2→1；cluster 路徑也可能是 5→2→1。**取決於走 summary 或 cluster**，不能把兩種 universe 大小都寫成精確的 √u。u=2 時成本是常數，而不是 log log 2=0 次工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正確性：每個操作如何維持集合表示

以 m 作歸納。m=1 可列出空、{0}、{1}、{0,1} 四種合法集合。較大節點的集合由獨立 min 與各 cluster 還原值的聯集唯一決定。

查詢：min/max 守住邊界；同組存在答案時，其餘更高組不可能更小。否則 summary 找到下一個非空組，再讀該組 min。前驅另外納入未放進組內的 min。

插入：新 min 與舊 min 交換，只改變保存位置，不遺失鍵；將待插入鍵加入正確 cluster，且只在該組首次非空時加入摘要。

刪除：刪 min 時提升目前最小的 cluster 鍵，再刪其組內副本；其餘情況只刪目標鍵。空組同步退出摘要，max 由更新後狀態重建。歸納假設保證子操作正確，因此每步恰好實現目標集合變化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建立空結構也有成本：Θ(u) 空間與初始化

本節採預先配置的完整樹，不把配置藏進 O(1) 的空節點插入。u>2 時，H 個 cluster 各有 universe L，另有 universe H 的 summary；本層指標與欄位成本至多 cH：

$$P(u) \le HP(L) + P(H) + cH, \qquad P(2) \le p_0.$$

可用強歸納證明 $P(u) \le A(u-1) - B$。選 B≥c、A≥B+p₀，則子項代入後：

$$P(u) \le A[H(L-1) + (H-1)] - B(H+1) + cH \le A(u-1) - B.$$

因此 P(u)=O(u)。只計 cluster 分支的底層節點已需 Ω(u) 空間，故為 Θ(u)；逐節點建立與清空也需 Θ(u) 時間。此推導同時容許 H≠L。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

公開介面、重複鍵與空間取捨

情境	本節的完整規格
重複 INSERT(x)	先 MEMBER；存在就回傳 ALREADY_PRESENT
不存在的 DELETE(x)	先 MEMBER；不存在就回傳 NOT_FOUND
x<0 或 x≥u	在任何索引前拒絕
需要重複鍵	另維護 multiplicity；計數由 0↔1 時才改鍵集合
需要附帶資料	另定義 key→record 對應與所有權

多一次 MEMBER 仍是兩個 O(1+log log u) 的相加，不改漸近界。重複鍵的計數假設可在常數時間讀寫、且不溢位；不能直接把重複鍵丟入 INSERT-ABSENT。

完整樹適合可負擔 u 的情境。若改成按需配置與 hash table，需要重新分析空間、配置與期望時間；**不能直接沿用本節的確定性最壞界**。n 很小而 u 很大時，O(n) 空間的平衡樹通常更合理。

把攤銷、期望與最壞保證放回各自的前提

結構／操作	保證	主要前提
紅黑樹查詢／更新	最壞 O(log(n+2))	比較鍵；O(n) 空間
雜湊查找	期望 O(1+α)	宣告的雜湊機率模型；不支援排序鄰居
Fibonacci 遞減鍵值	攤銷 O(1)	已知 handle；同一序列勢能
完整 vEB 前驅／更新	最壞 O(1+log log u)	整數 word RAM；Θ(u) 空間／建立

若先建立完整 vEB 再做 q 次操作，總上界為 $O(u + q(1 + \log\log u))$；初始化不能被省略。一般鍵的比較模型、整數 universe 模型與隨機模型各有適用條件，沒有一張脫離模型的「最快資料結構」排名。

參考資料與署名

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing，個人手寫資料結構筆記：佇列、搜尋樹、紅黑樹、多路搜尋樹與堆積。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study